



Miku Inn

Documentacion Tecnica del Backend

Java + Javalin | UNIS 2026

Generado el 9 de abril de 2026



Indice de contenidos

GENERAL

- Inicio

API

- Controllers
- Services
- Repositories

ESTRUCTURA

- Models
- DTO
- Helpers

INFRAESTRUCTURA

- Config
- Data

GENERAL

Inicio

Backend Java/Javalin para la cadena hotelera Miku Inn. Proyecto final UNIS 2026.

Tecnologias

- Java / Javalin
- Oracle Database
- Arquitectura por capas: controllers, services, repositories

Estructura

- controllers - Endpoints REST
- services - Logica de negocio
- repositories - Acceso a datos
- models - Entidades
- dtos - Objetos de transferencia
- helpers - Utilidades
- config - Configuracion del servidor
- data - Conexion a base de datos

API

Controllers

AdminBusquedaController

Controller para el modulo de reportes de busquedas del panel de administracion. Expone endpoints para listar busquedas con filtros, obtener un resumen estadistico y exportar el reporte por correo. Solo accesible para usuarios con rol de administrador.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra las rutas del modulo de reportes de busquedas en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

AerolineaAdminController

Controller que registra las rutas HTTP de aerolineas aliadas para el panel de administracion. Expone endpoints exclusivamente para el rol Administrador (rol 2). Tambien expone el endpoint de usuarios webservice libres, usado al crear entidades.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de administracion de aerolineas en la aplicacion Javalin. Todas las rutas requieren rol Administrador (rol 2).

- Param app - instancia de Javalin donde se registran las rutas.

AerolineaWebserviceController

Controller que registra las rutas HTTP relacionadas con aerolineas aliadas para el portal webservice. Expone endpoints exclusivamente para el rol Webservice (rol 3).

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de aerolineas aliadas para el webservice en la aplicacion Javalin. Todas las rutas requieren rol Webservice (rol 3).

- Param app - instancia de Javalin donde se registran las rutas.

AgenciaController

Controller que registra las rutas HTTP relacionadas con agencias. Expone endpoints para el rol Webservice (rol 3) y para el rol Administrador (rol 2).

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de agencias en la aplicacion Javalin. Las rutas bajo /webservice requieren rol 3 y las de /admin requieren rol 2.

- Param app - instancia de Javalin donde se registran las rutas.

AuthController

Controller que maneja la autentificacion de usuarios. Expone endpoints para login y logout usando cookies HttpOnly como mecanismo de sesion.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra las rutas de autentificacion en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

BusquedaAerolineaController

Controller que expone el endpoint de busqueda para aerolineas aliadas. Las peticiones se autentican mediante el header X-Aerolinea-Token.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de busqueda de aerolineas en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

BusquedaAgenciaController

Controller que expone el endpoint de busqueda para agencias externas. Las peticiones se autentican mediante el header X-Agencia-Token.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de busqueda de agencias en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

BusquedaController

Controller que expone el endpoint publico de busqueda de vuelos. Si el usuario tiene sesion activa, asocia el usuarioid a la busqueda; de lo contrario la procesa de forma anonima.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de busqueda en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

CancelacionAgenciaController

Controller que expone los endpoints de cancelacion de reservaciones para agencias externas. Todas las rutas requieren autenticacion mediante el header X-Agencia-Token.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra las rutas de cancelacion de agencias en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

CancelacionController

Controller que expone el endpoint de cancelacion de reservaciones para usuarios autenticados. Requiere sesion activa; el usuarioid se obtiene del contexto inyectado por el middleware JWT.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de cancelacion en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

ComentarioController

Controller que gestiona los endpoints relacionados con comentarios de hoteles. Expone rutas publicas, rutas protegidas por sesion de usuario y rutas para agencias externas.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de comentarios en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

DestinosController

Controller que expone el endpoint publico de destinos disponibles. No requiere autenticacion; retorna hoteles activos con sus imagenes.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de destinos en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

DownsController

Controller que gestiona las valoraciones negativas (downs) sobre comentarios. Todas las rutas requieren sesion activa; el usuarioid se obtiene del contexto JWT.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de downs en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

EmailReservacionController

Controller que gestiona el envio de correos electronicos relacionados con reservaciones, formulario de contacto y suscripciones al boletin informativo.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de correo en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

HandshakeAerolineaController

Controller que expone el endpoint publico de handshake para aerolineas aliadas. Permite a una aerolinea externa autenticarse ante el sistema hotelero presentando su URL y un token de entrada, y recibir un token de sesion para sus comunicaciones posteriores. El endpoint no requiere autenticacion previa ya que es el primer punto de contacto.

```
CODIGO  
public void registerRoutes(Javalin app)
```

Registra las rutas publicas del handshake de aerolineas en el servidor Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

HotelAgenciaController

Controller que expone el endpoint de consulta de hoteles para agencias externas. Requiere autenticacion mediante el header X-Agencia-Token.

```
CODIGO  
public void registrarRutas(Javalin app)
```

Registra la ruta de hoteles para agencias en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

HotelController

Controller que centraliza la administracion de hoteles, habitaciones, amenidades, imagenes, reservaciones y metricas del sistema. Todas las rutas requieren rol Administrador (rol 2).

```
CODIGO  
public void registerRoutes(Javalin app)
```

Registra todas las rutas administrativas en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

ImagenController

Controller que expone los endpoints publicos de descarga de imagenes. Sirve imagenes de hoteles, habitaciones y amenidades en formato JPEG.

```
CODIGO  
public void registerRoutes(Javalin app)
```

Registra las rutas de imagenes en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

PagoAgenciaController

Controller que gestiona el procesamiento de pagos de reservaciones realizadas por agencias externas. Requiere autenticacion mediante el header X-Agencia-Token.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de pago de agencias en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

PagoController

Controller que gestiona el procesamiento de pagos de reservaciones para usuarios autenticados. Requiere sesion activa; el usuariold se obtiene del contexto inyectado por el middleware JWT.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de pago en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

PdfReservacionController

Controller que gestiona la descarga del comprobante PDF de una reservacion. Requiere sesion activa; el usuariold se obtiene del contexto inyectado por el middleware JWT.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de descarga de PDF en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

ReservacionAgenciaController

Controller que gestiona las reservaciones realizadas por agencias externas. Todas las rutas requieren autenticacion mediante el header X-Agencia-Token.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de reservaciones de agencias en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

ReservacionController

Controller que gestiona las reservaciones de usuarios autenticados. Requiere sesion activa; el usuariold se obtiene del contexto inyectado por el middleware JWT.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra las rutas de reservaciones en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

SesionController

Controller que expone el endpoint de consulta del estado de sesion actual. Es publico para que el frontend pueda verificar la sesion sin importar si hay token o no.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de sesion en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registra la ruta.

TokenAerolineaController

Controller que expone el endpoint de generacion de tokens de alianza. Solo aerolineas autenticadas mediante X-Aerolinea-Token pueden acceder.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de generacion de tokens en la aplicacion Javalin. <p>Endpoint: POST /aerolinea/token</p> <p>Header requerido: X-Aerolinea-Token</p> <p>Body esperado: { "ciudad": "Paris", "pais": "Francia" }</p> <p>Respuesta exitosa 201: token generado, URL de redireccion y fecha de expiracion.</p>

- Param app - instancia de Javalin donde se registra la ruta.

TokenValidacionController

Controller que expone el endpoint de validacion de tokens de alianza. Requiere sesion activa del usuario; el token de alianza se recibe como query parameter en la URL.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra la ruta de validacion de tokens en la aplicacion Javalin. <p>Endpoint: GET /alianza/validar?token=uuid</p> <p>Requiere JWT activo en el header Authorization.</p> <p>Respuesta exitosa 200: ciudad, pais, porcentaje de descuento y fecha de expiracion.</p> <p>Respuesta 400: si el token no existe, ya fue usado o expiro.</p>

- Param app - instancia de Javalin donde se registra la ruta.

UsuarioController

Controller que gestiona las operaciones sobre usuarios del sistema. Expone rutas publicas de registro y validacion, rutas privadas para el perfil del usuario autenticado, y rutas administrativas exclusivas para rol 2.

CODIGO

```
public void registerRoutes(Javalin app)
```

Registra todas las rutas de usuarios en la aplicacion Javalin.

- Param app - instancia de Javalin donde se registran las rutas.

Services

AdminBusquedaService

Service para el modulo de busquedas del panel de administracion. Maneja listado paginado, resumen para dashboard y exportacion por correo.

CODIGO

```
public Map<String, Object> listar(
```

Retorna una pagina de busquedas aplicando los filtros dados.

- Param destino - nombre del destino a filtrar, o null para ignorar.
- Param usuarioAgencia - nombre de usuario o agencia a filtrar, o null para ignorar.
- Param tipo - "web", "rest" o "todos".
- Param fechaDesdeStr - fecha de inicio en formato ISO (yyyy-MM-dd), o null.
- Param fechaHastaStr - fecha de fin en formato ISO (yyyy-MM-dd), o null.
- Param pagina - numero de pagina (base 1).
- Param porPagina - cantidad de registros por pagina.
- Returns - mapa con "busquedas" (lista) y "total" (int).

CODIGO

```
public Map<String, Object> resumen()
```

Retorna un resumen de busquedas para el dashboard de admin. Incluye totales por tipo, busquedas por dia y top destinos.

- Returns - mapa con totalWeb, totalRest, porDia y topDestinos.

CODIGO

```
public void exportar(String email, String destino, String usuarioAgencia,
```

Genera un reporte HTML con las busquedas filtradas y lo envia por correo.

- Param email - destinatario del reporte.
- Param destino - filtro de destino, o null.
- Param usuarioAgencia - filtro de usuario o agencia, o null.
- Param tipo - "web", "rest" o "todos".
- Param fechaDesdeStr - fecha de inicio, o null.
- Param fechaHastaStr - fecha de fin, o null.

AdminReservacionService

Service para la gestion de reservaciones desde el panel de administracion. Permite listar todas las reservaciones y cancelarlas con validacion de estado.

CODIGO

```
public List<Map<String, Object>> listarTodas()
```

Retorna todas las reservaciones registradas en el sistema.

- Returns - lista de mapas con los datos de cada reservacion.

CODIGO

```
public void cancelarReservacion(int reservacionId, String motivo)
```

Cancela una reservacion si su estado actual lo permite. Solo se pueden cancelar reservaciones en estado Pendiente (1) o Confirmada (2).

- Param reservacionId - ID de la reservacion a cancelar.
- Param motivo - razon de la cancelacion.
- Throws IllegalArgumentException - si la reservacion no existe o su estado no permite cancelacion.

AerolineaAdminService

Service para la gestion de aerolineas aliadas desde el panel de administracion. Cubre operaciones de listado, creacion, edicion y consulta de usuarios libres.

CODIGO

```
public List<AerolineaAdminDTO> listarTodas()
```

Retorna todas las aerolineas aliadas registradas en el sistema.

- Returns - lista completa de aerolineas.

CODIGO

```
public void editar(int aerolineaId, EditarAerolineaRequestDTO req)
```

Edita los datos de una aerolinea existente.

- Param aerolineaId - ID de la aerolinea a editar.
- Param req - datos actualizados de la aerolinea.

CODIGO

```
public List<UsuarioWebserviceLibreDTO> listarWebserviceLibres()
```

Retorna los usuarios webservice disponibles para ser asignados a una entidad. Un usuario se considera libre si no tiene ni agencia ni aerolinea registrada.

- Returns - lista de usuarios webservice sin entidad asignada.

AerolineaWebserviceService

Service para la gestion de aerolineas aliadas desde el portal webservice. Cubre operaciones del usuario webservice dueño de la aerolinea: consulta, registro y cambio de estado.

```
CODIGO  
public List<AerolineaWebserviceDTO> listarPorUsuario(int usuarioId)
```

Retorna las aerolineas aliadas asociadas a un usuario webservice especifico.

- Param usuarioId - ID del usuario webservice propietario de las aerolineas.
- Returns - lista de aerolineas del usuario.

```
CODIGO  
public AerolineaWebserviceDTO crear(int usuarioId, CrearAerolineaRequestDTO req)
```

Registra una nueva aerolinea aliada vinculada al usuario dado.

- Param usuarioId - ID del usuario webservice que crea la aerolinea.
- Param req - datos de la nueva aerolinea.
- Returns - DTO con los datos de la aerolinea creada.

```
CODIGO  
public void cambiarEstado(int aerolineaId, int usuarioId, int nuevoEstadoId)
```

Cambia el estado de una aerolinea entre Activo (1) y Cerrado (2).

- Param aerolineaId - ID de la aerolinea a modificar.
- Param usuarioId - ID del usuario webservice dueño de la aerolinea.
- Param nuevoEstadoId - nuevo estado: 1 para Activo, 2 para Cerrado.
- Throws IllegalArgumentException - si el estado no es 1 ni 2.

AgenciaService

Service para la gestion de agencias de viaje. Cubre operaciones del webservice (usuario dueño) y del panel de administracion.

```
CODIGO  
public List<AgenciaDTO> listarPorUsuario(int usuarioId)
```

Retorna las agencias asociadas a un usuario especifico.

- Param usuarioId - ID del usuario dueño de las agencias.
- Returns - lista de agencias del usuario.

```
CODIGO  
public AgenciaDTO crear(int usuarioId, CrearAgenciaRequestDTO req)
```

Crea una nueva agencia vinculada al usuario dado (flujo del portal webservice).

- Param usuariold - ID del usuario que crea la agencia.
- Param req - datos de la nueva agencia.
- Returns - DTO con los datos de la agencia creada.

CODIGO

```
public void cambiarEstado(int agenciaId, int usuarioId, int nuevoEstadoId)
```

Cambia el estado de una agencia entre Activo (1) y Cerrado (2).

- Param agenciaId - ID de la agencia a modificar.
- Param usuariold - ID del usuario webservice dueño de la agencia.
- Param nuevoEstadoId - nuevo estado: 1 para Activo, 2 para Cerrado.
- Throws IllegalArgumentException - si el estado no es 1 ni 2.

CODIGO

```
public void eliminar(int agenciaId, int usuarioId)
```

Elimina una agencia del usuario dado.

- Param agenciaId - ID de la agencia a eliminar.
- Param usuariold - ID del usuario webservice dueño de la agencia.

CODIGO

```
public List<AgenciaDTO> listarTodas()
```

Retorna todas las agencias registradas en el sistema. Solo disponible para administradores.

- Returns - lista completa de agencias.

CODIGO

```
public void editar(int agenciaId, EditarAgenciaRequestDTO req)
```

Edita los datos de una agencia existente. Solo disponible para administradores.

- Param agenciaId - ID de la agencia a editar.
- Param req - datos actualizados de la agencia.

AuthService

Service de autenticacion de usuarios. Valida credenciales y genera el token JWT para la sesion.

CODIGO

```
public record LoginResultado(String token, LoginResponseDTO respuesta)
```

Agrupar el token JWT y los datos de respuesta tras un login exitoso.

- Param token - JWT generado para la sesion.
- Param respuesta - datos del usuario autenticado.

BusquedaAerolineaService

Service de busqueda de hoteles para aerolineas aliadas. Aplica el descuento de la aerolinea a los precios y calcula combinaciones de habitaciones para grupos grandes.

CODIGO

```
public List<HotelResultadoDTO> buscar(BusquedaRequestDTO request, String token)
```

Busca hoteles disponibles para una aerolinea autenticada por token. Valida que el token corresponda a una aerolinea activa, obtiene su descuento, guarda la busqueda y retorna los hoteles con precios ya descontados.

- Param request - criterios de busqueda: ciudad, pais, fechas y cantidad de personas.
- Param token - token de acceso de la aerolinea aliada.
- Returns - lista de hoteles con tipos de habitacion, amenidades y combinaciones disponibles.
- Throws IllegalArgumentException - si el token es invalido o la ciudad no existe.

BusquedaAgenciaService

Service de busqueda de hoteles para agencias de viaje. Aplica el descuento de la agencia a los precios y calcula combinaciones de habitaciones para grupos grandes.

CODIGO

```
public List<HotelResultadoDTO> buscar(BusquedaRequestDTO request, int usuarioId)
```

Busca hoteles disponibles para una agencia autenticada por sesion de usuario. Valida que el usuario tenga una agencia activa, obtiene su descuento, guarda la busqueda y retorna los hoteles con precios ya descontados.

- Param request - criterios de busqueda: ciudad, pais, fechas y cantidad de personas.
- Param usuarioId - ID del usuario dueño de la agencia.
- Returns - lista de hoteles con tipos de habitacion, amenidades y combinaciones disponibles.
- Throws IllegalArgumentException - si el usuario no tiene agencia activa o la ciudad no existe.

CODIGO

```
public List<HotelResultadoDTO> buscarPorToken(BusquedaRequestDTO request, String token)
```

Busca hoteles disponibles para una agencia autenticada por token de API. Funciona igual que buscar() pero valida la agencia con un token en lugar de sesion de usuario. La busqueda se guarda sin vincular a ningun usuario.

- Param request - criterios de busqueda: ciudad, pais, fechas y cantidad de personas.
- Param token - token de acceso de la agencia.
- Returns - lista de hoteles con tipos de habitacion, amenidades y combinaciones disponibles.
- Throws IllegalArgumentException - si el token es invalido, la agencia no esta activa o la ciudad no existe.

BusquedaService

Service de búsqueda de hoteles para usuarios web. Localiza hoteles por ciudad, guarda el registro de búsqueda y calcula combinaciones de habitaciones para grupos grandes.

CODIGO

```
public List<HotelResultadoDTO> buscar(BusquedaRequestDTO request, Integer usuarioId)
```

Busca hoteles disponibles segun los criterios del request. Valida que la ciudad exista, guarda la búsqueda y enriquece cada hotel con imagenes, amenidades, tipos de habitacion y combinaciones.

- Param request - criterios: ciudad, pais, fechas y cantidad de personas.
- Param usuariold - ID del usuario que realiza la búsqueda, puede ser null si no hay sesion.
- Returns - lista de hoteles con toda la informacion necesaria para mostrar resultados.
- Throws IllegalArgumentException - si la ciudad no existe en la base de datos.

CancelacionService

Service para cancelacion de reservaciones. Maneja cancelaciones de usuarios web y de agencias, validando estado y la regla de las 24 horas antes del check-in.

CODIGO

```
public void cancelarReservacion(int reservacionId, int usuarioId, String motivoCancelacion)
```

Cancela una reservacion de un usuario web. Verifica que la reservacion exista, pertenezca al usuario, tenga un estado valido y cumpla la regla de las 24 horas. o faltan menos de 24 horas para el check-in.

- Param reservacionId - ID de la reservacion a cancelar.
- Param usuariold - ID del usuario dueno de la reservacion.
- Param motivoCancelacion - razon de la cancelacion.
- Throws IllegalArgumentException - si la reservacion no existe, el estado no lo permite

CODIGO

```
public PuedeCancelarDTO puedeCancelar(int reservacionId, int agenciaId)
```

Verifica si una reservacion de agencia puede cancelarse sin ejecutar la cancelacion. Util para que el frontend consulte antes de mostrar el boton de cancelar.

- Param reservacionId - ID de la reservacion a evaluar.
- Param agenciaId - ID de la agencia duena de la reservacion.
- Returns - DTO con el resultado (puede o no cancelar) y un mensaje explicativo.

CODIGO

```
public void cancelarReservacionAgencia(int reservacionId, int agenciaId, String motivo)
```

Cancela una reservacion perteneciente a una agencia. Aplica las mismas validaciones de estado y plazo que cancelarReservacion. o faltan menos de 24 horas para el check-in.

- Param reservacionId - ID de la reservacion a cancelar.
- Param agenciaId - ID de la agencia duena de la reservacion.

- Param motivo - razon de la cancelacion.
- Throws IllegalArgumentException - si la reservacion no existe, el estado no lo permite

ComentarioService

Service para la gestion de comentarios y resenas de hoteles. Diferencia entre comentarios de hotel (con resena) y respuestas a comentarios (sin resena).

CODIGO

```
public ComentarioResponseDTO agregarComentario(ComentarioRequestDTO request, int usuarioId)
```

Agrega un comentario o respuesta segun el contenido del request. Si tiene comentarioPadreId es una respuesta, si no es un comentario de hotel con resena. Valida contenido, resena y que el usuario no haya comentado ya en ese hotel. o el usuario ya tiene resena en ese hotel.

- Param request - datos del comentario: contenido, resena, hotelId y comentarioPadreId opcional.
- Param usuarioId - ID del usuario que escribe el comentario.
- Returns - DTO con los datos del comentario recién creado.
- Throws IllegalArgumentException - si el contenido es invalido, la resena esta fuera de rango

CODIGO

```
public List<ComentarioResponseDTO> obtenerComentariosPorUsuario(int usuarioId)
```

Retorna todos los comentarios escritos por un usuario especifico.

- Param usuarioId - ID del usuario.
- Returns - lista de comentarios del usuario.

CODIGO

```
public List<ComentarioResponseDTO> obtenerComentariosPorHotel(int hotelId)
```

Retorna todos los comentarios de un hotel especifico.

- Param hotelId - ID del hotel.
- Returns - lista de comentarios del hotel.

DestinosService

Service para obtener el listado de destinos disponibles. Retorna todos los hoteles con sus imagenes para la pagina de destinos.

CODIGO

```
public List<HotelResultadoDTO> obtenerDestinos()
```

Obtiene todos los hoteles disponibles como destinos y les asigna sus imagenes.

- Returns - lista de hoteles con sus IDs de imagenes cargados.

DownsService

Service para la gestion de downs (votos) en comentarios. Un down puede ser positivo (1) o negativo (-1) y cada usuario solo puede tener un down por comentario.

CODIGO

```
public void agregarDown(int comentarioId, int usuarioId, int valor)
```

Agrega un down de un usuario a un comentario. Valida que el valor sea 1 o -1, que el comentario exista y que el usuario no haya votado ya en ese comentario. o el usuario ya tiene un down en ese comentario.

- Param comentarioId - ID del comentario a votar.
- Param usuarioId - ID del usuario que vota.
- Param valor - 1 para voto positivo, -1 para voto negativo.
- Throws IllegalArgumentException - si el valor es invalido, el comentario no existe

CODIGO

```
public void eliminarDown(int comentarioId, int usuarioId)
```

Elimina el down de un usuario en un comentario y ajusta el contador.

- Param comentarioId - ID del comentario.
- Param usuarioId - ID del usuario cuyo down se va a eliminar.
- Throws IllegalArgumentException - si el usuario no tiene down en ese comentario.

CODIGO

```
public void actualizarDown(int comentarioId, int usuarioId, int nuevoValor)
```

Cambia el valor del down existente de un usuario en un comentario. Elimina el voto anterior e inserta el nuevo ajustando el contador en ambos pasos. o el nuevo valor es igual al actual.

- Param comentarioId - ID del comentario.
- Param usuarioId - ID del usuario que actualiza su voto.
- Param nuevoValor - nuevo valor: 1 o -1.
- Throws IllegalArgumentException - si el valor es invalido, el usuario no tiene down

CODIGO

```
public List<DownResponseDTO> obtenerDownsDeUsuario(int usuarioId)
```

Retorna todos los downs registrados por un usuario.

- Param usuarioId - ID del usuario.
- Returns - lista de downs del usuario.

CODIGO

```
public List<DownResponseDTO> obtenerDownsDeUsuarioPorHotel(int usuarioId, int hotelId)
```

Retorna los downs de un usuario filtrados por hotel. Util para saber como voto el usuario en los comentarios de un hotel especifico.

- Param usuariold - ID del usuario.
- Param hotelId - ID del hotel a filtrar.
- Returns - lista de downs del usuario en comentarios de ese hotel.

EmailReservacionService

Service para enviar el resumen de una reservacion por correo electronico. Construye un HTML con los detalles de la reservacion y lo envia al correo del usuario.

CODIGO

```
public void enviarCorreoReservacion(int reservacionId, int usuarioId)
```

Envia el correo de confirmacion de una reservacion al usuario. Valida que la reservacion pertenezca al usuario, obtiene su correo y construye el HTML con los detalles y factura antes de enviarlo. no se encuentra el correo o la reservacion no tiene detalles.

- Param reservacionId - ID de la reservacion a enviar.
- Param usuariold - ID del usuario destinatario.
- Throws IllegalArgumentException - si la reservacion no pertenece al usuario,

ExpiracionService

Servicio en segundo plano que expira reservaciones pendientes vencidas. Corre en un hilo separado cada minuto desde que arranca el servidor.

CODIGO

```
public void iniciar()
```

Arranca el hilo programado que revisa y expira reservaciones cada minuto. El primer ciclo inicia un minuto despues de llamar a este metodo.

CODIGO

```
public void detener()
```

Detiene el hilo del scheduler al apagar el servidor. Se llama desde el ShutdownHook registrado en Main.

HandshakeAerolineaService

Service para el proceso de handshake entre el sistema hotelero y una aerolinea aliada externa. Valida la URL de la aerolinea registrada en la tabla AerolineaAliado, genera un token de salida y persiste ambos tokens para que la aerolinea pueda autenticarse en llamadas futuras.

CODIGO

```
public HandshakeResponseDTO procesarHandshake(HandshakeRequestDTO dto)
```

Procesa el handshake de una aerolinea aliada externa. Busca la aerolinea por su URL registrada en la base de datos, genera un token de respuesta y guarda el token de entrada y el de salida vinculados a ese registro. o si los tokens no se pudieron persistir en la base de datos.

- Param dto - datos del handshake con la URL de la aerolinea y el token de entrada.
- Returns - DTO con el token de salida generado para que la aerolinea lo use en requests futuros.
- Throws IllegalArgumentException - si no existe una aerolinea registrada con esa URL

HandshakeService

Service para el proceso de handshake entre el sistema y una agencia externa. Valida la URL de la agencia, genera un token de salida y persiste ambos tokens.

CODIGO

```
public HandshakeResponseDTO procesarHandshake(HandshakeRequestDTO dto)
```

Procesa el handshake de una agencia externa. Busca la agencia por su URL, genera un token de respuesta y guarda el token de entrada y el de salida en la base de datos.

- Param dto - datos del handshake con la URL de la agencia y el token de entrada.
- Returns - DTO con el token de salida generado para que la agencia lo use en requests futuros.
- Throws IllegalArgumentException - si no existe una agencia con esa URL o si los tokens no se pudieron guardar.

HotelAgenciaService

Service para exponer informacion de hoteles al modulo de agencias.

CODIGO

```
public List<HotelAgenciaDTO> obtenerHotelesParaAgencia()
```

Retorna la lista de hoteles disponibles para ser consultados por agencias.

- Returns - lista de hoteles con los datos relevantes para el contexto de agencia.

HotelService

Service principal para la gestion de hoteles desde el panel de administracion. Cubre catalogos, hoteles, amenidades, habitaciones, imagenes y metricas.

CODIGO

```
public List<AmenidadDTO> listarAmenidades()
```

Retorna todas las amenidades disponibles en el catalogo.

- Returns - lista de amenidades.

```
CODIGO  
public List<PaisDTO> listarPaises()
```

Retorna todos los paises registrados en el sistema.

- Returns - lista de paises.

```
CODIGO  
public List<CiudadDTO> listarCiudades()
```

Retorna todas las ciudades registradas en el sistema.

- Returns - lista de ciudades.

```
CODIGO  
public Map<String, Object> crearAmenidad(String nombre)
```

Crea una nueva amenidad en el catalogo.

- Param nombre - nombre de la amenidad, no puede ser nulo ni vacio.
- Returns - mapa con el ID, nombre y mensaje de confirmacion.
- Throws IllegalArgumentException - si el nombre es nulo o vacio.

```
CODIGO  
public List<HotelAdminDTO> listarTodos()
```

Retorna todos los hoteles con su cantidad de habitaciones e IDs de imagenes.

- Returns - lista de hoteles con datos enriquecidos para el panel de admin.

```
CODIGO  
public Map<String, Object> crearHotel(CrearHotelRequestDTO req)
```

Crea un nuevo hotel, buscando o creando el pais y ciudad si no existen.

- Param req - datos del hotel: nombre, rating, estado, ciudad y pais.
- Returns - mapa con el ID del hotel creado y mensaje de confirmacion.
- Throws IllegalArgumentException - si algun campo obligatorio es invalido.

```
CODIGO  
public void editarHotel(int hotelId, EditarHotelRequestDTO req)
```

Edita los datos de un hotel existente.

- Param hotelId - ID del hotel a editar.
- Param req - datos actualizados del hotel.

- Throws `IllegalArgumentException` - si el hotel no existe o los datos son invalidos.

```
CODIGO  
public void eliminarHotel(int hotelId)
```

Elimina un hotel del sistema.

- Param `hotelId` - ID del hotel a eliminar.
- Throws `IllegalArgumentException` - si el hotel no existe.

```
CODIGO  
public List<HotelAmenidadDTO> listarAmenidadesHotel(int hotelId)
```

Retorna las amenidades de un hotel con sus IDs de imagenes.

- Param `hotelId` - ID del hotel.
- Returns - lista de amenidades del hotel.
- Throws `IllegalArgumentException` - si el hotel no existe.

```
CODIGO  
public Map<String, Object> agregarAmenidadHotel(int hotelId, AgregarAmenidadRequestDTO req)
```

Agrega una amenidad del catalogo a un hotel. Valida que el hotel exista y que no tenga ya esa amenidad asignada. o el hotel ya tiene esa amenidad.

- Param `hotelId` - ID del hotel.
- Param `req` - datos con el ID de amenidad y descripcion opcional.
- Returns - mapa con el ID del registro creado y mensaje de confirmacion.
- Throws `IllegalArgumentException` - si el hotel no existe, la amenidad es invalida

```
CODIGO  
public void actualizarAmenidadHotel(int hotelAmenidadId, AgregarAmenidadRequestDTO req)
```

Actualiza la descripcion de una amenidad asignada a un hotel.

- Param `hotelAmenidadId` - ID del registro hotel-amenidad.
- Param `req` - datos con la nueva descripcion.

```
CODIGO  
public void eliminarAmenidadHotel(int hotelAmenidadId)
```

Elimina una amenidad asignada a un hotel.

- Param `hotelAmenidadId` - ID del registro hotel-amenidad a eliminar.

```
CODIGO  
public Map<String, Object> agregarImagenAmenidad(int hotelAmenidadId, String base64)
```

Agrega una imagen a una amenidad de hotel decodificando el base64 recibido.

- Param hotelAmenidadId - ID del registro hotel-amenidad.
- Param base64 - imagen codificada en base64, con o sin prefijo data URI.
- Returns - mapa con el ID de la imagen creada y mensaje de confirmacion.

CODIGO

```
public void eliminarImagenAmenidad(int imagenId)
```

Elimina una imagen de amenidad por su ID.

- Param imagenId - ID de la imagen a eliminar.

CODIGO

```
public Map<String, Object> agregarImagenHotel(int hotelId, String base64)
```

Agrega una imagen al hotel decodificando el base64 recibido.

- Param hotelId - ID del hotel.
- Param base64 - imagen codificada en base64.
- Returns - mapa con el ID de la imagen creada y mensaje de confirmacion.
- Throws IllegalArgumentException - si el hotel no existe.

CODIGO

```
public void eliminarImagenHotel(int imagenId)
```

Elimina una imagen de hotel por su ID.

- Param imagenId - ID de la imagen a eliminar.

CODIGO

```
public List<HabitacionAdminDTO> listarHabitaciones(int hotelId)
```

Retorna las habitaciones de un hotel con sus IDs de imagenes.

- Param hotelId - ID del hotel.
- Returns - lista de habitaciones con datos para el panel de admin.
- Throws IllegalArgumentException - si el hotel no existe.

CODIGO

```
public Map<String, Object> crearHabitacion(CrearHabitacionRequestDTO req)
```

Crea una nueva habitacion en un hotel. El numero de habitacion se auto-asigna en el repositorio.

- Param req - datos de la habitacion: hotelId, tipo, descripcion y estado.
- Returns - mapa con el ID de la habitacion creada y mensaje de confirmacion.
- Throws IllegalArgumentException - si el hotel no existe, el tipo es invalido o el estado no es valido.

CODIGO

```
public void editarHabitacion(int habitacionId, EditarHabitacionRequestDTO req)
```

Edita los datos de una habitacion existente.

- Param habitacionId - ID de la habitacion a editar.
- Param req - datos actualizados de la habitacion.
- Throws IllegalArgumentException - si la habitacion no existe, el tipo es invalido o el estado no es valido.

CODIGO

```
public void eliminarHabitacion(int habitacionId)
```

Elimina una habitacion del sistema.

- Param habitacionId - ID de la habitacion a eliminar.
- Throws IllegalArgumentException - si la habitacion no existe.

CODIGO

```
public Map<String, Object> agregarImagenHabitacion(int habitacionId, String base64)
```

Agrega una imagen a una habitacion decodificando el base64 recibido.

- Param habitacionId - ID de la habitacion.
- Param base64 - imagen codificada en base64.
- Returns - mapa con el ID de la imagen creada y mensaje de confirmacion.
- Throws IllegalArgumentException - si la habitacion no existe.

CODIGO

```
public void eliminarImagenHabitacion(int imagenId)
```

Elimina una imagen de habitacion por su ID.

- Param imagenId - ID de la imagen a eliminar.

CODIGO

```
public List<Map<String, Object>> listarTodasReservaciones()
```

Retorna todas las reservaciones del sistema para el panel de admin.

- Returns - lista de reservaciones con sus datos principales.

CODIGO

```
public Map<String, Object> obtenerMetricas()
```

Retorna metricas generales del sistema: reservaciones, ingresos, ocupacion, etc.

- Returns - mapa con los valores de cada metrica.

ImagenService

Service para recuperar imagenes almacenadas en la base de datos. Sirve los bytes de imagenes de hoteles, habitaciones y amenidades.

```
CODIGO  
public byte[] obtenerImagenHotel(int id)
```

Retorna los bytes de una imagen de hotel.

- Param id - ID de la imagen.
- Returns - array de bytes de la imagen.

```
CODIGO  
public byte[] obtenerImagenHabitacion(int id)
```

Retorna los bytes de una imagen de habitacion.

- Param id - ID de la imagen.
- Returns - array de bytes de la imagen.

```
CODIGO  
public byte[] obtenerImagenAmenidad(int id)
```

Retorna los bytes de una imagen de amenidad.

- Param id - ID de la imagen.
- Returns - array de bytes de la imagen.

PagoAgenciaService

Service para procesar pagos de reservaciones realizadas por agencias. Valida los datos de facturacion, confirma la reservacion y genera la factura.

```
CODIGO  
public PagoResponseDTO procesarPago(int reservacionId, int agenciaId, PagoAgenciaRequestDTO request)
```

Procesa el pago de una reservacion de agencia. Valida NIT y codigo postal, verifica que la reservacion pertenezca a la agencia y este en estado Pendiente, luego la confirma y genera la factura. o su estado no permite el pago.

- Param reservacionId - ID de la reservacion a pagar.
- Param agenciaId - ID de la agencia duena de la reservacion.
- Param request - datos de facturacion: NIT y codigo postal.
- Returns - DTO con los datos de la factura generada.
- Throws IllegalArgumentException - si los datos son invalidos, la reservacion no existe

PagoService

Service para procesar pagos de reservaciones de usuarios web. Valida la reservacion, simula la verificacion de tarjeta y genera la factura.

CODIGO

```
public PagoResponseDTO procesarPago(int reservacionId, int usuarioId, PagoRequestDTO request)
```

Procesa el pago de una reservacion de usuario web. Verifica que la reservacion exista y este pendiente, valida la tarjeta en memoria, confirma la reservacion y genera la factura. Si se incluye un token de alianza, valida que el hotel de la reservacion se encuentre en la misma ciudad para la que fue generado el token antes de aplicar el descuento. su estado no permite el pago, el token es invalido o el token no aplica para la ciudad del hotel.

- Param reservacionId - ID de la reservacion a pagar.
- Param usuarioId - ID del usuario dueño de la reservacion.
- Param request - datos de pago: tarjeta, NIT, codigo postal y token opcional.
- Returns - DTO con los datos de la factura generada.
- Throws IllegalArgumentException - si la reservacion no existe, no pertenece al usuario,

PdfReservacionService

Service para la generacion de PDFs de reservaciones. Valida que la reservacion pertenezca al usuario y delega la construccion del PDF a PdfHelper.

CODIGO

```
public byte[] generarPdf(int reservacionId, int usuarioId)
```

Genera el PDF de una reservacion para un usuario especifico. Verifica que la reservacion pertenezca al usuario, obtiene los detalles y la factura, y construye el PDF con esos datos. o no tiene detalles registrados.

- Param reservacionId - ID de la reservacion a exportar.
- Param usuarioId - ID del usuario dueño de la reservacion.
- Returns - array de bytes del PDF generado.
- Throws IllegalArgumentException - si la reservacion no pertenece al usuario

ReservacionAgenciaService

Service para la gestion de reservaciones realizadas por agencias. Maneja la creacion con descuento aplicado, consulta de reservaciones y expiracion manual de reservaciones pendientes.

CODIGO

```
public ReservacionAgenciaResponseDTO crearReservacion(ReservacionRequestDTO request, int agenciaId)
```

Crea una nueva reservacion para una agencia aplicando su descuento. Valida disponibilidad de cada habitacion, calcula totales con el descuento de la agencia, genera el numero de reservacion y persiste los detalles. La reservacion expira automaticamente en 15 minutos si no se paga. las fechas son invalidas o alguna habitacion no esta disponible.

- Param request - datos de la reservacion: lista de habitaciones con fechas y personas.
- Param agenciaId - ID de la agencia que realiza la reservacion.
- Returns - DTO con los datos de la reservacion creada y el desglose por habitacion.
- Throws IllegalArgumentException - si la agencia no esta activa, no hay habitaciones,

CODIGO

```
public List<ReservacionDetalleDTO> obtenerReservaciones(int agenciaId)
```

Retorna todas las reservaciones asociadas a una agencia.

- Param agenciaId - ID de la agencia.
- Returns - lista de reservaciones con sus detalles.

CODIGO

```
public void expirarReservacion(int reservacionId, int agenciaId)
```

Expira manualmente una reservacion pendiente de una agencia. Solo aplica si la reservacion pertenece a la agencia y esta en estado Pendiente. o no esta en estado pendiente.

- Param reservacionId - ID de la reservacion a expirar.
- Param agenciaId - ID de la agencia duena de la reservacion.
- Throws IllegalArgumentException - si la reservacion no existe, no pertenece a la agencia

CODIGO

```
public List<ReservacionDetalleDTO> obtenerDetalleReservacion(int reservacionId, int agenciaId)
```

Retorna el detalle completo de una reservacion de agencia con imagenes incluidas.

- Param reservacionId - ID de la reservacion.
- Param agenciaId - ID de la agencia duena de la reservacion.
- Returns - lista de DTOs con detalles de habitaciones, hotel e imagenes.
- Throws IllegalArgumentException - si la reservacion no existe o no pertenece a la agencia.

ReservacionService

Service para la gestion de reservaciones de habitaciones. Maneja la creacion de reservaciones con validacion de fechas, disponibilidad y calculo de precios, incluyendo cargos por personas extra.

CODIGO

```
public ReservacionResponseDTO crearReservacion(ReservacionRequestDTO request, int usuarioId)
```

Crea una nueva reservacion para el usuario indicado. Valida fechas, disponibilidad de habitaciones y capacidad maxima antes de persistir. El numero de reservacion se genera automaticamente con el prefijo MIKU-. La reservacion queda en estado pendiente y expira en 10 minutos si no se confirma. hay traslape de disponibilidad, o se excede la capacidad permitida.

- Param request - datos de la reservacion, incluyendo la lista de habitaciones solicitadas.
- Param usuariold - ID del usuario que realiza la reservacion.
- Returns - DTO con los datos de la reservacion creada.

- Throws `IllegalArgumentException` - si no se incluyen habitaciones, las fechas son invalidas,

CODIGO

```
public List<ReservacionDetalleDTO> obtenerReservaciones(int usuarioId)
```

Obtiene todas las reservaciones de un usuario con sus imagenes asociadas. Por cada reservacion se cargan las imagenes del hotel y de la habitacion correspondiente.

- Param `usuarioId` - ID del usuario del que se quieren obtener las reservaciones.
- Returns - lista de DTOs con el detalle de cada reservacion e imagenes incluidas.

SesionService

Service para la gestion de la sesion activa del usuario. Construye el estado de sesion a partir de los datos del token JWT ya validado por el middleware.

CODIGO

```
public SesionDTO obtenerSesion(int usuarioId, String username, int rolId)
```

Construye el DTO de sesion para un usuario autenticado. Los datos del usuario vienen del token ya validado por el middleware, y se complementa con el nombre del rol consultado en base de datos.

- Param `usuarioId` - ID del usuario autenticado.
- Param `username` - nombre de usuario extraido del token.
- Param `rolId` - ID del rol del usuario.
- Returns - DTO con los datos de sesion y autenticado en true.

TokenAerolineaService

Service encargado de generar tokens de alianza para aerolineas. Valida la identidad de la aerolinea, resuelve la ciudad destino y persiste el token con una ventana de expiracion de 15 minutos.

CODIGO

```
public TokenAerolineaResponseDTO generarToken(TokenAerolineaRequestDTO request, String tokenHash)
```

Genera y persiste un token de alianza para una aerolinea autenticada. El token expira a los 15 minutos y no tiene reservacion asociada todavia. La URL de redireccion se construye con el token para que la aerolinea pueda enviarsela directamente al usuario. o si la ciudad no existe en la base de datos.

- Param `request` - datos de la solicitud: ciudad y pais destino del pasajero.
- Param `tokenHash` - hash del token de la aerolinea enviado en el header.
- Returns - `TokenAerolineaResponseDTO` con el token generado, URL de redireccion y fecha de expiracion.
- Throws `IllegalArgumentException` - si el token no corresponde a una aerolinea activa

TokenValidacionService

Service encargado de validar tokens de alianza enviados por usuarios que llegan desde una aerolinea aliada.

```
CODIGO
public TokenValidacionResponseDTO validar(String token)
```

Valida un token de alianza y retorna los datos necesarios para configurar la búsqueda con descuento en el frontend. No marca el token como usado; eso ocurre al momento del pago.

- Param token - string UUID recibido desde la URL del usuario.
- Returns - TokenValidacionResponseDTO con ciudad, país, descuento y expiración.
- Throws IllegalArgumentException - si el token no existe, ya fue usado o expiro.

UsuarioService

Service para la gestión de usuarios del sistema. Cubre el registro, validación de campos únicos, consulta de perfil, actualización de datos personales y operaciones administrativas de rol.

```
CODIGO
public UsuarioValidacionResponseDTO validarDisponibilidad(UsuarioValidacionRequestDTO request)
```

Verifica si el username, correo o pasaporte ya están registrados en el sistema.

- Param request - DTO con los campos a validar.
- Returns - DTO indicando cuáles campos ya existen.

```
CODIGO
public int registrarUsuario(UsuarioValidacionRequestDTO request)
```

Registra un nuevo usuario en el sistema. Valida que no existan campos duplicados, hashea la contraseña, resuelve país y ciudad (creándolos si no existen), asigna nacionalidades y envía un correo de bienvenida. El correo no bloquea el registro si falla.

- Param request - datos del usuario a registrar.
- Returns - ID del usuario recién creado.
- Throws CamposDuplicadosException - si el username, correo o pasaporte ya están en uso.

```
CODIGO
public UsuarioPerfilResponseDTO obtenerPerfil(int usuarioId)
```

Retorna el perfil completo de un usuario incluyendo sus nacionalidades.

- Param usuarioId - ID del usuario a consultar.
- Returns - DTO con los datos del perfil y lista de nacionalidades.
- Throws RuntimeException - si el usuario no existe.

```
CODIGO
public void cambiarTelefono(int usuarioId, String nuevoTelefono)
```

Actualiza el numero de telefono de un usuario.

- Param usuariold - ID del usuario a actualizar.
- Param nuevoTelefono - nuevo numero de telefono; no puede ser nulo ni vacio.
- Throws IllegalArgumentException - si el telefono es nulo o esta en blanco.

CODIGO

```
public void cambiarContrasena(int usuarioId, String contrasenaActual, String contrasenaNueva)
```

Cambia la contraseña de un usuario previa verificación de la contraseña actual.

- Param usuariold - ID del usuario.
- Param contrasenaActual - contraseña actual en texto plano para verificación.
- Param contrasenaNueva - nueva contraseña en texto plano que será hasheada.
- Throws CredencialesInvalidasException - si la contraseña actual no coincide.

CODIGO

```
public List<UsuarioAdminDTO> listarTodosUsuarios()
```

Retorna todos los usuarios registrados junto con su rol asignado. Uso exclusivo del panel de administración.

- Returns - lista de DTOs con los datos de cada usuario y su rol.

CODIGO

```
public void cambiarRol(int usuarioId, int nuevoRolId)
```

Cambia el rol de un usuario. Solo se aceptan los roles definidos en el sistema.

- Param usuariold - ID del usuario al que se le cambiara el rol.
 - Param nuevoRolId - ID del nuevo rol: 1 (Usuario), 2 (Administrador) o 3 (Webservice).
 - Throws IllegalArgumentException - si el ID de rol no corresponde a ninguno de los roles validos.
-

Repositories

AdminBusquedaRepository

*Repository para consultas del modulo de reportes de busquedas en el panel de administracion.
Soporta filtros por destino, usuario o agencia, tipo de busqueda y rango de fechas.*

CODIGO

```
public List<Map<String, Object>> listar()
```

Retorna una pagina de busquedas aplicando los filtros indicados. Todos los parametros de filtro son opcionales; si son null o vacios se ignoran.

- Param destino - nombre de ciudad a filtrar con LIKE, o null para no filtrar.
- Param usuarioAgencia - username del usuario o nombre de la agencia a filtrar, o null.
- Param tipoBusquedaId - 1 para busquedas web, 2 para busquedas REST, null para todas.
- Param fechaDesde - fecha minima de la busqueda, o null para no filtrar.
- Param fechaHasta - fecha maxima de la busqueda, o null para no filtrar.
- Param offset - numero de filas a saltar para la paginacion.
- Param porPagina - cantidad de filas a retornar por pagina.
- Returns - lista de mapas con los datos de cada busqueda encontrada.

CODIGO

```
public int contar(String destino, String usuarioAgencia, Integer tipoBusquedaId,
```

Cuenta el total de busquedas que coinciden con los filtros indicados. Se usa para calcular el total de paginas en la paginacion.

- Param destino - nombre de ciudad a filtrar con LIKE, o null para no filtrar.
- Param usuarioAgencia - username del usuario o nombre de la agencia a filtrar, o null.
- Param tipoBusquedaId - 1 para busquedas web, 2 para busquedas REST, null para todas.
- Param fechaDesde - fecha minima de la busqueda, o null para no filtrar.
- Param fechaHasta - fecha maxima de la busqueda, o null para no filtrar.
- Returns - total de busquedas que coinciden con los filtros.

CODIGO

```
public int contarPorTipo(Integer tipoBusquedaId)
```

Cuenta el total de busquedas filtradas por tipo. Si tipoBusquedaId es null retorna el total sin filtro de tipo.

- Param tipoBusquedaId - 1 para web, 2 para REST, null para todas.
- Returns - total de busquedas del tipo indicado.

CODIGO

```
public List<Map<String, Object>> busquedasPorDia()
```

Retorna el conteo de busquedas agrupado por dia para los ultimos 30 dias. Cada elemento del resultado contiene la fecha y el total de busquedas de ese dia.

- Returns - lista de mapas con los campos "dia" (YYYY-MM-DD) y "total".

CODIGO

```
public List<Map<String, Object>> topDestinos()
```

Retorna los 10 destinos mas buscados ordenados por frecuencia descendente. Cada elemento contiene el nombre de la ciudad y el total de busquedas.

- Returns - lista de mapas con los campos "nombre" y "total".

CODIGO

```
public List<Map<String, Object>> exportar()
```

Retorna todas las busquedas que coinciden con los filtros sin paginacion. Se usa para generar el reporte completo a exportar por correo.

- Param destino - nombre de ciudad a filtrar con LIKE, o null para no filtrar.
- Param usuarioAgencia - username del usuario o nombre de la agencia a filtrar, o null.
- Param tipoBusquedaId - 1 para busquedas web, 2 para busquedas REST, null para todas.
- Param fechaDesde - fecha minima de la busqueda, o null para no filtrar.
- Param fechaHasta - fecha maxima de la busqueda, o null para no filtrar.
- Returns - lista de mapas con los datos de cada busqueda encontrada.

AdminReservacionRepository

Repository para la gestion de reservaciones desde el panel de administracion. Maneja la consulta de todas las reservaciones y la logica de cancelacion a nivel de base de datos.

CODIGO

```
public List<Map<String, Object>> listarTodas()
```

Retorna todas las reservaciones registradas en el sistema con sus datos completos. Usa subqueries escalares para garantizar una sola fila por reservacion, evitando duplicados cuando una reservacion tiene habitaciones en distintos hoteles.

- Returns - lista de mapas con los datos de cada reservacion, ordenadas por fecha de creacion descendente.

CODIGO

```
public Object[] obtenerReservacion(int reservacionId)
```

Busca una reservacion por su ID y retorna sus datos basicos de estado.

- Param reservacionId - ID de la reservacion a buscar.
- Returns - arreglo con {ID, EstadoID, Estado} o null si no existe.

CODIGO

```
public void cancelarReservacion(int reservacionId, String motivo)
```

Actualiza el estado de una reservacion a Cancelada (EstadoID = 4) y registra el motivo. Si el motivo es nulo o vacio, se guarda un texto por defecto.

- Param reservacionId - ID de la reservacion a cancelar.
- Param motivo - razon de la cancelacion ingresada por el administrador.

AerolineaAdminRepository

Repository para la gestion de aerolineas aliadas desde el panel de administracion. Cubre operaciones de listado, creacion y edicion para el rol Administrador (rol 2). Tambien expone la consulta de usuarios webservice libres (sin entidad asignada).

CODIGO

```
public List<AerolineaAdminDTO> listarTodas()
```

Retorna todas las aerolineas aliadas registradas en el sistema, incluyendo el username del usuario webservice propietario.

- Returns - lista completa de aerolineas con datos de su usuario webservice.

CODIGO

```
public void editar(int aerolineaId, EditarAerolineaRequestDTO req)
```

Actualiza los datos de una aerolinea existente desde el panel de administracion. Valida nombre, URLs, porcentaje de descuento y estado antes de aplicar los cambios.

- Param aerolineaId - ID de la aerolinea a editar.
- Param req - datos actualizados de la aerolinea.
- Throws IllegalArgumentException - si algun campo es invalido.

CODIGO

```
public List<UsuarioWebserviceLibreDTO> listarWebserviceLibres()
```

Retorna los usuarios con rol Webservice (rol 3) que no tienen ninguna entidad registrada (ni agencia ni aerolinea). Se usan para poblar el selector al crear una entidad desde el panel de administracion.

- Returns - lista de UsuarioWebserviceLibreDTO disponibles para asignacion.

AerolineaAliadaRepository

Repository para autentificacion y busqueda de hoteles desde el canal de aerolineas aliadas. Tambien gestiona el proceso de handshake con aerolineas externas, incluyendo la busqueda por URL y la persistencia de tokens de autentificacion.

CODIGO

```
public AerolineaIdentidadDTO obtenerAerolineaPorToken(String token)
```

Retorna la identidad de la aerolinea activa que corresponde al token dado.

- Param token - hash del token de la aerolinea aliada.
- Returns - AerolinealIdentidadDTO si existe y esta activa, null si no.

CODIGO

```
public Double obtenerDescuentoAerolinea(String token)
```

Retorna el porcentaje de descuento de la aerolinea activa identificada por su token.

- Param token - hash del token de la aerolinea aliada.
- Returns - porcentaje de descuento como Double, o null si no se encuentra.

CODIGO

```
public Integer buscarCiudadId(String nombreCiudad, String nombrePais)
```

Busca el ID de una ciudad comparando nombre de ciudad y pais sin distincion de mayusculas.

- Param nombreCiudad - nombre de la ciudad a buscar.
- Param nombrePais - nombre del pais al que pertenece la ciudad.
- Returns - ID de la ciudad, o null si no se encuentra.

CODIGO

```
public void guardarBusqueda(int ciudadId, Date fechaCheckIn, Date fechaCheckOut,
```

Registra una busqueda realizada desde una aerolinea aliada (TipoBusquedaID = 3). Se guarda sin usuario porque la sesion pertenece al sistema de la aerolinea.

- Param ciudadId - ID de la ciudad destino de la busqueda.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Param cantidadPersonas - numero de personas para la busqueda.

CODIGO

```
public List<HotelResultadoDTO> buscarHotelesPorCiudad(int ciudadId)
```

Busca hoteles activos en la ciudad indicada.

- Param ciudadId - ID de la ciudad a buscar.
- Returns - lista de HotelResultadoDTO con los hoteles disponibles.

CODIGO

```
public List<Integer> buscarImagenesHotel(int hotelId)
```

Retorna los IDs de las imagenes asociadas a un hotel.

- Param hotelId - ID del hotel.
- Returns - lista de IDs de imagenes.

CODIGO

```
public List<AmenidadHotelDTO> buscarAmenidadesHotel(int hotelId)
```

Retorna las amenidades registradas para un hotel con sus descripciones.

- Param hotelId - ID del hotel.
- Returns - lista de AmenidadHotelDTO con los datos de cada amenidad.

CODIGO

```
public List<Integer> buscarImagenesAmenidad(int hotelAmenidadId)
```

Retorna los IDs de las imagenes asociadas a una amenidad de hotel.

- Param hotelAmenidadId - ID del registro HotelAmenidad.
- Returns - lista de IDs de imagenes.

CODIGO

```
public List<TipoHabitacionResultadoDTO> buscarTiposHabitacionDisponibles(
```

Busca tipos de habitacion disponibles en un hotel segun fechas y capacidad minima.

- Param hotelId - ID del hotel.
- Param capacidadMinima - numero minimo de personas que debe aceptar el tipo de habitacion.
- Param fechaCheckIn - fecha de inicio de la estancia.
- Param fechaCheckOut - fecha de fin de la estancia.
- Returns - lista de TipoHabitacionResultadoDTO disponibles para el rango de fechas.

CODIGO

```
public List<HabitacionResumenDTO> buscarHabitacionesResumenPorTipo(
```

Busca habitaciones disponibles de un tipo especifico en un hotel y rango de fechas.

- Param hotelId - ID del hotel.
- Param tipoHabitacionId - ID del tipo de habitacion.
- Param fechaCheckIn - fecha de inicio de la estancia.
- Param fechaCheckOut - fecha de fin de la estancia.
- Returns - lista de HabitacionResumenDTO con ID y numero de habitacion disponibles.

CODIGO

```
public List<Integer> buscarImagenesHabitacion(int habitacionId)
```

Retorna los IDs de las imagenes asociadas a una habitacion.

- Param habitacionId - ID de la habitacion.
- Returns - lista de IDs de imagenes.

CODIGO

```
public Integer obtenerAerolineaIdPorURL(String urlAerolinea)
```

Busca el ID de una aerolinea aliada a partir de su URL registrada en la base de datos. Se usa durante el handshake para identificar a la aerolinea que se esta autenticando.

- Param urlAerolinea - URL unica asociada a la aerolinea aliada (campo URL en AerolineaAliado).
- Returns - ID de la aerolinea aliada, o null si no se encuentra ninguna con esa URL.

CODIGO

```
public boolean guardarTokensAerolinea(int aerolineaId, String tokenEntrada, String tokenSalida)
```

Persiste el token de sesion del handshake en el registro de la aerolinea aliada. Se guarda el token de SALIDA (generado por el hotel) y NO el de entrada. De esta forma ambas bases de datos quedan con el mismo token: - Oracle (hotel) : TokenHASH = tokenSalida (guardado aqui) - SQL Server (aerolinea): TokenHASH = tokenSalida (guardado por HandshakeHotelService) Cuando la aerolinea llame al hotel en requests futuros enviara ese tokenSalida, y el hotel podra validarlo buscando en AerolineaAliado WHERE TokenHASH = tokenSalida.

- Param aerolineaId - ID del registro AerolineaAliado a actualizar.
- Param tokenEntrada - token enviado por la aerolinea (no se persiste, ya no es necesario).
- Param tokenSalida - token generado por el hotel; se guarda en ambas BDs para que coincidan.
- Returns - true si se actualizo al menos un registro, false si no se encontro la aerolinea.

AerolineaWebserviceRepository

Repository para la gestion de aerolineas aliadas desde el portal webservice. Cubre operaciones de consulta, creacion y cambio de estado para el usuario webservice dueno. El token de autenticacion no se gestiona aqui; se asigna automaticamente al conectar.

CODIGO

```
public List<AerolineaWebserviceDTO> listarPorUsuario(int usuarioId)
```

Retorna las aerolineas aliadas asociadas a un usuario webservice especifico. La consulta no realiza JOIN con EstadoAliado para evitar dependencia del ID de estado.

- Param usuarioId - ID del usuario webservice propietario de las aerolineas.
- Returns - lista de AerolineaWebserviceDTO pertenecientes al usuario.

CODIGO

```
public AerolineaWebserviceDTO crear(int usuarioId, CrearAerolineaRequestDTO req)
```

Crea una nueva aerolinea aliada vinculada al usuario webservice indicado. Valida que los campos obligatorios esten presentes y que el usuario no tenga ya una aerolinea ni una agencia registrada, ya que solo se permite una entidad por usuario webservice. o si el usuario ya tiene una aerolinea o una agencia registrada.

- Param usuarioId - ID del usuario webservice que sera propietario de la aerolinea.
- Param req - datos de la nueva aerolinea (nombre, URL del sistema y URL para usuario).
- Returns - AerolineaWebserviceDTO con los datos de la aerolinea recién creada.
- Throws IllegalArgumentException - si algun campo obligatorio esta vacio,

CODIGO

```
public void cambiarEstado(int aerolineaId, int usuarioId, int nuevoEstadoId)
```

Cambia el estado de una aerolinea verificando que pertenezca al usuario webservice indicado.

- Param aerolineaId - ID de la aerolinea a modificar.
- Param usuarioId - ID del usuario webservice propietario de la aerolinea.
- Param nuevoEstadoId - nuevo estado a asignar.
- Throws IllegalArgumentException - si la aerolinea no existe o no pertenece al usuario.

AgenciaRepository

Repository para la gestion de agencias de viaje. Cubre operaciones de consulta, creacion, edicion, cambio de estado y eliminacion, tanto para el panel de administracion como para usuarios webservice.

CODIGO

```
public List<AgenciaDTO> listarTodas()
```

Retorna todas las agencias registradas en el sistema, ordenadas por ID.

- Returns - lista de AgenciaDTO con todas las agencias.

CODIGO

```
public List<AgenciaDTO> listarPorUsuario(int usuarioId)
```

Retorna las agencias asociadas a un usuario webservice especifico.

- Param usuarioId - ID del usuario webservice propietario de las agencias.
- Returns - lista de AgenciaDTO pertenecientes al usuario.

CODIGO

```
public AgenciaDTO crear(int usuarioId, CrearAgenciaRequestDTO req)
```

Crea una nueva agencia vinculada al usuario webservice indicado (flujo del portal webservice). Valida que los campos obligatorios esten presentes y que el usuario no tenga ya una agencia ni una aerolinea registrada, ya que solo se permite una entidad por usuario webservice. o si el usuario ya tiene una agencia o una aerolinea registrada.

- Param usuarioId - ID del usuario webservice que sera propietario de la agencia.
- Param req - datos de la nueva agencia (nombre, correo y URL del sistema externo).
- Returns - AgenciaDTO con los datos de la agencia recién creada.
- Throws IllegalArgumentException - si algun campo obligatorio esta vacio,

CODIGO

```
public void editar(int agenciaId, EditarAgenciaRequestDTO req)
```

Actualiza los datos de una agencia existente desde el panel de administracion. Valida nombre, correo, porcentaje de descuento y estado antes de aplicar los cambios.

- Param agenciaId - ID de la agencia a editar.
- Param req - datos actualizados de la agencia.
- Throws IllegalArgumentException - si algun campo es invalido o el estado no corresponde a Activa o Inactiva.

CODIGO

```
public void cambiarEstado(int agenciaId, int usuarioId, int nuevoEstadoId)
```

Cambia el estado de una agencia verificando que pertenezca al usuario webservice indicado.

- Param agenciaId - ID de la agencia a modificar.
- Param usuarioId - ID del usuario webservice propietario de la agencia.
- Param nuevoEstadoId - nuevo estado a asignar.
- Throws IllegalArgumentException - si la agencia no existe o no pertenece al usuario.

CODIGO

```
public void eliminar(int agenciaId, int usuarioId)
```

Elimina una agencia verificando que pertenezca al usuario webservice indicado.

- Param agenciaId - ID de la agencia a eliminar.
- Param usuarioId - ID del usuario webservice propietario de la agencia.
- Throws IllegalArgumentException - si la agencia no existe o no pertenece al usuario.

CODIGO

```
public Integer obtenerAgenciaIdPorURL(String urlAgencia)
```

Busca el ID de una agencia a partir de su URL registrada.

- Param urlAgencia - URL unica asociada a la agencia.
- Returns - ID de la agencia, o null si no se encuentra ninguna con esa URL.

CODIGO

```
public boolean guardarTokens(int agenciaId, String tokenEntrada, String tokenSalida)
```

Guarda los tokens de entrada y salida asociados a una agencia.

- Param agenciaId - ID de la agencia a actualizar.
- Param tokenEntrada - hash del token de entrada.
- Param tokenSalida - hash del token de salida.
- Returns - true si se actualizo al menos un registro, false si no se encontro la agencia.

CODIGO

```
public AgenciaIdentidad obtenerAgenciaPorToken(String token)
```

Busca una agencia por su token de entrada y retorna su informacion de identidad.

- Param token - hash del token de entrada a buscar.
- Returns - AgenciaIdentidad con los datos basicos de la agencia, o null si no existe.

AuthRepository

Repository para la autenticacion de usuarios. Maneja la busqueda de credenciales en la base de datos durante el proceso de login.

CODIGO

```
public Usuario buscarPorIdentificador(String identificador)
```

Busca un usuario por su username o correo electronico. El identificador se compara contra ambos campos en una sola consulta.

- Param identificador - username o correo del usuario a buscar.
- Returns - instancia de Usuario con sus credenciales, o null si no existe.

BusquedaAgenciaRepository

Repository para la busqueda de hoteles y habitaciones disponibles desde el canal de agencias. Maneja consultas de disponibilidad, descuentos, imagenes y registro de busquedas.

CODIGO

```
public Double obtenerDescuentoAgencia(int usuarioId)
```

Retorna el porcentaje de descuento de la agencia activa asociada a un usuario webservice.

- Param usuarioId - ID del usuario webservice vinculado a la agencia.
- Returns - porcentaje de descuento como Double, o null si no tiene agencia activa.

CODIGO

```
public Integer buscarCiudadId(String nombreCiudad, String nombrePais)
```

Busca el ID de una ciudad comparando nombre de ciudad y nombre de pais sin distincion de mayusculas.

- Param nombreCiudad - nombre de la ciudad a buscar.
- Param nombrePais - nombre del pais al que pertenece la ciudad.
- Returns - ID de la ciudad si existe, o null si no se encuentra.

CODIGO

```
public void guardarBusqueda(int ciudadId, Date fechaCheckIn, Date fechaCheckOut,
```

Registra una busqueda realizada por un usuario webservice autenticado (TipoBusquedaID = 2).

- Param ciudadId - ID de la ciudad consultada.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Param cantidadPersonas - numero de personas para la busqueda.
- Param usuarioId - ID del usuario webservice que realiza la busqueda.

CODIGO

```
public List<HotelResultadoDTO> buscarHotelesPorCiudad(int ciudadId)
```

Retorna los hoteles activos ubicados en una ciudad especifica.

- Param ciudadId - ID de la ciudad donde se buscan los hoteles.
- Returns - lista de HotelResultadoDTO con los hoteles activos encontrados.

CODIGO

```
public List<Integer> buscarImagenesHotel(int hotelId)
```

Retorna los IDs de las imagenes asociadas a un hotel.

- Param hotelId - ID del hotel del que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes del hotel.

CODIGO

```
public List<AmenidadHotelDTO> buscarAmenidadesHotel(int hotelId)
```

Retorna las amenidades registradas para un hotel junto con su descripcion y nombre.

- Param hotelId - ID del hotel del que se quieren obtener las amenidades.
- Returns - lista de AmenidadHotelDTO con los datos de cada amenidad.

CODIGO

```
public List<Integer> buscarImagenesAmenidad(int hotelAmenidadId)
```

Retorna los IDs de las imagenes asociadas a una amenidad de hotel.

- Param hotelAmenidadId - ID del registro HotelAmenidad del que se buscan las imagenes.
- Returns - lista de IDs de imagenes de la amenidad.

CODIGO

```
public List<TipoHabitacionResultadoDTO> buscarTiposHabitacionDisponibles(
```

Retorna los tipos de habitacion disponibles en un hotel para un rango de fechas y capacidad minima. Excluye habitaciones con reservaciones activas (Pendiente o Confirmada) que se traslapen con el rango solicitado. El precio y tipo de cama se obtienen desde TipoHabitacion, no desde Habitacion.

- Param hotelId - ID del hotel donde se busca disponibilidad.
- Param capacidadMinima - capacidad minima requerida por tipo de habitacion.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Returns - lista de TipoHabitacionResultadoDTO con los tipos de habitacion disponibles.

CODIGO

```
public List<HabitacionResumenDTO> buscarHabitacionesResumenPorTipo(
```

Retorna las habitaciones concretas disponibles de un tipo especifico en un hotel para un rango de fechas. Excluye habitaciones con reservaciones activas que se traslapen con el rango solicitado.

- Param hotelId - ID del hotel donde se busca disponibilidad.
- Param tipoHabitacionId - ID del tipo de habitacion requerido.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Returns - lista de HabitacionResumenDTO con el ID y numero de cada habitacion disponible.

```
CODIGO  
public List<Integer> buscarImagenesHabitacion(int habitacionId)
```

Retorna los IDs de las imagenes asociadas a una habitacion especifica.

- Param habitacionId - ID de la habitacion de la que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes de la habitacion.

```
CODIGO  
public void guardarBusquedaSinUsuario(int ciudadId, Date fechaCheckIn, Date fechaCheckOut,
```

Registra una busqueda anonima sin usuario autenticado (TipoBusquedaID = 2).

- Param ciudadId - ID de la ciudad consultada.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Param cantidadPersonas - numero de personas para la busqueda.

```
CODIGO  
public Double obtenerDescuentoAgenciaPorToken(String token)
```

Retorna el porcentaje de descuento de la agencia activa identificada por su token de entrada.

- Param token - hash del token de entrada asociado a la agencia.
- Returns - porcentaje de descuento como Double, o null si no se encuentra una agencia activa con ese token.

BusquedaRepository

*Repository para la busqueda de hoteles y habitaciones disponibles desde el canal web de usuarios.
Maneja consultas de disponibilidad, amenidades, imagenes y registro de busquedas.*

```
CODIGO  
public Integer buscarCiudadId(String nombreCiudad, String nombrePais)
```

Busca el ID de una ciudad comparando nombre de ciudad y nombre de pais sin distincion de mayusculas.

- Param nombreCiudad - nombre de la ciudad a buscar.
- Param nombrePais - nombre del pais al que pertenece la ciudad.
- Returns - ID de la ciudad si existe, o null si no se encuentra.

CODIGO

```
public void guardarBusqueda(int ciudadId, Date fechaCheckIn, Date fechaCheckOut,
```

Registra una busqueda de disponibilidad realizada por un usuario autenticado o anonimo. El TipoBusquedaID se asigna como 1 si hay sesion activa, o null si la busqueda es anonima.

- Param ciudadId - ID de la ciudad consultada.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Param cantidadPersonas - numero de personas para la busqueda.
- Param usuarioid - ID del usuario autenticado, o null si no hay sesion.

CODIGO

```
public List<HotelResultadoDTO> buscarHotelesPorCiudad(int ciudadId)
```

Retorna los hoteles activos ubicados en una ciudad especifica.

- Param ciudadId - ID de la ciudad donde se buscan los hoteles.
- Returns - lista de HotelResultadoDTO con los hoteles activos encontrados.

CODIGO

```
public List<Integer> buscarImagenesHotel(int hotelId)
```

Retorna los IDs de las imagenes asociadas a un hotel.

- Param hotelId - ID del hotel del que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes del hotel.

CODIGO

```
public List<AmenidadHotelDTO> buscarAmenidadesHotel(int hotelId)
```

Retorna las amenidades registradas para un hotel junto con su descripcion y nombre.

- Param hotelId - ID del hotel del que se quieren obtener las amenidades.
- Returns - lista de AmenidadHotelDTO con los datos de cada amenidad.

CODIGO

```
public List<Integer> buscarImagenesAmenidad(int hotelAmenidadId)
```

Retorna los IDs de las imagenes asociadas a una amenidad de hotel.

- Param hotelAmenidadId - ID del registro HotelAmenidad del que se buscan las imagenes.
- Returns - lista de IDs de imagenes de la amenidad.

CODIGO

```
public List<TipoHabitacionResultadoDTO> buscarTiposHabitacionDisponibles(
```

Retorna los tipos de habitacion disponibles en un hotel para un rango de fechas y capacidad minima. Excluye habitaciones con reservaciones activas (Pendiente o Confirmada) que se traslapen con el rango solicitado.

- Param hotelId - ID del hotel donde se busca disponibilidad.
- Param capacidadMinima - capacidad minima requerida por tipo de habitacion.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Returns - lista de TipoHabitacionResultadoDTO con los tipos de habitacion disponibles.

CODIGO

```
public List<HabitacionResumenDTO> buscarHabitacionesResumenPorTipo(
```

Retorna las habitaciones concretas disponibles de un tipo especifico en un hotel para un rango de fechas. Excluye habitaciones con reservaciones activas que se traslapen con el rango solicitado.

- Param hotelId - ID del hotel donde se busca disponibilidad.
- Param tipoHabitacionId - ID del tipo de habitacion requerido.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Returns - lista de HabitacionResumenDTO con el ID y numero de cada habitacion disponible.

CODIGO

```
public List<Integer> buscarImagenesHabitacion(int habitacionId)
```

Retorna los IDs de las imagenes asociadas a una habitacion especifica.

- Param habitacionId - ID de la habitacion de la que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes de la habitacion.

CancelacionRepository

*Repository para la cancelacion de reservaciones, tanto de usuarios directos como de agencias.
Maneja la consulta de estado previo a la cancelacion y la actualizacion del registro.*

CODIGO

```
public Object[] obtenerReservacionParaCancelar(int reservacionId, int usuarioId)
```

Busca una reservacion verificando que pertenezca al usuario indicado, para validar antes de cancelar.

- Param reservacionId - ID de la reservacion a consultar.
- Param usuarioid - ID del usuario propietario de la reservacion.
- Returns - arreglo con {ID, EstadoID, Estado} o null si no existe o no pertenece al usuario.

CODIGO

```
public java.sql.Date obtenerFechaCheckInMasReciente(int reservacionId)
```

Retorna la fecha de check-in mas proxima entre todas las habitaciones de una reservacion. Se usa para validar si la cancelacion esta dentro del plazo permitido.

- Param reservacionId - ID de la reservacion de la que se quiere obtener la fecha.
- Returns - fecha de check-in mas reciente, o null si no se encuentran detalles.

CODIGO

```
public void cancelarReservacion(int reservacionId, String motivoCancelacion)
```

Actualiza el estado de una reservacion a Cancelada (EstadoID = 4) y registra el motivo y la fecha.

- Param reservacionId - ID de la reservacion a cancelar.
- Param motivoCancelacion - razon de la cancelacion ingresada por el usuario.

CODIGO

```
public Object[] obtenerReservacionAgenciaParaCancelar(int reservacionId, int agenciaId)
```

Busca una reservacion verificando que pertenezca a una agencia especifica, para validar antes de cancelar. La relacion se resuelve a traves del usuario webservice vinculado a la agencia.

- Param reservacionId - ID de la reservacion a consultar.
- Param agenciaId - ID de la agencia propietaria de la reservacion.
- Returns - arreglo con {ID, EstadoID, Estado} o null si no existe o no pertenece a la agencia.

CiudadRepository

Repository para la gestion de ciudades. Permite buscar o crear ciudades de forma segura ante condiciones de carrera entre hilos.

CODIGO

```
public int buscarOCrearPorNombre(String nombre, int paisId)
```

Busca una ciudad por nombre y pais, y la crea si no existe. Si dos hilos intentan insertar la misma ciudad simultaneamente y se produce un conflicto de clave unica (ORA-00001), reintenta la busqueda antes de propagar el error.

- Param nombre - nombre de la ciudad a buscar o crear.
- Param paisId - ID del pais al que pertenece la ciudad.
- Returns - ID de la ciudad existente o recién creada.
- Throws RuntimeException - si ocurre un error de base de datos distinto a una colision de clave unica.

ComentarioRepository

Repository para la gestion de comentarios y reseñas de hoteles. Cubre la creacion de comentarios, validacion de reseñas previas y actualizacion del rating del hotel.

CODIGO

```
public boolean existeComentarioConResena(int usuarioId, int hotelId)
```

Verifica si un usuario ya tiene un comentario con resena registrado para un hotel específico.

- Param usuariold - ID del usuario a verificar.
- Param hotelId - ID del hotel a verificar.
- Returns - true si ya existe un comentario con resena del usuario en ese hotel, false en caso contrario.

CODIGO

```
public int crearComentario(int usuarioId, int hotelId, Integer comentarioPadreId,
```

Inserta un nuevo comentario en la base de datos y retorna el ID generado. El comentario puede ser una resena principal o una respuesta a otro comentario si se indica el padre.

- Param usuariold - ID del usuario que publica el comentario.
- Param hotelId - ID del hotel al que pertenece el comentario.
- Param comentarioPadreId - ID del comentario padre si es una respuesta, o null si es raiz.
- Param resena - puntuacion de la resena (1-5), o null si es solo un comentario.
- Param contenido - texto del comentario.
- Returns - ID del comentario recién insertado.

CODIGO

```
public void actualizarRatingHotel(int hotelId)
```

Recalcula y actualiza el rating de un hotel en base al promedio de sus resenas activas.

- Param hotelId - ID del hotel cuyo rating se debe actualizar.

CODIGO

```
public ComentarioResponseDTO obtenerComentario(int comentarioId)
```

Retorna un comentario especifico con los datos del usuario que lo publico.

- Param comentarioId - ID del comentario a buscar.
- Returns - ComentarioResponseDTO con los datos del comentario, o null si no existe.

CODIGO

```
public List<ComentarioResponseDTO> obtenerComentariosPorUsuario(int usuarioId)
```

Retorna todos los comentarios publicados por un usuario, ordenados por fecha descendente.

- Param usuariold - ID del usuario del que se quieren obtener los comentarios.
- Returns - lista de ComentarioResponseDTO con los comentarios del usuario.

CODIGO

```
public List<ComentarioResponseDTO> obtenerComentariosPorHotel(int hotelId)
```

Retorna todos los comentarios registrados para un hotel, ordenados por fecha descendente.

- Param hotelId - ID del hotel del que se quieren obtener los comentarios.
- Returns - lista de ComentarioResponseDTO con los comentarios del hotel.

DestinosRepository

Repository para la consulta de destinos turisticos disponibles. Provee acceso a los hoteles activos y sus imagenes para la vista de destinos.

```
CODIGO  
public List<HotelResultadoDTO> obtenerTodosLosHoteles()
```

Retorna todos los hoteles activos del sistema ordenados por pais, ciudad y nombre.

- Returns - lista de HotelResultadoDTO con los datos de cada hotel activo.

```
CODIGO  
public List<Integer> obtenerImagenesHotel(int hotelId)
```

Retorna los IDs de las imagenes asociadas a un hotel especifico.

- Param hotelId - ID del hotel del que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes del hotel.

DownsRepository

Repository para la gestion de downs (votos negativos) sobre comentarios de hoteles. Permite consultar, registrar, eliminar y contabilizar downs por usuario y comentario.

```
CODIGO  
public List<DownResponseDTO> obtenerDownsDeUsuario(int usuarioId)
```

Retorna todos los downs registrados por un usuario, ordenados por fecha descendente.

- Param usuarioId - ID del usuario del que se quieren obtener los downs.
- Returns - lista de DownResponseDTO con los datos de cada down del usuario.

```
CODIGO  
public List<DownResponseDTO> obtenerDownsDeUsuarioPorHotel(int usuarioId, int hotelId)
```

Retorna los downs de un usuario filtrados por un hotel especifico, ordenados por fecha descendente.

- Param usuarioId - ID del usuario del que se quieren obtener los downs.
- Param hotelId - ID del hotel por el que se filtra.
- Returns - lista de DownResponseDTO con los downs del usuario en ese hotel.

```
CODIGO  
public Integer obtenerValorDown(int usuarioId, int comentarioId)
```

Retorna el valor del down que un usuario tiene registrado sobre un comentario especifico. Se usa para verificar si ya existe un down antes de insertar o eliminar.

- Param usuarioid - ID del usuario a verificar.
- Param comentarioid - ID del comentario a verificar.
- Returns - valor del down existente, o null si el usuario no ha marcado ese comentario.

CODIGO

```
public void insertarDown(int usuarioId, int comentarioId, int valor)
```

Registra un nuevo down de un usuario sobre un comentario.

- Param usuarioid - ID del usuario que registra el down.
- Param comentarioid - ID del comentario sobre el que se aplica el down.
- Param valor - valor del down a registrar.

CODIGO

```
public void eliminarDown(int usuarioId, int comentarioId)
```

Elimina el down que un usuario tiene registrado sobre un comentario.

- Param usuarioid - ID del usuario cuyo down se eliminara.
- Param comentarioid - ID del comentario del que se quita el down.

CODIGO

```
public void actualizarContadorDown(int comentarioId, int delta)
```

Incrementa o decrementa el contador de downs de un comentario segun el delta indicado. Se usa para mantener sincronizado el campo Downs del comentario tras insertar o eliminar un down.

- Param comentarioid - ID del comentario cuyo contador se actualiza.
- Param delta - valor a sumar al contador (puede ser negativo para restar).

HotelAgenciaRepository

Repository para la consulta de hoteles disponibles desde el canal de agencias. Provee el listado de hoteles activos con su ciudad y pais para uso en reservaciones de agencia.

CODIGO

```
public List<HotelAgenciaDTO> listarHotelesParaAgencia()
```

Retorna todos los hoteles activos con su informacion de ubicacion, ordenados por ID.

- Returns - lista de HotelAgenciaDTO con el ID, nombre, ciudad y pais de cada hotel activo.

HotelRepository

Repository para la gestion completa de hoteles desde el panel de administracion. Cubre amenidades, habitaciones, imagenes, reservaciones y metricas del sistema.

CODIGO

```
public List<AmenidadDTO> listarAmenidades()
```

Retorna el catalogo completo de amenidades disponibles, ordenadas por ID.

- Returns - lista de AmenidadDTO con el ID y nombre de cada amenidad.

CODIGO

```
public int crearAmenidad(String nombre)
```

Crea una nueva amenidad en el catalogo y retorna el ID generado.

- Param nombre - nombre de la amenidad a crear.
- Returns - ID de la amenidad recién insertada.

CODIGO

```
public List<HotelAdminDTO> listarTodos()
```

Retorna todos los hoteles registrados con su estado y ubicacion, ordenados por ID.

- Returns - lista de HotelAdminDTO con los datos completos de cada hotel.

CODIGO

```
public int crearHotel(String nombre, String direccion, String descripcion,
```

Inserta un nuevo hotel en la base de datos y retorna el ID generado.

- Param nombre - nombre del hotel.
- Param direccion - direccion fisica del hotel.
- Param descripcion - descripcion general del hotel.
- Param rating - calificacion inicial del hotel.
- Param estadoId - ID del estado inicial del hotel.
- Param ciudadId - ID de la ciudad donde se ubica el hotel.
- Returns - ID del hotel recién insertado.

CODIGO

```
public void actualizarHotel(int hotelId, String nombre, String direccion,
```

Actualiza los datos principales de un hotel existente.

- Param hotelId - ID del hotel a actualizar.
- Param nombre - nuevo nombre del hotel.
- Param direccion - nueva direccion del hotel.
- Param descripcion - nueva descripcion del hotel.
- Param rating - nuevo rating del hotel.
- Param estadoId - nuevo ID de estado del hotel.

CODIGO

```
public void eliminarHotel(int hotelId)
```

Elimina un hotel y todos sus registros dependientes en cascada. Borra en orden: imagenes de habitaciones, habitaciones, imagenes de amenidades, amenidades del hotel, imagenes del hotel y finalmente el hotel.

- Param hotelId - ID del hotel a eliminar.

CODIGO

```
public boolean existe(int hotelId)
```

Verifica si un hotel existe en la base de datos.

- Param hotelId - ID del hotel a verificar.
- Returns - true si el hotel existe, false en caso contrario.

CODIGO

```
public int contarHabitaciones(int hotelId)
```

Retorna la cantidad de habitaciones registradas para un hotel.

- Param hotelId - ID del hotel a consultar.
- Returns - numero de habitaciones del hotel, o 0 si no tiene ninguna.

CODIGO

```
public List<HotelAmenidadDTO> listarAmenidadesHotel(int hotelId)
```

Retorna las amenidades asignadas a un hotel con su descripcion y nombre, ordenadas por ID.

- Param hotelId - ID del hotel del que se quieren obtener las amenidades.
- Returns - lista de HotelAmenidadDTO con los datos de cada amenidad del hotel.

CODIGO

```
public boolean tieneAmenidad(int hotelId, int amenidadId)
```

Verifica si un hotel ya tiene asignada una amenidad especifica.

- Param hotelId - ID del hotel a verificar.
- Param amenidadId - ID de la amenidad a verificar.
- Returns - true si la amenidad ya esta asignada al hotel, false en caso contrario.

CODIGO

```
public int agregarAmenidadHotel(int hotelId, int amenidadId, String descripcion)
```

Asigna una amenidad a un hotel con una descripcion personalizada y retorna el ID generado.

- Param hotelId - ID del hotel al que se agrega la amenidad.
- Param amenidadId - ID de la amenidad a agregar.
- Param descripcion - descripcion particular de la amenidad en ese hotel.

- Returns - ID del registro HotelAmenidad recién insertado.

CODIGO

```
public void actualizarAmenidadHotel(int hotelAmenidadId, String descripcion)
```

Actualiza la descripción de una amenidad asignada a un hotel.

- Param hotelAmenidadId - ID del registro HotelAmenidad a actualizar.
- Param descripcion - nueva descripción de la amenidad.

CODIGO

```
public void eliminarAmenidadHotel(int hotelAmenidadId)
```

Elimina una amenidad de un hotel junto con todas sus imágenes asociadas.

- Param hotelAmenidadId - ID del registro HotelAmenidad a eliminar.

CODIGO

```
public List<Integer> obtenerImagenesAmenidadIds(int hotelAmenidadId)
```

Retorna los IDs de las imágenes asociadas a una amenidad de hotel, ordenados por ID.

- Param hotelAmenidadId - ID del registro HotelAmenidad del que se buscan las imágenes.
- Returns - lista de IDs de imágenes de la amenidad.

CODIGO

```
public int agregarImagenAmenidad(int hotelAmenidadId, byte[] imagen)
```

Agrega una imagen a una amenidad de hotel y retorna el ID generado.

- Param hotelAmenidadId - ID del registro HotelAmenidad al que se agrega la imagen.
- Param imagen - bytes de la imagen a guardar.
- Returns - ID de la imagen recién insertada.

CODIGO

```
public void eliminarImagenAmenidad(int imagenId)
```

Elimina una imagen de amenidad de hotel por su ID.

- Param imagenId - ID de la imagen a eliminar.

CODIGO

```
public List<Integer> obtenerImagenesIds(int hotelId)
```

Retorna los IDs de las imágenes asociadas a un hotel, ordenados por ID.

- Param hotelId - ID del hotel del que se quieren obtener los IDs de imágenes.
- Returns - lista de IDs de imágenes del hotel.

CODIGO

```
public int agregarImagenHotel(int hotelId, byte[] imagen)
```

Agrega una imagen a un hotel y retorna el ID generado.

- Param hotelId - ID del hotel al que se agrega la imagen.
- Param imagen - bytes de la imagen a guardar.
- Returns - ID de la imagen recién insertada.

CODIGO

```
public void eliminarImagenHotel(int imagenId)
```

Elimina una imagen de hotel por su ID.

- Param imagenId - ID de la imagen a eliminar.

CODIGO

```
public List<HabitacionAdminDTO> listarHabitacionesPorHotel(int hotelId)
```

Retorna todas las habitaciones de un hotel con sus datos de tipo, cama, precio y estado.

- Param hotelId - ID del hotel del que se quieren listar las habitaciones.
- Returns - lista de HabitacionAdminDTO con los datos completos de cada habitacion.

CODIGO

```
public int crearHabitacion(int hotelId, int tipoHabitacionId,
```

Crea una nueva habitacion en un hotel y auto-asigna el numero correlativo basado en la cantidad de habitaciones ya existentes en ese hotel.

- Param hotelId - ID del hotel donde se crea la habitacion.
- Param tipoHabitacionId - ID del tipo de habitacion a asignar.
- Param descripcion - descripcion particular de la habitacion.
- Param estadoId - ID del estado inicial de la habitacion.
- Returns - ID de la habitacion recién insertada.

CODIGO

```
public boolean existeHabitacion(int habitacionId)
```

Verifica si una habitacion existe en la base de datos.

- Param habitacionId - ID de la habitacion a verificar.
- Returns - true si la habitacion existe, false en caso contrario.

CODIGO

```
public void actualizarHabitacion(int habitacionId, int tipoHabitacionId,
```

Actualiza los datos de una habitacion existente.

- Param habitacionId - ID de la habitacion a actualizar.
- Param tipoHabitacionId - nuevo ID del tipo de habitacion.
- Param numeroHabitacion - nuevo numero de habitacion.
- Param descripcion - nueva descripcion de la habitacion.
- Param estadoId - nuevo ID de estado de la habitacion.

CODIGO

```
public void eliminarHabitacion(int habitacionId)
```

Elimina una habitacion y todas sus imagenes asociadas.

- Param habitacionId - ID de la habitacion a eliminar.

CODIGO

```
public List<Integer> obtenerImagenesHabitacionIds(int habitacionId)
```

Retorna los IDs de las imagenes asociadas a una habitacion, ordenados por ID.

- Param habitacionId - ID de la habitacion de la que se quieren obtener los IDs de imagenes.
- Returns - lista de IDs de imagenes de la habitacion.

CODIGO

```
public int agregarImagenHabitacion(int habitacionId, byte[] imagen)
```

Agrega una imagen a una habitacion y retorna el ID generado.

- Param habitacionId - ID de la habitacion a la que se agrega la imagen.
- Param imagen - bytes de la imagen a guardar.
- Returns - ID de la imagen recién insertada.

CODIGO

```
public void eliminarImagenHabitacion(int imagenId)
```

Elimina una imagen de habitacion por su ID.

- Param imagenId - ID de la imagen a eliminar.

CODIGO

```
public List<java.util.Map<String, Object>> listarTodasReservaciones()
```

Retorna todas las reservaciones del sistema con datos del usuario, hotel y fechas, ordenadas por fecha de creacion descendente.

- Returns - lista de mapas con los datos resumidos de cada reservacion.

CODIGO

```
public java.util.Map<String, Object> obtenerMetricas()
```

Retorna un mapa con las metricas generales del sistema para el panel de administracion. Incluye totales de usuarios, hoteles, reservaciones por estado e ingresos confirmados. ingresosTotales, reservasPorEstado y hotesTotales.

- Returns - mapa con las claves: totalUsuarios, hotelesActivos, reservasActivas, reservasTotales,

ImagenRepository

Repository para la recuperacion de imagenes almacenadas en la base de datos. Cubre imagenes de hoteles, habitaciones y amenidades.

```
CODIGO  
public byte[] obtenerImagenHotel(int id)
```

Retorna los bytes de una imagen de hotel por su ID.

- Param id - ID de la imagen a recuperar.
- Returns - arreglo de bytes con la imagen, o null si no existe.

```
CODIGO  
public byte[] obtenerImagenHabitacion(int id)
```

Retorna los bytes de una imagen de habitacion por su ID.

- Param id - ID de la imagen a recuperar.
- Returns - arreglo de bytes con la imagen, o null si no existe.

```
CODIGO  
public byte[] obtenerImagenAmenidad(int id)
```

Retorna los bytes de una imagen de amenidad de hotel por su ID.

- Param id - ID de la imagen a recuperar.
- Returns - arreglo de bytes con la imagen, o null si no existe.

NacionalidadRepository

Repository para la gestion de nacionalidades. Permite buscar o crear una nacionalidad de forma idempotente por nombre.

```
CODIGO  
public int buscarOCrearPorNombre(String nombre)
```

Busca una nacionalidad por nombre y la crea si no existe. La comparacion del nombre es case-insensitive.

- Param nombre - nombre de la nacionalidad a buscar o crear.
- Returns - ID de la nacionalidad existente o recién creada.

PagoAgenciaRepository

Repository para el procesamiento de pagos de reservaciones realizadas por agencias. Maneja la validacion de reservaciones, confirmacion y generacion de facturas.

CODIGO

```
public Object[] obtenerReservacionParaPago(int reservacionId, int agenciaId)
```

Busca una reservacion verificando que pertenezca a la agencia indicada. Se usa para validar la reservacion antes de procesar el pago.

- Param reservacionId - ID de la reservacion a consultar.
- Param agenciaId - ID de la agencia propietaria de la reservacion.
- Returns - arreglo con {ID, No_Reservacion, Total, Estado, EstadoID} o null si no existe o no pertenece a la agencia.

CODIGO

```
public void confirmarReservacion(int reservacionId)
```

Confirma una reservacion actualizando su estado a Confirmada (EstadoID = 2) y eliminando la fecha de expiracion.

- Param reservacionId - ID de la reservacion a confirmar.

CODIGO

```
public int crearFactura(int reservacionId, String nit, String codigoPostal, double total)
```

Crea una factura asociada a una reservacion y retorna el ID generado.

- Param reservacionId - ID de la reservacion a facturar.
- Param nit - numero de identificacion tributaria del cliente.
- Param codigoPostal - codigo postal del cliente.
- Param total - monto total de la factura.
- Returns - ID de la factura recién insertada.

CODIGO

```
public PagoResponseDTO obtenerFactura(int facturaId)
```

Retorna los datos completos de una factura junto con el estado y numero de su reservacion asociada.

- Param facturaId - ID de la factura a consultar.
- Returns - PagoResponseDTO con los datos de la factura, o null si no existe.

PagoRepository

Repository para el procesamiento de pagos de reservaciones de usuarios directos. Maneja la validacion de reservaciones, confirmacion y generacion de facturas.

CODIGO

```
public Object[] obtenerReservacionParaPago(int reservacionId, int usuarioId)
```

Busca una reservacion verificando que pertenezca al usuario indicado. Se usa para validar la reservacion antes de procesar el pago.

- Param reservacionId - ID de la reservacion a consultar.
- Param usuariold - ID del usuario propietario de la reservacion.
- Returns - arreglo con {ID, No_Reservacion, Total, Estado, EstadoID} o null si no existe o no pertenece al usuario.

CODIGO

```
public void confirmarReservacion(int reservacionId)
```

Confirma una reservacion actualizando su estado a Confirmada (EstadoID = 2) y eliminando la fecha de expiracion.

- Param reservacionId - ID de la reservacion a confirmar.

CODIGO

```
public int crearFactura(int reservacionId, String nit, String codigoPostal, double total)
```

Crea una factura asociada a una reservacion y retorna el ID generado.

- Param reservacionId - ID de la reservacion a facturar.
- Param nit - numero de identificacion tributaria del cliente.
- Param codigoPostal - codigo postal del cliente.
- Param total - monto total de la factura.
- Returns - ID de la factura recién insertada.

CODIGO

```
public PagoResponseDTO obtenerFactura(int facturaId)
```

Retorna los datos completos de una factura junto con el estado y numero de su reservacion asociada.

- Param facturald - ID de la factura a consultar.
- Returns - PagoResponseDTO con los datos de la factura, o null si no existe.

CODIGO

```
public void actualizarTotalReservacion(int reservacionId, double nuevoTotal)
```

Actualiza el total de una reservacion tras aplicar un descuento de alianza.

- Param reservacionId - ID de la reservacion a actualizar.
- Param nuevoTotal - total ya con el descuento aplicado.

CODIGO

```
public void actualizarTotalDetalles(int reservacionId, double factorDescuento)
```

Aplica el mismo factor de descuento proporcionalmente a todos los detalles de una reservacion, para que la suma de detalles coincida con el total de la reservacion.

- Param reservacionId - ID de la reservacion cuyos detalles se van a actualizar.
- Param factorDescuento - factor multiplicador (ej. 0.90 para 10% de descuento).

CODIGO

```
public String obtenerCiudadReservacion(int reservacionId)
```

Obtiene el nombre de la ciudad del hotel asociado a una reservacion. Se usa para validar que el token de alianza aplique unicamente a hoteles en la ciudad para la que fue generado el token.

- Param reservacionId - ID de la reservacion.
- Returns - Nombre de la ciudad del hotel o null si no se encuentra.

PaisRepository

Repository para la gestion de paises y ciudades. Permite buscar o crear paises de forma segura ante condiciones de carrera, y provee listados de paises y ciudades para uso en formularios del panel de administracion.

CODIGO

```
public int buscarOCrearPorNombre(String nombre)
```

Busca un pais por nombre y lo crea si no existe. Si dos hilos intentan insertar el mismo pais simultaneamente y se produce un conflicto de clave unica (ORA-00001), reintenta la busqueda antes de propagar el error.

- Param nombre - nombre del pais a buscar o crear.
- Returns - ID del pais existente o recién creado.
- Throws RuntimeException - si ocurre un error de base de datos distinto a una colision de clave unica.

CODIGO

```
public List<PaisDTO> listarPaises()
```

Retorna todos los paises registrados ordenados alfabeticamente por nombre. Se usa para poblar dropdowns en el panel de administracion.

- Returns - lista de PaisDTO con el ID y nombre de cada pais.

CODIGO

```
public List<CiudadDTO> listarCiudades()
```

Retorna todas las ciudades registradas con su pais asociado, ordenadas por pais y nombre de ciudad. Se usa para poblar dropdowns en el panel de administracion.

- Returns - lista de CiudadDTO con el ID, nombre, ID de pais y nombre de pais de cada ciudad.

PdfReservacionRepository

Repository para la generacion de PDFs de reservaciones. Provee los datos necesarios para construir el documento: correo del usuario, validacion de pertenencia, detalles de la reservacion y factura asociada.

CODIGO

```
public String obtenerCorreoUsuario(int usuarioId)
```

Retorna el correo electronico de un usuario por su ID.

- Param usuarioId - ID del usuario del que se quiere obtener el correo.
- Returns - correo del usuario, o null si no existe.

CODIGO

```
public boolean perteneceAlUsuario(int reservacionId, int usuarioId)
```

Verifica si una reservacion pertenece a un usuario especifico.

- Param reservacionId - ID de la reservacion a verificar.
- Param usuarioId - ID del usuario a validar como propietario.
- Returns - true si la reservacion pertenece al usuario, false en caso contrario.

CODIGO

```
public List<ReservacionDetalleDTO> obtenerDetalles(int reservacionId)
```

Retorna los detalles completos de una reservacion para la generacion del PDF. Incluye datos de la reservacion, habitaciones, tipo de cama, hotel y fechas de check-in y check-out.

- Param reservacionId - ID de la reservacion de la que se quieren obtener los detalles.
- Returns - lista de ReservacionDetalleDTO con una entrada por cada habitacion reservada.

CODIGO

```
public Object[] obtenerFactura(int reservacionId)
```

Retorna los datos de la factura asociada a una reservacion, si existe.

- Param reservacionId - ID de la reservacion de la que se busca la factura.
- Returns - arreglo con {ID, Fecha, NIT,Codigo_Postal, Total} o null si no existe factura.

ReservacionAgenciaRepository

Repository para la gestion de reservaciones realizadas por agencias de viaje. Cubre la creacion, consulta, validacion de disponibilidad y expiracion de reservaciones, asi como la obtencion de datos de agencia y precios de habitaciones.

CODIGO

```
public int[] obtenerDatosAgencia(int agenciaId)
```

Retorna el ID del usuario webservice y el porcentaje de descuento de una agencia activa.

- Param agenciaId - ID de la agencia a consultar.
- Returns - arreglo con {UsuarioWEBIs_ID, PorcentajeDescuento (entero)} o null si la agencia no existe o no esta activa.

CODIGO

```
public double obtenerDescuentoAgencia(int agenciaId)
```

Retorna el porcentaje de descuento exacto con decimales de una agencia activa.

- Param agenciaId - ID de la agencia a consultar.
- Returns - porcentaje de descuento como double, o 0.0 si la agencia no existe o no esta activa.

CODIGO

```
public double[] obtenerPrecios(int habitacionId)
```

Retorna el precio por noche, precio por persona y capacidad maxima de una habitacion. Los valores se retornan en ese orden: precios[0] = precioPorNoche, precios[1] = precioPorPersona, precios[2] = capacidadMaxima.

- Param habitacionId - ID de la habitacion a consultar.
- Returns - arreglo de doubles con los tres valores de precio y capacidad.
- Throws RuntimeException - si la habitacion no existe en la base de datos.

CODIGO

```
public boolean existeTraslape(int habitacionId, Date fechaCheckIn, Date fechaCheckOut)
```

Verifica si una habitacion tiene reservaciones activas que se traslapen con el rango de fechas indicado. Solo considera reservaciones en estado Pendiente o Confirmada.

- Param habitacionId - ID de la habitacion a verificar.
- Param fechaCheckIn - fecha de entrada solicitada.
- Param fechaCheckOut - fecha de salida solicitada.
- Returns - true si existe al menos un traslape, false si la habitacion esta disponible.

CODIGO

```
public int crearReservacion(String noReservacion, double total, int usuarioWebisId,
```

Crea una nueva reservacion en estado Pendiente (EstadoID = 1) y retorna el ID generado.

- Param noReservacion - codigo unico de la reservacion.
- Param total - monto total de la reservacion.
- Param usuarioWebisId - ID del usuario webservice asociado a la agencia.
- Param fechaCreacion - fecha y hora de creacion de la reservacion.
- Param fechaExpiracion - fecha y hora limite para completar el pago.
- Returns - ID de la reservacion recién insertada.

CODIGO

```
public boolean perteneceAAgenciaYEstaPendiente(int reservacionId, int agenciaId)
```

Verifica si una reservacion pertenece a una agencia especifica y se encuentra en estado Pendiente.

- Param reservacionId - ID de la reservacion a verificar.
- Param agenciaId - ID de la agencia a validar como propietaria.
- Returns - true si la reservacion pertenece a la agencia y esta pendiente, false en caso contrario.

CODIGO

```
public void expirarReservacion(int reservacionId)
```

Marca una reservacion como Expirada si actualmente se encuentra en estado Pendiente.

- Param reservacionId - ID de la reservacion a expirar.

CODIGO

```
public void crearDetalle(int reservacionId, int habitacionId,
```

Inserta un detalle de reservacion con los datos de una habitacion y el rango de fechas solicitado.

- Param reservacionId - ID de la reservacion a la que pertenece el detalle.
- Param habitacionId - ID de la habitacion reservada.
- Param fechaCheckIn - fecha de entrada.
- Param fechaCheckOut - fecha de salida.
- Param cantidadPersonas - numero de personas para este detalle.
- Param total - costo total de este detalle.

CODIGO

```
public Object[] obtenerReservacion(int reservacionId)
```

Retorna los datos resumidos de una reservacion para construir la respuesta al cliente. o null si no existe.

- Param reservacionId - ID de la reservacion a consultar.
- Returns - arreglo con {ID, NoReservacion, Total, FechaCreacion, Fecha_Expiracion, Estado}

CODIGO

```
public List<ReservacionDetalleDTO> obtenerReservacionesDeAgencia(int agenciaId)
```

Retorna todas las reservaciones asociadas a una agencia con el detalle completo de habitaciones y hotel.

- Param agenciaId - ID de la agencia de la que se quieren obtener las reservaciones.
- Returns - lista de ReservacionDetalleDTO con una entrada por cada habitacion reservada.

CODIGO

```
public List<Integer> obtenerImagenesHotel(int hotelId)
```

Retorna los IDs de las imagenes asociadas a un hotel.

- Param hotelId - ID del hotel del que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes del hotel.

CODIGO

```
public List<Integer> obtenerImagenesHabitacion(int habitacionId)
```

Retorna los IDs de las imagenes asociadas a una habitacion.

- Param habitacionId - ID de la habitacion de la que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes de la habitacion.

CODIGO

```
public List<ReservacionDetalleDTO> obtenerDetalleReservacionAgencia(int reservacionId, int agenciaId)
```

Retorna el detalle completo de una reservacion especifica verificando que pertenezca a la agencia indicada. Primero obtiene el ID del usuario webservice de la agencia y luego filtra la reservacion por ese usuario. o la reservacion no le pertenece.

- Param reservacionId - ID de la reservacion a consultar.
- Param agenciaId - ID de la agencia propietaria de la reservacion.
- Returns - lista de ReservacionDetalleDTO con una entrada por cada habitacion, o lista vacia si la agencia no existe

ReservacionRepository

Repository para el acceso a datos relacionados con reservaciones. Maneja consultas y actualizaciones sobre reservaciones, detalles e imagenes.

CODIGO

```
public double[] obtenerPrecios(int habitacionId)
```

Retorna los precios y la capacidad maxima de una habitacion segun su tipo.

- Param habitacionId - ID de la habitacion a consultar.
- Returns - arreglo con precio por noche, precio por persona y capacidad maxima.
- Throws RuntimeException - si la habitacion no existe en la base de datos.

CODIGO

```
public boolean existeTraslape(int habitacionId, Date fechaCheckIn, Date fechaCheckOut)
```

Verifica si existe un traslape de fechas para una habitacion con reservaciones activas. Solo considera reservaciones en estado pendiente o confirmada.

- Param habitacionId - ID de la habitacion a verificar.
- Param fechaCheckIn - fecha de entrada de la nueva reservacion.
- Param fechaCheckOut - fecha de salida de la nueva reservacion.
- Returns - true si existe al menos un traslape, false en caso contrario.

CODIGO

```
public void expirarPendientesDeUsuario(int usuarioId, int reservacionIdExcluir)
```

Marca como expiradas todas las reservaciones pendientes de un usuario, exceptuando la reservacion que se acaba de crear o actualizar.

- Param `usuarioid` - ID del usuario dueño de las reservaciones.
- Param `reservacionIdExcluid` - ID de la reservacion que no debe expirar.

CODIGO

```
public int crearReservacion(String noReservacion, double total, int usuarioId,
```

Inserta una nueva reservacion en la base de datos con estado inicial pendiente.

- Param `noReservacion` - codigo unico que identifica la reservacion.
- Param `total` - monto total de la reservacion.
- Param `usuarioid` - ID del usuario que realiza la reservacion.
- Param `fechaCreacion` - fecha y hora en que se creo la reservacion.
- Param `fechaExpiracion` - fecha y hora en que expira la reservacion si no se confirma.
- Returns - ID generado por la base de datos para la nueva reservacion.

CODIGO

```
public void crearDetalle(int reservacionId, int habitacionId,
```

Inserta el detalle de una habitacion asociado a una reservacion existente.

- Param `reservacionId` - ID de la reservacion a la que pertenece el detalle.
- Param `habitacionId` - ID de la habitacion reservada.
- Param `fechaCheckIn` - fecha de entrada a la habitacion.
- Param `fechaCheckOut` - fecha de salida de la habitacion.
- Param `cantidadPersonas` - numero de personas para esta habitacion.
- Param `total` - costo total calculado para este detalle.

CODIGO

```
public Object[] obtenerReservacion(int reservacionId)
```

Retorna los datos principales de una reservacion junto con su estado actual.

- Param `reservacionId` - ID de la reservacion a consultar.
- Returns - arreglo con los campos de la reservacion, o null si no existe.

CODIGO

```
public List<ReservacionDetalleDTO> obtenerReservacionesDeUsuario(int usuarioId)
```

Retorna todas las reservaciones de un usuario con el detalle completo de cada habitacion. Incluye informacion del hotel, tipo de habitacion, cama y fechas de cada detalle.

- Param `usuarioid` - ID del usuario cuyas reservaciones se desean consultar.
- Returns - lista de DTOs con la informacion consolidada de reservaciones y detalles.

CODIGO

```
public List<Integer> obtenerImagenesHotel(int hotelId)
```

Retorna los IDs de todas las imagenes registradas para un hotel.

- Param hotelId - ID del hotel del que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes del hotel.

CODIGO

```
public List<Integer> obtenerImagenesHabitacion(int habitacionId)
```

Retorna los IDs de todas las imagenes registradas para una habitacion.

- Param habitacionId - ID de la habitacion de la que se quieren obtener las imagenes.
- Returns - lista de IDs de imagenes de la habitacion.

CODIGO

```
public int expirarReservacionesVencidas()
```

Marca como expiradas todas las reservaciones pendientes cuya fecha de expiracion ya paso.

- Returns - numero de filas actualizadas en la base de datos.

SesionRepository

Repository para el acceso a datos relacionados con sesiones de usuario. Actualmente provee consultas sobre roles del sistema.

CODIGO

```
public String obtenerNombreRol(int rolId)
```

Retorna el nombre del rol correspondiente a un ID dado. Si el rol no existe, devuelve el valor por defecto "Desconocido".

- Param rolId - ID del rol a consultar.
- Returns - nombre del rol, o "Desconocido" si no se encuentra.

TokenAerolineaRepository

Repository para la creacion y consulta de tokens de alianza entre aerolineas y hoteles.

CODIGO

```
public Integer obtenerAerolineaIdPorToken(String tokenHash)
```

Retorna el ID de la aerolinea aliada activa que corresponde al token HASH dado.

- Param tokenHash - hash del token enviado en el header por la aerolinea.
- Returns - ID de la aerolinea si existe y esta activa, null si no.

CODIGO

```
public Integer buscarCiudadId(String nombreCiudad, String nombrePais)
```

Busca el ID de una ciudad por su nombre y el nombre de su país, sin distinción de mayúsculas ni espacios.

- Param nombreCiudad - nombre de la ciudad destino.
- Param nombrePaís - nombre del país al que pertenece la ciudad.
- Returns - ID de la ciudad si existe, null si no se encuentra.

CODIGO

```
public void insertarToken(int aerolineaAliadoId, int ciudadId,
```

Inserta un nuevo token de alianza en la base de datos. El token se crea como no usado, sin fecha de uso, sin reservación, y con expiración a 15 minutos desde el momento de creación.

- Param aerolineaAliadoId - ID de la aerolínea aliada que genera el token.
- Param ciudadId - ID de la ciudad destino del pasajero.
- Param token - string único generado para este token de alianza.
- Param fechaExpiracion - timestamp calculado como now() mas 15 minutos.

TokenValidacionRepository

Repository para la validación de tokens de alianza recibidos desde aerolíneas.

CODIGO

```
public TokenValidacionResponseDTO buscarTokenValido(String token)
```

Busca un token válido: que exista, no esté usado y no haya expirado. Retorna ciudad, país y porcentaje de descuento si el token es válido.

- Param token - string UUID recibido desde la URL del usuario.
- Returns - TokenValidacionResponseDTO con los datos del token, o null si no es válido.

CODIGO

```
public void marcarTokenUsado(String token, int reservacionId)
```

Marca un token de alianza como usado, registra la fecha de uso y vincula la reservación que se generó con ese descuento.

- Param token - string UUID del token a cerrar.
- Param reservacionId - ID de la reservación pagada con este token.

UsuarioNacionalidadRepository

Repository para gestionar la relación entre usuarios y sus nacionalidades.

CODIGO

```
public void asignarNacionalidades(int usuarioId, List<Integer> nacionalidadIds)
```

Inserta en la base de datos las nacionalidades asociadas a un usuario. Si la combinación ya existe (ORA-00001), la ignora y continúa.

- Param usuariold - ID del usuario al que se le asignan las nacionalidades.
- Param nacionalidadIds - lista de IDs de nacionalidades a asociar.

UsuarioRepository

Repository para el acceso a datos de usuarios. Cubre validaciones de unicidad, creacion, consulta de perfil, gestion de contraseñas y operaciones administrativas de rol.

```
CODIGO  
public boolean existeUsername(String username)
```

Verifica si ya existe un usuario con el username dado.

- Param username - nombre de usuario a verificar.
- Returns - true si el username ya esta en uso, false en caso contrario.

```
CODIGO  
public boolean existeCorreo(String correo)
```

Verifica si ya existe un usuario registrado con el correo dado.

- Param correo - correo electronico a verificar.
- Returns - true si el correo ya esta en uso, false en caso contrario.

```
CODIGO  
public boolean existePasaporte(String pasaporte)
```

Verifica si ya existe un usuario registrado con el numero de pasaporte dado. Retorna false directamente si el pasaporte es nulo o esta vacio.

- Param pasaporte - numero de pasaporte a verificar.
- Returns - true si el pasaporte ya esta en uso, false en caso contrario.

```
CODIGO  
public int crearUsuario(  

```

Inserta un nuevo usuario en la base de datos con rol de cliente por defecto. Si Oracle lanza ORA-00001 (restriccion unica violada) por una race condition, re-verifica que campos estan duplicados y lanza {@link CamposDuplicadosException}.

- Param correo - correo electronico del usuario.
- Param contraseñaHashada - contraseña ya procesada con hash.
- Param pasaporte - numero de pasaporte del usuario, puede ser nulo.
- Param username - nombre de usuario unico.
- Param nombre - nombre de pila del usuario.
- Param apellido - apellido del usuario.
- Param telefono - numero de telefono de contacto.
- Param fechaNacimiento - fecha de nacimiento del usuario.
- Param ciudadId - ID de la ciudad de residencia del usuario.

- Returns - ID generado por la base de datos para el nuevo usuario.
- Throws CamposDuplicadosException - si correo, pasaporte o username ya existen.

CODIGO

```
public UsuarioPerfilResponseDTO obtenerPerfil(int usuarioId)
```

Retorna el perfil completo de un usuario incluyendo su ciudad y pais.

- Param usuariold - ID del usuario a consultar.
- Returns - DTO con los datos del perfil, o null si el usuario no existe.

CODIGO

```
public List<String> obtenerNacionalidades(int usuarioId)
```

Retorna los nombres de las nacionalidades registradas para un usuario.

- Param usuariold - ID del usuario a consultar.
- Returns - lista de nombres de nacionalidades del usuario.

CODIGO

```
public void actualizarTelefono(int usuarioId, String telefono)
```

Actualiza el numero de telefono de un usuario.

- Param usuariold - ID del usuario a modificar.
- Param telefono - nuevo numero de telefono.

CODIGO

```
public String obtenerContrasena(int usuarioId)
```

Retorna la contraseña hashada almacenada para un usuario.

- Param usuariold - ID del usuario a consultar.
- Returns - contraseña hashada, o null si el usuario no existe.

CODIGO

```
public void actualizarContrasena(int usuarioId, String contraseñaHashada)
```

Actualiza la contraseña de un usuario con el nuevo hash proporcionado.

- Param usuariold - ID del usuario a modificar.
- Param contraseñaHashada - nueva contraseña ya procesada con hash.

CODIGO

```
public List<UsuarioAdminDTO> listarTodosConRol()
```

Retorna todos los usuarios del sistema con su rol, ciudad y pais asignados.

- Returns - lista de DTOs con la informacion administrativa de cada usuario.
-

CODIGO

```
public void actualizarRol(int usuarioId, int nuevoRolId)
```

Actualiza el rol asignado a un usuario.

- Param usuariold - ID del usuario a modificar.
 - Param nuevoRolId - ID del nuevo rol a asignar.
-

ESTRUCTURA

Models

Usuario

Modelo que representa un usuario registrado en el sistema. Contiene los datos personales, credenciales de acceso y ubicacion del usuario.

```
CODIGO  
public int getId()
```

- Returns - ID unico del usuario.

```
CODIGO  
public void setId(int id)
```

- Param id - ID unico del usuario.

```
CODIGO  
public String getCorreo()
```

- Returns - correo electronico del usuario.

```
CODIGO  
public void setCorreo(String correo)
```

- Param correo - correo electronico del usuario.

```
CODIGO  
public String getContrasena()
```

- Returns - contrasena hasheada del usuario.

```
CODIGO  
public void setContrasena(String contrasena)
```

- Param contrasena - contrasena hasheada del usuario.

```
CODIGO  
public String getPasaporte()
```

- Returns - numero de pasaporte del usuario.

CODIGO

```
public void setPasaporte(String pasaporte)
```

- Param pasaporte - numero de pasaporte del usuario.

CODIGO

```
public String getUsername()
```

- Returns - nombre de usuario unico para iniciar sesion.

CODIGO

```
public void setUsername(String username)
```

- Param username - nombre de usuario unico para iniciar sesion.

CODIGO

```
public String getNombre()
```

- Returns - nombre de pila del usuario.

CODIGO

```
public void setNombre(String nombre)
```

- Param nombre - nombre de pila del usuario.

CODIGO

```
public String getApellido()
```

- Returns - apellido del usuario.

CODIGO

```
public void setApellido(String apellido)
```

- Param apellido - apellido del usuario.

CODIGO

```
public int getRolId()
```

- Returns - ID del rol asignado al usuario.

CODIGO

```
public void setRolId(int rolId)
```

- Param rolId - ID del rol asignado al usuario.

```
CODIGO  
public String getTelefono()
```

- Returns - numero de telefono del usuario.

```
CODIGO  
public void setTelefono(String telefono)
```

- Param telefono - numero de telefono del usuario.

```
CODIGO  
public void setFechaNacimiento(java.time.LocalDate fechaNacimiento)
```

- Param fechaNacimiento - fecha de nacimiento del usuario.

```
CODIGO  
public Integer getCiudadId()
```

- Returns - ID de la ciudad de residencia del usuario, o null si no fue registrada.

```
CODIGO  
public void setCiudadId(Integer ciudadId)
```

- Param ciudadId - ID de la ciudad de residencia del usuario.
-

DTO

AerolineaAdminDTO

DTO que representa los datos de una aerolinea aliada para el panel de administracion. Incluye el username del usuario webservice propietario para facilitar la gestion. No expone el TokenHASH de autenticacion ya que ese se gestiona internamente.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la aerolinea.

- Returns - ID de la aerolinea.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la aerolinea.

- Returns - nombre de la aerolinea.

```
CODIGO  
public int getUsuarioWebis()
```

Retorna el ID del usuario webservice propietario de la aerolinea.

- Returns - ID del usuario webservice.

```
CODIGO  
public String getUsuarioUsername()
```

Retorna el username del usuario webservice propietario.

- Returns - username del usuario webservice.

```
CODIGO  
public double getPorcentajeDescuento()
```

Retorna el porcentaje de descuento asignado.

- Returns - porcentaje de descuento.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado actual de la aerolinea.

- Returns - ID del estado.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual de la aerolinea.

- Returns - nombre del estado.

```
CODIGO  
public String getUrl()
```

Retorna la URL del sistema externo de la aerolinea.

- Returns - URL del sistema externo.

```
CODIGO  
public String getUrlParaUsuario()
```

Retorna la URL de redireccion para el usuario final.

- Returns - URL para el usuario final.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la aerolinea.

- Param id - ID de la aerolinea.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la aerolinea.

- Param nombre - nombre de la aerolinea.

```
CODIGO  
public void setUsuarioWebis(int usuarioWebis)
```

Asigna el ID del usuario webservice propietario.

- Param usuarioWebis - ID del usuario webservice.

```
CODIGO  
public void setUsuarioUsername(String usuarioUsername)
```


Asigna el username del usuario webservice propietario.

- Param usuarioUsername - username del usuario webservice.

```
CODIGO  
public void setPorcentajeDescuento(double porcentajeDescuento)
```

Asigna el porcentaje de descuento.

- Param porcentajeDescuento - porcentaje de descuento.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del estado actual.

- Param estadoId - ID del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual.

- Param estado - nombre del estado.

```
CODIGO  
public void setUrl(String url)
```

Asigna la URL del sistema externo.

- Param url - URL del sistema externo.

```
CODIGO  
public void setUrlParaUsuario(String urlParaUsuario)
```

Asigna la URL de redirección para el usuario final.

- Param urlParaUsuario - URL para el usuario final.

AerolineaIdentidadDTO

DTO con los datos de identidad de una aerolínea aliada autenticada por token.

```
CODIGO  
public AerolineaIdentidadDTO(int id, String nombre, String urlAerolinea)
```

```
CODIGO  
public int getId()
```

```
CODIGO  
public String getNombre()
```

```
CODIGO  
public String getUrlAerolinea()
```

AerolineaWebserviceDTO

DTO que representa los datos de una aerolinea aliada para el portal webservice. No expone el TokenHASH de autenticacion ya que ese se gestiona internamente. Incluye el nombre del estado resuelto mediante join con la tabla EstadoAliado.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la aerolinea.

- Returns - ID de la aerolinea.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la aerolinea.

- Returns - nombre de la aerolinea.

```
CODIGO  
public int getUsuarioWebis()
```

Retorna el ID del usuario webservice propietario de la aerolinea.

- Returns - ID del usuario webservice.

```
CODIGO  
public double getPorcentajeDescuento()
```

Retorna el porcentaje de descuento asignado por el administrador.

- Returns - porcentaje de descuento.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado actual de la aerolinea.

- Returns - ID del estado.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual resuelto desde la tabla EstadoAliado.

- Returns - nombre del estado.

```
CODIGO  
public String getUrl()
```

Retorna la URL del sistema externo de la aerolinea.

- Returns - URL del sistema externo.

```
CODIGO  
public String getUrlParaUsuario()
```

Retorna la URL de redireccion para usuarios finales.

- Returns - URL para el usuario final.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la aerolinea.

- Param id - ID de la aerolinea.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la aerolinea.

- Param nombre - nombre de la aerolinea.

```
CODIGO  
public void setUsuarioWebis(int usuarioWebis)
```

Asigna el ID del usuario webservice propietario de la aerolinea.

- Param usuarioWebis - ID del usuario webservice.

CODIGO

```
public void setPorcentajeDescuento(double porcentajeDescuento)
```

Asigna el porcentaje de descuento asignado por el administrador.

- Param porcentajeDescuento - porcentaje de descuento.

CODIGO

```
public void setEstadoId(int estadoId)
```

Asigna el ID del estado actual de la aerolinea.

- Param estadoId - ID del estado.

CODIGO

```
public void setEstado(String estado)
```

Asigna el nombre del estado actual resuelto desde la tabla EstadoAliado.

- Param estado - nombre del estado.

CODIGO

```
public void setUrl(String url)
```

Asigna la URL del sistema externo de la aerolinea.

- Param url - URL del sistema externo.

CODIGO

```
public void setUrlParaUsuario(String urlParaUsuario)
```

Asigna la URL de redireccion para usuarios finales.

- Param urlParaUsuario - URL para el usuario final.

AgenciaDTO

DTO que representa los datos de una agencia para transferencia entre capas. Incluye el nombre del estado resuelto mediante join con la tabla ESTADO, y la URL del sistema externo registrada al momento de la creacion.

CODIGO

```
public int getId()
```

Retorna el identificador unico de la agencia.

- Returns - ID de la agencia.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la agencia.

- Returns - nombre de la agencia.

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico de la agencia.

- Returns - correo de la agencia.

```
CODIGO  
public int getUsuarioWebisId()
```

Retorna el ID del usuario webservice asociado a la agencia.

- Returns - ID del usuario webservice.

```
CODIGO  
public double getPorcentajeDescuento()
```

Retorna el porcentaje de descuento aplicado a las reservaciones de la agencia.

- Returns - porcentaje de descuento.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado actual de la agencia.

- Returns - ID del estado.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual resuelto desde la tabla ESTADO.

- Returns - nombre del estado.

```
CODIGO  
public String getUrlAgencia()
```

Retorna la URL del sistema externo de la agencia.

- Returns - URL de la agencia.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la agencia.

- Param id - ID de la agencia.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la agencia.

- Param nombre - nombre de la agencia.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico de la agencia.

- Param correo - correo de la agencia.

```
CODIGO  
public void setUserWebisId(int usuarioWebisId)
```

Asigna el ID del usuario webservice asociado a la agencia.

- Param usuarioWebisId - ID del usuario webservice.

```
CODIGO  
public void setPorcentajeDescuento(double porcentajeDescuento)
```

Asigna el porcentaje de descuento aplicado a las reservaciones de la agencia.

- Param porcentajeDescuento - porcentaje de descuento.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del estado actual de la agencia.

- Param estadoId - ID del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual resuelto desde la tabla ESTADO.

- Param estado - nombre del estado.

```
CODIGO  
public void setUrlAgencia(String urlAgencia)
```

Asigna la URL del sistema externo de la agencia.

- Param urlAgencia - URL del sistema externo.

AgencialIdentidad

DTO que representa la identidad basica de una agencia externa. Se usa para identificar la agencia durante el proceso de handshake.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la agencia.

- Returns - ID de la agencia.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la agencia.

- Param id - ID de la agencia.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la agencia.

- Returns - nombre de la agencia.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la agencia.

- Param nombre - nombre de la agencia.

```
CODIGO  
public String getUrlAgencia()
```

Retorna la URL base del sistema de la agencia externa.

- Returns - URL de la agencia.

```
CODIGO  
public void setUrlAgencia(String urlAgencia)
```

Asigna la URL base del sistema de la agencia externa.

- Param urlAgencia - URL de la agencia.

AgregarAmenidadRequestDTO

DTO con los datos necesarios para agregar o actualizar una amenidad en un hotel.

```
CODIGO  
public int getAmenidadId()
```

Retorna el ID de la amenidad del catalogo a asociar al hotel.

- Returns - ID de la amenidad.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion personalizada de la amenidad para ese hotel.

- Returns - descripcion de la amenidad.

```
CODIGO  
public void setAmenidadId(int amenidadId)
```

Asigna el ID de la amenidad del catalogo a asociar al hotel.

- Param amenidadId - ID de la amenidad.

```
CODIGO  
public void setDescripcion(String descripcion)
```

Asigna la descripcion personalizada de la amenidad para ese hotel.

- Param descripcion - descripcion de la amenidad.

AmenidadDTO

DTO que representa una amenidad del catalogo del sistema. Contiene el identificador y nombre de cada una de las amenidades disponibles.

```
CODIGO  
public AmenityDTO(int id, String nombre)
```

Constructor para crear una amenidad con todos sus datos.

- Param id - identificador unico de la amenidad.
- Param nombre - nombre descriptivo de la amenidad.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la amenidad.

- Returns - ID de la amenidad.

```
CODIGO  
public String getNombre()
```

Retorna el nombre descriptivo de la amenidad.

- Returns - nombre de la amenidad.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la amenidad.

- Param id - ID de la amenidad.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre descriptivo de la amenidad.

- Param nombre - nombre de la amenidad.

AmenidadHotelDTO

DTO que representa una amenidad asignada a un hotel especifico. Incluye los IDs de sus imagenes para consultarlas via GET /imagenes/amenidad/{id}.

```
CODIGO  
public int getHotelAmenidadId()
```

Retorna el ID de la relacion entre el hotel y la amenidad.

- Returns - ID de la relacion hotel-amenidad.

```
CODIGO  
public void setHotelAmenidadId(int hotelAmenidadId)
```

Asigna el ID de la relacion entre el hotel y la amenidad.

- Param hotelAmenidadId - ID de la relacion hotel-amenidad.

```
CODIGO  
public int getAmenidadId()
```

Retorna el ID de la amenidad en el catalogo del sistema.

- Returns - ID de la amenidad.

```
CODIGO  
public void setAmenidadId(int amenidadId)
```

Asigna el ID de la amenidad en el catalogo del sistema.

- Param amenidadId - ID de la amenidad.

```
CODIGO  
public String getNombre()
```

Retorna el nombre de la amenidad.

- Returns - nombre de la amenidad.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre de la amenidad.

- Param nombre - nombre de la amenidad.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion personalizada de la amenidad para este hotel.

- Returns - descripcion de la amenidad.

```
CODIGO  
public void setDescripcion(String descripcion)
```

Asigna la descripcion personalizada de la amenidad para este hotel.

- Param descripcion - descripcion de la amenidad.

```
CODIGO  
public List<Integer> getImagenesIds()
```

Retorna los IDs de imagenes asociadas a esta amenidad.

- Returns - lista de IDs de imagenes.

```
CODIGO  
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de imagenes asociadas a esta amenidad.

- Param imagenesIds - lista de IDs de imagenes.

BusquedaRequestDTO

DTO con los criterios de busqueda de habitaciones disponibles. Las fechas deben enviarse en formato YYYY-MM-DD.

```
CODIGO  
public String getPais()
```

Retorna el pais destino de la busqueda.

- Returns - nombre del pais.

```
CODIGO  
public void setPais(String pais)
```

Asigna el pais destino de la busqueda.

- Param pais - nombre del pais.

```
CODIGO  
public String getCiudad()
```

Retorna la ciudad destino de la busqueda.

- Returns - nombre de la ciudad.

```
CODIGO  
public void setCiudad(String ciudad)
```

Asigna la ciudad destino de la busqueda.

- Param ciudad - nombre de la ciudad.

```
CODIGO  
public String getFechaCheckIn()
```

Retorna la fecha de entrada en formato YYYY-MM-DD.

- Returns - fecha de check-in.

```
CODIGO  
public void setFechaCheckIn(String fechaCheckIn)
```

Asigna la fecha de entrada en formato YYYY-MM-DD.

- Param fechaCheckIn - fecha de check-in.

```
CODIGO  
public String getFechaCheckOut()
```

Retorna la fecha de salida en formato YYYY-MM-DD.

- Returns - fecha de check-out.

```
CODIGO  
public void setFechaCheckOut(String fechaCheckOut)
```

Asigna la fecha de salida en formato YYYY-MM-DD.

- Param fechaCheckOut - fecha de check-out.

```
CODIGO  
public int getCantidadPersonas()
```

Retorna el numero de personas para filtrar habitaciones con capacidad suficiente.

- Returns - cantidad de personas.

```
CODIGO  
public void setCantidadPersonas(int cantidadPersonas)
```

Asigna el numero de personas para filtrar habitaciones con capacidad suficiente.

- Param cantidadPersonas - cantidad de personas.

CambiarContrasenaRequestDTO

DTO con los datos necesarios para cambiar la contraseña de un usuario autenticado.

```
CODIGO  
public String getContrasenaActual()
```

Retorna la contraseña actual del usuario para verificar su identidad.

- Returns - contraseña actual.

```
CODIGO  
public void setContrasenaActual(String contrasenaActual)
```

Asigna la contraseña actual del usuario para verificar su identidad.

- Param contrasenaActual - contraseña actual.

```
CODIGO  
public String getContrasenaNueva()
```

Retorna la nueva contraseña que reemplazara a la actual.

- Returns - nueva contraseña.

```
CODIGO  
public void setContrasenaNueva(String contrasenaNueva)
```

Asigna la nueva contraseña que reemplazara a la actual.

- Param contrasenaNueva - nueva contraseña.

CambiarRolRequestDTO

DTO con el nuevo rol a asignar a un usuario desde el panel de administracion.

```
CODIGO  
public int getRolId()
```

Retorna el ID del rol a asignar al usuario.

- Returns - ID del rol.

```
CODIGO  
public void setRolId(int rolId)
```

Asigna el ID del rol a asignar al usuario.

- Param rolId - ID del rol.

CambiarTelefonoRequestDTO

DTO con el nuevo numero de telefono a asignar al usuario autenticado.

```
CODIGO  
public String getTelefono()
```

Retorna el nuevo numero de telefono del usuario.

- Returns - numero de telefono.

```
CODIGO  
public void setTelefono(String telefono)
```

Asigna el nuevo numero de telefono del usuario.

- Param telefono - numero de telefono.

CancelacionRequestDTO

DTO con el motivo de cancelacion de una reservacion.

```
CODIGO  
public String getMotivoCancelacion()
```

Retorna el motivo por el que se cancela la reservacion.

- Returns - motivo de cancelacion.

```
CODIGO  
public void setMotivoCancelacion(String motivoCancelacion)
```

Asigna el motivo por el que se cancela la reservacion.

- Param motivoCancelacion - motivo de cancelacion.

CiudadDTO

DTO que representa una ciudad del catalogo del sistema. Incluye el nombre del pais al que pertenece resuelto mediante join.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la ciudad.

- Returns - ID de la ciudad.

```
CODIGO  
public String getNombre()
```

Retorna el nombre de la ciudad.

- Returns - nombre de la ciudad.

```
CODIGO  
public int getPaisId()
```

Retorna el ID del pais al que pertenece la ciudad.

- Returns - ID del pais.

```
CODIGO  
public String getPaisNombre()
```

Retorna el nombre del pais al que pertenece la ciudad.

- Returns - nombre del pais.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la ciudad.

- Param id - ID de la ciudad.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre de la ciudad.

- Param nombre - nombre de la ciudad.

```
CODIGO  
public void setPaisId(int paisId)
```

Asigna el ID del pais al que pertenece la ciudad.

- Param paisId - ID del pais.

```
CODIGO  
public void setPaisNombre(String paisNombre)
```

Asigna el nombre del país al que pertenece la ciudad.

- Param paisNombre - nombre del país.

ComentarioRequestDTO

DTO con los datos necesarios para publicar un comentario o respuesta en un hotel. Si comentarioPadreId es null se trata de un comentario raíz; si tiene valor es una respuesta. La resena solo aplica para comentarios raíz, no para respuestas.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel sobre el que se publica el comentario.

- Returns - ID del hotel.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel sobre el que se publica el comentario.

- Param hotelId - ID del hotel.

```
CODIGO  
public Integer getComentarioPadreId()
```

Retorna el ID del comentario padre si es una respuesta, null si es un comentario raíz.

- Returns - ID del comentario padre, o null.

```
CODIGO  
public void setComentarioPadreId(Integer comentarioPadreId)
```

Asigna el ID del comentario padre si es una respuesta, null si es un comentario raíz.

- Param comentarioPadreId - ID del comentario padre, o null.

```
CODIGO  
public Integer getResena()
```

Retorna la puntuación de la resena del hotel, null si el comentario es una respuesta.

- Returns - puntuación de la resena, o null.

```
CODIGO  
public void setResena(Integer resena)
```


Asigna la puntuacion de la resena del hotel, null si el comentario es una respuesta.

- Param resena - puntuacion de la resena, o null.

```
CODIGO  
public String getContenido()
```

Retorna el texto del comentario o respuesta.

- Returns - contenido del comentario.

```
CODIGO  
public void setContenido(String contenido)
```

Asigna el texto del comentario o respuesta.

- Param contenido - contenido del comentario.

ComentarioResponseDTO

DTO con los datos completos de un comentario para retornar al cliente. Incluye informacion del autor, contenido, resena y total de downs recibidos.

```
CODIGO  
public class ComentarioResponseDTO
```

DTO con los datos completos de un comentario para retornar al cliente. Incluye informacion del autor, contenido, resena y total de downs recibidos.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del comentario.

- Returns - ID del comentario.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del comentario.

- Param id - ID del comentario.

```
CODIGO  
public int getUserId()
```

Retorna el ID del usuario que publico el comentario.

- Returns - ID del usuario.

```
CODIGO  
public void setUsuarioId(int usuarioId)
```

Asigna el ID del usuario que publico el comentario.

- Param usuariold - ID del usuario.

```
CODIGO  
public String getUsername()
```

Retorna el nombre de usuario del autor del comentario.

- Returns - username del autor.

```
CODIGO  
public void setUsername(String username)
```

Asigna el nombre de usuario del autor del comentario.

- Param username - username del autor.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel al que pertenece el comentario.

- Returns - ID del hotel.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel al que pertenece el comentario.

- Param hotelId - ID del hotel.

```
CODIGO  
public Integer getComentarioPadreId()
```

Retorna el ID del comentario padre si es una respuesta, null si es un comentario raiz.

- Returns - ID del comentario padre, o null.

```
CODIGO  
public void setComentarioPadreId(Integer comentarioPadreId)
```

Asigna el ID del comentario padre si es una respuesta, null si es un comentario raiz.

- Param comentarioPadreId - ID del comentario padre, o null.

```
CODIGO  
public Integer getResena()
```

Retorna la puntuacion de la resena, null si el comentario es una respuesta.

- Returns - puntuacion de la resena, o null.

```
CODIGO  
public void setResena(Integer resena)
```

Asigna la puntuacion de la resena, null si el comentario es una respuesta.

- Param resena - puntuacion de la resena, o null.

```
CODIGO  
public String getContenido()
```

Retorna el texto del comentario o respuesta.

- Returns - contenido del comentario.

```
CODIGO  
public void setContenido(String contenido)
```

Asigna el texto del comentario o respuesta.

- Param contenido - contenido del comentario.

```
CODIGO  
public String getFecha()
```

Retorna la fecha de publicacion del comentario.

- Returns - fecha del comentario.

```
CODIGO  
public void setFecha(String fecha)
```

Asigna la fecha de publicacion del comentario.

- Param fecha - fecha del comentario.

```
CODIGO  
public int getDowns()
```

Retorna la cantidad total de downs recibidos por el comentario.

- Returns - total de downs.

```
CODIGO  
public void setDowns(int downs)
```

Asigna la cantidad total de downs recibidos por el comentario.

- Param downs - total de downs.

CrearAerolineaAdminRequestDTO

DTO con los datos necesarios para que el administrador cree una aerolinea aliada y la asigne directamente a un usuario webservice existente. El porcentaje de descuento inicia en 0 y el token se genera al conectar.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la nueva aerolinea.

- Returns - nombre de la aerolinea.

```
CODIGO  
public String getUrl()
```

Retorna la URL del sistema externo de la nueva aerolinea.

- Returns - URL del sistema externo.

```
CODIGO  
public String getUrlParaUsuario()
```

Retorna la URL de redireccion para el usuario final.

- Returns - URL para el usuario final.

```
CODIGO  
public int getUsuarioWebisId()
```

Retorna el ID del usuario webservice asignado a la aerolinea.

- Returns - ID del usuario webservice.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la nueva aerolinea.

- Param nombre - nombre de la aerolinea.

```
CODIGO  
public void setUrl(String url)
```

Asigna la URL del sistema externo de la nueva aerolinea.

- Param url - URL del sistema externo.

```
CODIGO  
public void setUrlParaUsuario(String urlParaUsuario)
```

Asigna la URL de redireccion para el usuario final.

- Param urlParaUsuario - URL para el usuario final.

```
CODIGO  
public void setUsuarioWebisId(int usuarioWebisId)
```

Asigna el ID del usuario webservice que sera propietario de la aerolinea.

- Param usuarioWebisId - ID del usuario webservice.

CrearAerolineaRequestDTO

DTO con los datos necesarios para registrar una nueva aerolinea aliada desde el portal webservice. El porcentaje de descuento siempre inicia en 0% y solo el administrador puede modificarlo. El token de autenticacion se genera automaticamente al establecer la conexion.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la nueva aerolinea.

- Returns - nombre de la aerolinea.

```
CODIGO  
public String getUrl()
```

Retorna la URL del sistema externo de la aerolinea.

- Returns - URL del sistema externo.

```
CODIGO  
public String getUrlParaUsuario()
```

Retorna la URL de redireccion para usuarios finales.

- Returns - URL para el usuario final.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la nueva aerolinea.

- Param nombre - nombre de la aerolinea.

```
CODIGO  
public void setUrl(String url)
```

Asigna la URL del sistema externo de la aerolinea.

- Param url - URL del sistema externo.

```
CODIGO  
public void setUrlParaUsuario(String urlParaUsuario)
```

Asigna la URL de redireccion para usuarios finales.

- Param urlParaUsuario - URL para el usuario final.

CrearAgenciaAdminRequestDTO

DTO con los datos necesarios para que el administrador cree una agencia y la asigne directamente a un usuario webservice existente. El porcentaje de descuento inicia en 0 y los tokens se generan al conectar.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la nueva agencia.

- Returns - nombre de la agencia.

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico de la nueva agencia.

- Returns - correo de la agencia.

```
CODIGO  
public String getUrlAgencia()
```

Retorna la URL del sistema externo de la nueva agencia.

- Returns - URL de la agencia.

```
CODIGO  
public int getUsuarioWebisId()
```

Retorna el ID del usuario webservice asignado a la agencia.

- Returns - ID del usuario webservice.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la nueva agencia.

- Param nombre - nombre de la agencia.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico de la nueva agencia.

- Param correo - correo de la agencia.

```
CODIGO  
public void setUrlAgencia(String urlAgencia)
```

Asigna la URL del sistema externo de la nueva agencia.

- Param urlAgencia - URL de la agencia.

```
CODIGO  
public void setUsuarioWebisId(int usuarioWebisId)
```

Asigna el ID del usuario webservice que sera propietario de la agencia.

- Param usuarioWebisId - ID del usuario webservice.

CrearAgenciaRequestDTO

DTO con los datos necesarios para crear una nueva agencia. El porcentaje de descuento siempre inicia en 0% y solo el administrador puede modificarlo. Los tokens de autenticacion se generan automaticamente al establecer la conexion con handshake.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial de la nueva agencia.

- Returns - nombre de la agencia.

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico de la nueva agencia.

- Returns - correo de la agencia.

```
CODIGO  
public String getUrlAgencia()
```

Retorna la URL del sistema externo de la nueva agencia.

- Returns - URL de la agencia.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial de la nueva agencia.

- Param nombre - nombre de la agencia.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico de la nueva agencia.

- Param correo - correo de la agencia.

```
CODIGO  
public void setUrlAgencia(String urlAgencia)
```

Asigna la URL del sistema externo de la nueva agencia.

- Param urlAgencia - URL del sistema externo.

CrearHabitacionRequestDTO

DTO con los datos necesarios para crear una nueva habitacion en un hotel.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel al que pertenece la habitacion.

- Returns - ID del hotel.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel al que pertenece la habitacion.

- Param hotelId - ID del hotel.

```
CODIGO  
public int getTipoHabitacionId()
```

Retorna el ID del tipo de habitacion segun el catalogo del sistema.

- Returns - ID del tipo de habitacion.

```
CODIGO  
public void setTipoHabitacionId(int tipoHabitacionId)
```

Asigna el ID del tipo de habitacion segun el catalogo del sistema.

- Param tipoHabitacionId - ID del tipo de habitacion.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion detallada de la habitacion.

- Returns - descripcion de la habitacion.

```
CODIGO  
public void setDescripcion(String descripcion)
```

Asigna la descripcion detallada de la habitacion.

- Param descripcion - descripcion de la habitacion.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado inicial de la habitacion.

- Returns - ID del estado.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del estado inicial de la habitacion.

- Param estadold - ID del estado.

CrearHotelRequestDTO

DTO con los datos necesarios para crear un nuevo hotel en el sistema. Si la ciudad o el pais indicados no existen, el servicio los crea automaticamente.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del hotel.

- Returns - nombre del hotel.

```
CODIGO  
public String getDireccion()
```

Retorna la direccion fisica del hotel.

- Returns - direccion del hotel.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion general del hotel.

- Returns - descripcion del hotel.

```
CODIGO  
public double getRating()
```

Retorna la calificacion inicial del hotel.

- Returns - rating del hotel.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado inicial del hotel.

- Returns - ID del estado.

```
CODIGO  
public String getCiudad()
```

Retorna el nombre de la ciudad donde se ubica el hotel.

- Returns - nombre de la ciudad.

```
CODIGO  
public String getPaisNombre()
```

Retorna el nombre del pais donde se ubica el hotel.

- Returns - nombre del pais.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre del hotel.

- Param nombre - nombre del hotel.

```
CODIGO  
public void setDireccion(String direccion)
```

Asigna la direccion fisica del hotel.

- Param direccion - direccion del hotel.

```
CODIGO  
public void setDescription(String descripcion)
```

Asigna la descripcion general del hotel.

- Param descripcion - descripcion del hotel.

```
CODIGO  
public void setRating(double rating)
```

Asigna la calificacion inicial del hotel.

- Param rating - rating del hotel.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del estado inicial del hotel.

- Param estadold - ID del estado.

```
CODIGO  
public void setCiudad(String ciudad)
```

Asigna el nombre de la ciudad donde se ubica el hotel.

- Param ciudad - nombre de la ciudad.

```
CODIGO  
public void setPaisNombre(String paisNombre)
```

Asigna el nombre del pais donde se ubica el hotel.

- Param paisNombre - nombre del pais.

DownRequestDTO

DTO con el valor de un down aplicado a un comentario. El valor debe ser 1 para down positivo o -1 para down negativo.

```
CODIGO  
public int getValor()
```

Retorna el valor del down, que puede ser 1 o -1.

- Returns - valor del down.

```
CODIGO  
public void setValor(int valor)
```

Asigna el valor del down, que puede ser 1 o -1.

- Param valor - valor del down.

DownResponseDTO

DTO con los datos completos de un down para retornar al cliente. Incluye informacion del comentario valorado y el hotel al que pertenece.

```
CODIGO  
public class DownResponseDTO
```

DTO con los datos completos de un down para retornar al cliente. Incluye informacion del comentario valorado y el hotel al que pertenece.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del down.

- Returns - ID del down.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del down.

- Param id - ID del down.

```
CODIGO  
public int getComentarioId()
```

Retorna el ID del comentario sobre el que se aplico el down.

- Returns - ID del comentario.

```
CODIGO  
public void setComentarioId(int comentarioId)
```

Asigna el ID del comentario sobre el que se aplico el down.

- Param comentarioId - ID del comentario.

```
CODIGO  
public int getValor()
```

Retorna el valor del down, que puede ser 1 o -1.

- Returns - valor del down.

```
CODIGO  
public void setValor(int valor)
```

Asigna el valor del down, que puede ser 1 o -1.

- Param valor - valor del down.

```
CODIGO  
public String getFecha()
```

Retorna la fecha en que se registro el down.

- Returns - fecha del down.

```
CODIGO  
public void setFecha(String fecha)
```

Asigna la fecha en que se registro el down.

- Param fecha - fecha del down.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel al que pertenece el comentario valorado.

- Returns - ID del hotel.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel al que pertenece el comentario valorado.

- Param hotelId - ID del hotel.

```
CODIGO  
public String getContenidoComentario()
```

Retorna el texto del comentario sobre el que se aplico el down.

- Returns - contenido del comentario.

```
CODIGO  
public void setContenidoComentario(String contenidoComentario)
```

Asigna el texto del comentario sobre el que se aplico el down.

- Param contenidoComentario - contenido del comentario.

EditarAerolineaRequestDTO

DTO con los datos editables de una aerolinea aliada desde el panel de administracion. Permite modificar nombre, URLs, porcentaje de descuento y estado.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial actualizado de la aerolinea.

- Returns - nombre de la aerolinea.

```
CODIGO  
public String getUrl()
```

Retorna la URL del sistema externo actualizada.

- Returns - URL del sistema externo.

```
CODIGO  
public String getUrlParaUsuario()
```

Retorna la URL de redireccion para el usuario final actualizada.

- Returns - URL para el usuario final.

```
CODIGO  
public double getPorcentajeDescuento()
```

Retorna el porcentaje de descuento actualizado.

- Returns - porcentaje de descuento.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del nuevo estado de la aerolinea.

- Returns - ID del estado.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial actualizado de la aerolinea.

- Param nombre - nombre de la aerolinea.

```
CODIGO  
public void setUrl(String url)
```

Asigna la URL del sistema externo actualizada.

- Param url - URL del sistema externo.

```
CODIGO  
public void setUrlParaUsuario(String urlParaUsuario)
```

Asigna la URL de redireccion para el usuario final actualizada.

- Param urlParaUsuario - URL para el usuario final.

```
CODIGO  
public void setPorcentajeDescuento(double porcentajeDescuento)
```

Asigna el porcentaje de descuento actualizado.

- Param porcentajeDescuento - porcentaje de descuento.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del nuevo estado de la aerolinea.

- Param estadoId - ID del estado.

EditarAgenciaRequestDTO

DTO con los datos editables de una agencia desde el panel de administracion. Permite modificar nombre, correo, URL del sistema externo, porcentaje de descuento y estado.

```
CODIGO  
public String getNombre()
```

Retorna el nombre comercial actualizado de la agencia.

- Returns - nombre de la agencia.

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico actualizado de la agencia.

- Returns - correo de la agencia.

```
CODIGO  
public String getUrlAgencia()
```

Retorna la URL del sistema externo actualizada de la agencia.

- Returns - URL del sistema externo.

```
CODIGO  
public double getPorcentajeDescuento()
```

Retorna el porcentaje de descuento a aplicar en reservaciones de la agencia.

- Returns - porcentaje de descuento.

```
CODIGO  
public int getEstadoId()
```


Retorna el ID del nuevo estado de la agencia.

- Returns - ID del estado.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre comercial actualizado de la agencia.

- Param nombre - nombre de la agencia.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico actualizado de la agencia.

- Param correo - correo de la agencia.

```
CODIGO  
public void setUrlAgencia(String urlAgencia)
```

Asigna la URL del sistema externo actualizada de la agencia.

- Param urlAgencia - URL del sistema externo.

```
CODIGO  
public void setPorcentajeDescuento(double porcentajeDescuento)
```

Asigna el porcentaje de descuento a aplicar en reservaciones de la agencia.

- Param porcentajeDescuento - porcentaje de descuento.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del nuevo estado de la agencia.

- Param estadoId - ID del estado.

EditarHabitacionRequestDTO

DTO con los datos editables de una habitacion desde el panel de administracion.

```
CODIGO  
public int getTipoHabitacionId()
```

Retorna el ID del tipo de habitacion segun el catalogo del sistema.

- Returns - ID del tipo de habitacion.

```
CODIGO  
public String getNumeroHabitacion()
```

Retorna el numero o identificador de la habitacion dentro del hotel.

- Returns - numero de la habitacion.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion actualizada de la habitacion.

- Returns - descripcion de la habitacion.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del nuevo estado de la habitacion.

- Returns - ID del estado.

```
CODIGO  
public void setTipoHabitacionId(int tipoHabitacionId)
```

Asigna el ID del tipo de habitacion segun el catalogo del sistema.

- Param tipoHabitacionId - ID del tipo de habitacion.

```
CODIGO  
public void setNumeroHabitacion(String numeroHabitacion)
```

Asigna el numero o identificador de la habitacion dentro del hotel.

- Param numeroHabitacion - numero de la habitacion.

```
CODIGO  
public void setDescripcion(String descripcion)
```

Asigna la descripcion actualizada de la habitacion.

- Param descripcion - descripcion de la habitacion.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del nuevo estado de la habitacion.

- Param estadold - ID del estado.

EditarHotelRequestDTO

DTO con los datos editables de un hotel desde el panel de administracion.

```
CODIGO  
public String getNombre()
```

Retorna el nombre actualizado del hotel.

- Returns - nombre del hotel.

```
CODIGO  
public String getDireccion()
```

Retorna la direccion fisica actualizada del hotel.

- Returns - direccion del hotel.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion actualizada del hotel.

- Returns - descripcion del hotel.

```
CODIGO  
public double getRating()
```

Retorna la calificacion actualizada del hotel.

- Returns - rating del hotel.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del nuevo estado del hotel.

- Returns - ID del estado.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre actualizado del hotel.

- Param nombre - nombre del hotel.

```
CODIGO  
public void setDireccion(String direccion)
```

Asigna la direccion fisica actualizada del hotel.

- Param direccion - direccion del hotel.

```
CODIGO  
public void setDescription(String descripcion)
```

Asigna la descripcion actualizada del hotel.

- Param descripcion - descripcion del hotel.

```
CODIGO  
public void setRating(double rating)
```

Asigna la calificacion actualizada del hotel.

- Param rating - calificacion del hotel.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del nuevo estado del hotel.

- Param estadoId - ID del estado.

HabitacionAdminDTO

DTO con los datos completos de una habitacion para el panel de administracion. Incluye campos propios de la habitacion y campos resueltos mediante join con TipoHabitacion.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la habitacion.

- Returns - ID de la habitacion.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel al que pertenece la habitacion.

- Returns - ID del hotel.

```
CODIGO  
public int getTipoHabitacionId()
```

Retorna el ID del tipo de habitacion en el catalogo.

- Returns - ID del tipo de habitacion.

```
CODIGO  
public String getTipoHabitacion()
```

Retorna el nombre del tipo de habitacion resuelto desde TipoHabitacion.

- Returns - nombre del tipo de habitacion.

```
CODIGO  
public String getNumeroHabitacion()
```

Retorna el numero o identificador de la habitacion dentro del hotel.

- Returns - numero de la habitacion.

```
CODIGO  
public String getTipoCama()
```

Retorna el tipo de cama de la habitacion resuelto desde TipoHabitacion.

- Returns - tipo de cama.

```
CODIGO  
public double getPrecioPorPersona()
```

Retorna el precio adicional por persona extra sobre la capacidad base.

- Returns - precio por persona adicional.

```
CODIGO  
public double getPrecioPorNoche()
```

Retorna el precio base por noche de la habitacion.

- Returns - precio por noche.

```
CODIGO  
public int getCapacidadMaxima()
```

Retorna la capacidad maxima de personas admitidas en la habitacion.

- Returns - capacidad maxima.

```
CODIGO  
public double getMetrosCuadrados()
```

Retorna la superficie de la habitacion en metros cuadrados.

- Returns - metros cuadrados de la habitacion.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion detallada de la habitacion.

- Returns - descripcion de la habitacion.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado actual de la habitacion.

- Returns - ID del estado.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual de la habitacion.

- Returns - nombre del estado.

```
CODIGO  
public List<Integer> getImagenesIds()
```

Retorna los IDs de imagenes asociadas a la habitacion.

- Returns - lista de IDs de imagenes.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la habitacion.

- Param id - ID de la habitacion.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel al que pertenece la habitacion.

- Param hotelId - ID del hotel.

```
CODIGO  
public void setTipoHabitacionId(int tipoHabitacionId)
```

Asigna el ID del tipo de habitacion en el catalogo.

- Param tipoHabitacionId - ID del tipo de habitacion.

```
CODIGO  
public void setTipoHabitacion(String tipoHabitacion)
```

Asigna el nombre del tipo de habitacion resuelto desde TipoHabitacion.

- Param tipoHabitacion - nombre del tipo de habitacion.

```
CODIGO  
public void setNumeroHabitacion(String numeroHabitacion)
```

Asigna el numero o identificador de la habitacion dentro del hotel.

- Param numeroHabitacion - numero de la habitacion.

```
CODIGO  
public void setTipoCama(String tipoCama)
```

Asigna el tipo de cama de la habitacion resuelto desde TipoHabitacion.

- Param tipoCama - tipo de cama.

```
CODIGO  
public void setPrecioPorPersona(double precioPorPersona)
```

Asigna el precio adicional por persona extra sobre la capacidad base.

- Param precioPorPersona - precio por persona adicional.

```
CODIGO  
public void setPrecioPorNoche(double precioPorNoche)
```

Asigna el precio base por noche de la habitacion.

- Param precioPorNoche - precio por noche.

```
CODIGO  
public void setCapacidadMaxima(int capacidadMaxima)
```

Asigna la capacidad maxima de personas admitidas en la habitacion.

- Param capacidadMaxima - capacidad maxima.

```
CODIGO  
public void setMetrosCuadrados(double metrosCuadrados)
```

Asigna la superficie de la habitacion en metros cuadrados.

- Param metrosCuadrados - metros cuadrados de la habitacion.

```
CODIGO  
public void setDescription(String descripcion)
```

Asigna la descripcion detallada de la habitacion.

- Param descripcion - descripcion de la habitacion.

```
CODIGO  
public void setEstadoId(int estadoId)
```

Asigna el ID del estado actual de la habitacion.

- Param estadoId - ID del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual de la habitacion.

- Param estado - nombre del estado.

```
CODIGO  
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de imagenes asociadas a la habitacion.

- Param imagenesIds - lista de IDs de imagenes.

HabitacionAgenciaDTO

DTO que extiende HabitacionDTO con los precios ajustados por el descuento negociado para la agencia solicitante.

```
CODIGO  
public double getPorcentajeDescuento()
```


Retorna el porcentaje de descuento aplicado a los precios de la habitacion.

- Returns - porcentaje de descuento.

```
CODIGO  
public void setPorcentajeDescuento(double porcentajeDescuento)
```

Asigna el porcentaje de descuento aplicado a los precios de la habitacion.

- Param porcentajeDescuento - porcentaje de descuento.

```
CODIGO  
public double getPrecioPorNocheConDescuento()
```

Retorna el precio por noche luego de aplicar el descuento de la agencia.

- Returns - precio por noche con descuento.

```
CODIGO  
public void setPrecioPorNocheConDescuento(double precioPorNocheConDescuento)
```

Asigna el precio por noche luego de aplicar el descuento de la agencia.

- Param precioPorNocheConDescuento - precio por noche con descuento.

```
CODIGO  
public double getPrecioPorPersonaConDescuento()
```

Retorna el precio por persona luego de aplicar el descuento de la agencia.

- Returns - precio por persona con descuento.

```
CODIGO  
public void setPrecioPorPersonaConDescuento(double precioPorPersonaConDescuento)
```

Asigna el precio por persona luego de aplicar el descuento de la agencia.

- Param precioPorPersonaConDescuento - precio por persona con descuento.

HabitacionAgenciaResponseDTO

DTO con el desglose de precios de una habitacion calculado para una agencia. Incluye el total a pagar considerando noches y personas extra.

```
CODIGO  
public class HabitacionAgenciaResponseDTO
```

DTO con el desglose de precios de una habitacion calculado para una agencia. Incluye el total a pagar considerando noches y personas extra.

CODIGO

```
public int getHabitacionId()
```

Retorna el ID de la habitacion cotizada.

- Returns - ID de la habitacion.

CODIGO

```
public void setHabitacionId(int habitacionId)
```

Asigna el ID de la habitacion cotizada.

- Param habitacionId - ID de la habitacion.

CODIGO

```
public double getPrecioPorNoche()
```

Retorna el precio base por noche aplicado a la reservacion.

- Returns - precio por noche.

CODIGO

```
public void setPrecioPorNoche(double precioPorNoche)
```

Asigna el precio base por noche aplicado a la reservacion.

- Param precioPorNoche - precio por noche.

CODIGO

```
public double getPrecioPorPersona()
```

Retorna el precio adicional por persona extra sobre la capacidad base.

- Returns - precio por persona adicional.

CODIGO

```
public void setPrecioPorPersona(double precioPorPersona)
```

Asigna el precio adicional por persona extra sobre la capacidad base.

- Param precioPorPersona - precio por persona adicional.

CODIGO

```
public int getPersonasExtra()
```

Retorna la cantidad de personas extra sobre la capacidad base de la habitacion.

- Returns - personas extra.

```
CODIGO  
public void setPersonasExtra(int personasExtra)
```

Asigna la cantidad de personas extra sobre la capacidad base de la habitacion.

- Param personasExtra - personas extra.

```
CODIGO  
public int getNoches()
```

Retorna el numero de noches de la estancia.

- Returns - cantidad de noches.

```
CODIGO  
public void setNoches(int noches)
```

Asigna el numero de noches de la estancia.

- Param noches - cantidad de noches.

```
CODIGO  
public double getTotal()
```

Retorna el monto total a pagar considerando noches, personas extra y descuento de agencia.

- Returns - total a pagar.

```
CODIGO  
public void setTotal(double total)
```

Asigna el monto total a pagar considerando noches, personas extra y descuento de agencia.

- Param total - monto total a pagar.

HabitacionDTO

DTO con los datos de una habitacion para mostrar en resultados de busqueda y detalle de hotel. Los IDs de imagenes se usan para consultarlas via GET /imagenes/habitacion/{id}.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la habitacion.

- Returns - ID de la habitacion.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la habitacion.

- Param id - ID de la habitacion.

```
CODIGO  
public String getTipoHabitacion()
```

Retorna el nombre del tipo de habitacion.

- Returns - tipo de habitacion.

```
CODIGO  
public void setTipoHabitacion(String tipoHabitacion)
```

Asigna el nombre del tipo de habitacion.

- Param tipoHabitacion - tipo de habitacion.

```
CODIGO  
public double getPrecioPorPersona()
```

Retorna el precio adicional por persona extra sobre la capacidad base.

- Returns - precio por persona adicional.

```
CODIGO  
public void setPrecioPorPersona(double precioPorPersona)
```

Asigna el precio adicional por persona extra sobre la capacidad base.

- Param precioPorPersona - precio por persona adicional.

```
CODIGO  
public double getPrecioPorNoche()
```

Retorna el precio base por noche de la habitacion.

- Returns - precio por noche.

```
CODIGO  
public void setPrecioPorNoche(double precioPorNoche)
```

Asigna el precio base por noche de la habitacion.

- Param precioPorNoche - precio por noche.

```
CODIGO  
public int getCapacidadMaxima()
```

Retorna la capacidad maxima de personas admitidas en la habitacion.

- Returns - capacidad maxima.

```
CODIGO  
public void setCapacidadMaxima(int capacidadMaxima)
```

Asigna la capacidad maxima de personas admitidas en la habitacion.

- Param capacidadMaxima - capacidad maxima.

```
CODIGO  
public String getTipoCama()
```

Retorna el tipo de cama disponible en la habitacion.

- Returns - tipo de cama.

```
CODIGO  
public void setTipoCama(String tipoCama)
```

Asigna el tipo de cama disponible en la habitacion.

- Param tipoCama - tipo de cama.

```
CODIGO  
public double getMetrosCuadrados()
```

Retorna la superficie de la habitacion en metros cuadrados.

- Returns - metros cuadrados de la habitacion.

```
CODIGO  
public void setMetrosCuadrados(double metrosCuadrados)
```

Asigna la superficie de la habitacion en metros cuadrados.

- Param metrosCuadrados - metros cuadrados de la habitacion.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion detallada de la habitacion.

- Returns - descripcion de la habitacion.

```
CODIGO  
public void setDescripcion(String descripcion)
```

Asigna la descripcion detallada de la habitacion.

- Param descripcion - descripcion de la habitacion.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual de la habitacion.

- Returns - nombre del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual de la habitacion.

- Param estado - nombre del estado.

```
CODIGO  
public List<Integer> getImagenesIds()
```

Retorna los IDs de imagenes asociadas a la habitacion.

- Returns - lista de IDs de imagenes.

```
CODIGO  
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de imagenes asociadas a la habitacion.

- Param imagenesIds - lista de IDs de imagenes.

HabitacionReservaRequestDTO

DTO con los datos de una habitacion dentro de una solicitud de reservacion. Las fechas deben enviarse en formato YYYY-MM-DD.

```
CODIGO  
public int getHabitacionId()
```

Retorna el ID de la habitacion a reservar.

- Returns - ID de la habitacion.

```
CODIGO  
public void setHabitacionId(int habitacionId)
```

Asigna el ID de la habitacion a reservar.

- Param habitacionId - ID de la habitacion.

```
CODIGO  
public int getCantidadPersonas()
```

Retorna el numero de personas que ocuparan la habitacion.

- Returns - cantidad de personas.

```
CODIGO  
public void setCantidadPersonas(int cantidadPersonas)
```

Asigna el numero de personas que ocuparan la habitacion.

- Param cantidadPersonas - cantidad de personas.

```
CODIGO  
public String getFechaCheckIn()
```

Retorna la fecha de entrada en formato YYYY-MM-DD.

- Returns - fecha de check-in.

```
CODIGO  
public void setFechaCheckIn(String fechaCheckIn)
```

Asigna la fecha de entrada en formato YYYY-MM-DD.

- Param fechaCheckIn - fecha de check-in.

```
CODIGO  
public String getFechaCheckOut()
```

Retorna la fecha de salida en formato YYYY-MM-DD.

- Returns - fecha de check-out.

```
CODIGO  
public void setFechaCheckOut(String fechaCheckOut)
```

Asigna la fecha de salida en formato YYYY-MM-DD.

- Param fechaCheckOut - fecha de check-out.

HabitacionResumenDTO

DTO con los datos minimos de una habitacion para usarse en listados y resúmenes.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la habitacion.

- Returns - ID de la habitacion.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la habitacion.

- Param id - ID de la habitacion.

```
CODIGO  
public String getNumeroHabitacion()
```

Retorna el numero o identificador de la habitacion dentro del hotel.

- Returns - numero de la habitacion.

```
CODIGO  
public void setNumeroHabitacion(String numeroHabitacion)
```

Asigna el numero o identificador de la habitacion dentro del hotel.

- Param numeroHabitacion - numero de la habitacion.

HandshakeRequestDTO

DTO con los datos enviados por una agencia externa para iniciar el proceso de handshake. Los campos se mapean desde snake_case del JSON mediante anotaciones de Jackson.

```
CODIGO  
public String getTokenEntrada()
```

Retorna el token de entrada proporcionado por la agencia para autenticarse.

- Returns - token de entrada.

CODIGO

```
public void setTokenEntrada(String tokenEntrada)
```

Asigna el token de entrada proporcionado por la agencia para autenticarse.

- Param tokenEntrada - token de entrada.

CODIGO

```
public String getUrlAgencia()
```

Retorna la URL base del sistema de la agencia externa.

- Returns - URL de la agencia.

CODIGO

```
public void setUrlAgencia(String urlAgencia)
```

Asigna la URL base del sistema de la agencia externa.

- Param urlAgencia - URL de la agencia.

HandshakeResponseDTO

DTO con la respuesta del servidor al proceso de handshake de una agencia externa. Contiene el token de salida que la agencia debera usar en peticiones posteriores.

CODIGO

```
public class HandshakeResponseDTO
```

DTO con la respuesta del servidor al proceso de handshake de una agencia externa. Contiene el token de salida que la agencia debera usar en peticiones posteriores.

CODIGO

```
public HandshakeResponseDTO(String tokenSalida)
```

Constructor que inicializa la respuesta con el token generado por el servidor.

- Param tokenSalida - token de salida que la agencia usara para autenticarse en siguientes peticiones.

CODIGO

```
public String getTokenSalida()
```

Retorna el token de salida generado por el servidor para la agencia.

- Returns - token de salida.

```
CODIGO  
public void setTokenSalida(String tokenSalida)
```

Asigna el token de salida generado por el servidor para la agencia.

- Param tokenSalida - token de salida.

HotelAdminDTO

DTO con los datos completos de un hotel para el panel de administracion. Incluye informacion de ubicacion, estado, cantidad de habitaciones e imagenes asociadas.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del hotel.

- Returns - ID del hotel.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del hotel.

- Returns - nombre del hotel.

```
CODIGO  
public String getDireccion()
```

Retorna la direccion fisica del hotel.

- Returns - direccion del hotel.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion general del hotel.

- Returns - descripcion del hotel.

```
CODIGO  
public double getRating()
```

Retorna la calificacion promedio del hotel.

- Returns - rating del hotel.

```
CODIGO  
public int getEstadoId()
```

Retorna el ID del estado actual del hotel.

- Returns - ID del estado.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual del hotel.

- Returns - nombre del estado.

```
CODIGO  
public String getCiudad()
```

Retorna la ciudad donde se ubica el hotel.

- Returns - nombre de la ciudad.

```
CODIGO  
public String getPais()
```

Retorna el pais donde se ubica el hotel.

- Returns - nombre del pais.

```
CODIGO  
public int getCantidadHabitaciones()
```

Retorna el total de habitaciones registradas en el hotel.

- Returns - cantidad de habitaciones.

```
CODIGO  
public List<Integer> getImagenesIds()
```

Retorna los IDs de imagenes asociadas al hotel.

- Returns - lista de IDs de imagenes.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del hotel.

- Param id - ID del hotel.

CODIGO

```
public void setNombre(String nombre)
```

Asigna el nombre del hotel.

- Param nombre - nombre del hotel.

CODIGO

```
public void setDireccion(String direccion)
```

Asigna la direccion fisica del hotel.

- Param direccion - direccion del hotel.

CODIGO

```
public void setDescripcion(String descripcion)
```

Asigna la descripcion general del hotel.

- Param descripcion - descripcion del hotel.

CODIGO

```
public void setRating(double rating)
```

Asigna la calificacion promedio del hotel.

- Param rating - calificacion del hotel.

CODIGO

```
public void setEstadoId(int estadoId)
```

Asigna el ID del estado actual del hotel.

- Param estadoId - ID del estado.

CODIGO

```
public void setEstado(String estado)
```

Asigna el nombre del estado actual del hotel.

- Param estado - nombre del estado.

CODIGO

```
public void setCiudad(String ciudad)
```

Asigna la ciudad donde se ubica el hotel.

- Param ciudad - nombre de la ciudad.

```
CODIGO  
public void setPais(String pais)
```

Asigna el pais donde se ubica el hotel.

- Param pais - nombre del pais.

```
CODIGO  
public void setCantidadHabitaciones(int cantidadHabitaciones)
```

Asigna el total de habitaciones registradas en el hotel.

- Param cantidadHabitaciones - total de habitaciones.

```
CODIGO  
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de imagenes asociadas al hotel.

- Param imagenesIds - lista de IDs de imagenes.

HotelAgenciaDTO

DTO con los datos basicos de un hotel para mostrar en el catalogo de agencias externas.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del hotel.

- Returns - ID del hotel.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del hotel.

- Param id - ID del hotel.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del hotel.

- Returns - nombre del hotel.

CODIGO

```
public void setNombre(String nombre)
```

Asigna el nombre del hotel.

- Param nombre - nombre del hotel.

CODIGO

```
public String getCiudad()
```

Retorna la ciudad donde se ubica el hotel.

- Returns - nombre de la ciudad.

CODIGO

```
public void setCiudad(String ciudad)
```

Asigna la ciudad donde se ubica el hotel.

- Param ciudad - nombre de la ciudad.

CODIGO

```
public String getPais()
```

Retorna el pais donde se ubica el hotel.

- Returns - nombre del pais.

CODIGO

```
public void setPais(String pais)
```

Asigna el pais donde se ubica el hotel.

- Param pais - nombre del pais.

HotelAmenidadDTO

DTO que representa una amenidad asignada a un hotel específico. Incluye la descripción personalizada y los IDs de imágenes de la tabla ImagenHotelAmenidad.

CODIGO

```
public int getId()
```

Retorna el ID del registro en la tabla HotelAmenidad.

- Returns - ID del registro.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel al que pertenece la amenidad.

- Returns - ID del hotel.

```
CODIGO  
public int getAmenidadId()
```

Retorna el ID de la amenidad en el catalogo del sistema.

- Returns - ID de la amenidad.

```
CODIGO  
public String getAmenidadNombre()
```

Retorna el nombre de la amenidad resuelto desde el catalogo.

- Returns - nombre de la amenidad.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion personalizada de la amenidad para este hotel.

- Returns - descripcion de la amenidad.

```
CODIGO  
public List<Integer> getImagenesIds()
```

Retorna los IDs de imagenes asociadas a esta amenidad en el hotel.

- Returns - lista de IDs de imagenes.

```
CODIGO  
public void setId(int id)
```

Asigna el ID del registro en la tabla HotelAmenidad.

- Param id - ID del registro.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel al que pertenece la amenidad.

- Param hotelId - ID del hotel.

CODIGO

```
public void setAmenidadId(int amenidadId)
```

Asigna el ID de la amenidad en el catalogo del sistema.

- Param amenidadId - ID de la amenidad.

CODIGO

```
public void setAmenidadNombre(String amenidadNombre)
```

Asigna el nombre de la amenidad resuelto desde el catalogo.

- Param amenidadNombre - nombre de la amenidad.

CODIGO

```
public void setDescripcion(String descripcion)
```

Asigna la descripcion personalizada de la amenidad para este hotel.

- Param descripcion - descripcion de la amenidad.

CODIGO

```
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de imagenes asociadas a esta amenidad en el hotel.

- Param imagenesIds - lista de IDs de imagenes.

HotelResultadoDTO

DTO con los datos completos de un hotel como resultado de una busqueda. Incluye amenidades, tipos de habitacion disponibles, combinaciones numericas y tipos de habitacion agrupados por capacidad para facilitar la seleccion al usuario.

CODIGO

```
public int getId()
```

Retorna el identificador unico del hotel.

- Returns - ID del hotel.

CODIGO

```
public void setId(int id)
```

Asigna el identificador unico del hotel.

- Param id - ID del hotel.


```
CODIGO  
public String getNombre()
```

Retorna el nombre del hotel.

- Returns - nombre del hotel.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre del hotel.

- Param nombre - nombre del hotel.

```
CODIGO  
public String getDireccion()
```

Retorna la direccion fisica del hotel.

- Returns - direccion del hotel.

```
CODIGO  
public void setDireccion(String direccion)
```

Asigna la direccion fisica del hotel.

- Param direccion - direccion del hotel.

```
CODIGO  
public String getCiudad()
```

Retorna la ciudad donde se ubica el hotel.

- Returns - nombre de la ciudad.

```
CODIGO  
public void setCiudad(String ciudad)
```

Asigna la ciudad donde se ubica el hotel.

- Param ciudad - nombre de la ciudad.

```
CODIGO  
public String getPais()
```

Retorna el pais donde se ubica el hotel.

- Returns - nombre del pais.

```
CODIGO  
public void setPais(String pais)
```

Asigna el pais donde se ubica el hotel.

- Param pais - nombre del pais.

```
CODIGO  
public String getDescripcion()
```

Retorna la descripcion general del hotel.

- Returns - descripcion del hotel.

```
CODIGO  
public void setDescripcion(String descripcion)
```

Asigna la descripcion general del hotel.

- Param descripcion - descripcion del hotel.

```
CODIGO  
public double getRating()
```

Retorna la calificacion promedio del hotel.

- Returns - rating del hotel.

```
CODIGO  
public void setRating(double rating)
```

Asigna la calificacion promedio del hotel.

- Param rating - calificacion del hotel.

```
CODIGO  
public String getEstado()
```

Retorna el estado actual del hotel.

- Returns - nombre del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el estado actual del hotel.

- Param estado - nombre del estado.

```
CODIGO  
public List<Integer> getImagenesIds()
```

Retorna los IDs de las imagenes asociadas al hotel.

- Returns - lista de IDs de imagenes.

```
CODIGO  
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de las imagenes asociadas al hotel.

- Param imagenesIds - lista de IDs de imagenes.

```
CODIGO  
public List<AmenidadHotelDTO> getAmenidades()
```

Retorna las amenidades asignadas al hotel.

- Returns - lista de amenidades con sus descripciones e imagenes.

```
CODIGO  
public void setAmenidades(List<AmenidadHotelDTO> amenidades)
```

Asigna las amenidades del hotel.

- Param amenidades - lista de amenidades con sus descripciones e imagenes.

```
CODIGO  
public List<TipoHabitacionResultadoDTO> getTiposHabitacion()
```

Retorna los tipos de habitacion disponibles en el hotel para la busqueda.

- Returns - lista de tipos de habitacion.

```
CODIGO  
public void setTiposHabitacion(List<TipoHabitacionResultadoDTO> tiposHabitacion)
```

Asigna los tipos de habitacion disponibles en el hotel.

- Param tiposHabitacion - lista de tipos de habitacion.

```
CODIGO  
public List<List<Integer>> getCombinacionesNumericas()
```

Retorna las combinaciones numericas de habitaciones que cubren la cantidad de personas solicitada.

- Returns - lista de combinaciones, donde cada combinacion es una lista de capacidades.

CODIGO

```
public void setCombinacionesNumericas(List<List<Integer>> combinacionesNumericas)
```

Asigna las combinaciones numericas de habitaciones.

- Param combinacionesNumericas - lista de combinaciones de capacidades.

CODIGO

```
public Map<Integer, List<TipoHabitacionResultadoDTO>> getTiposHabitacionPorCapacidad()
```

Retorna los tipos de habitacion agrupados por capacidad maxima. Util para mostrar al usuario las opciones segun el numero de personas por habitacion.

- Returns - mapa donde la clave es la capacidad y el valor es la lista de tipos de habitacion.

CODIGO

```
public void setTiposHabitacionPorCapacidad(Map<Integer, List<TipoHabitacionResultadoDTO>> tiposHabitacionPorCapacidad)
```

Asigna los tipos de habitacion agrupados por capacidad maxima.

- Param tiposHabitacionPorCapacidad - mapa de capacidad a lista de tipos de habitacion.

LoginRequestDTO

DTO con los datos necesarios para autenticar a un usuario en el sistema. El identificador puede ser el username o el correo electronico.

CODIGO

```
public String getIdentificador()
```

Retorna el identificador del usuario, que puede ser su username o correo.

- Returns - username o correo del usuario.

CODIGO

```
public void setIdentificador(String identificador)
```

Asigna el identificador del usuario, que puede ser su username o correo.

- Param identificador - username o correo del usuario.

```
CODIGO  
public String getContrasena()
```

Retorna la contraseña del usuario.

- Returns - contraseña del usuario.

```
CODIGO  
public void setContrasena(String contraseña)
```

Asigna la contraseña del usuario.

- Param contraseña - contraseña del usuario.

LoginResponseDTO

DTO con los datos retornados al cliente tras una autentificación exitosa. Incluye el mensaje de confirmación, el username y el rol del usuario autenticado.

```
CODIGO  
public class LoginResponseDTO
```

DTO con los datos retornados al cliente tras una autentificación exitosa. Incluye el mensaje de confirmación, el username y el rol del usuario autenticado.

```
CODIGO  
public LoginResponseDTO(String mensaje, String username, int rolId)
```

Constructor que inicializa la respuesta de login con todos sus datos.

- Param mensaje - mensaje de confirmación de la autentificación.
- Param username - nombre de usuario autenticado.
- Param rolId - ID del rol asignado al usuario.

```
CODIGO  
public String getMensaje()
```

Retorna el mensaje de confirmación de la autentificación.

- Returns - mensaje de confirmación.

```
CODIGO  
public String getUsername()
```

Retorna el nombre de usuario autenticado.

- Returns - username del usuario.

```
CODIGO  
public int getRolId()
```

Retorna el ID del rol asignado al usuario autenticado.

- Returns - ID del rol.

PagoAgenciaRequestDTO

DTO con los datos de facturacion necesarios para procesar el pago de una agencia.

```
CODIGO  
public String getNit()
```

Retorna el NIT de la agencia para la facturacion.

- Returns - NIT de la agencia.

```
CODIGO  
public void setNit(String nit)
```

Asigna el NIT de la agencia para la facturacion.

- Param nit - NIT de la agencia.

```
CODIGO  
public String getCodigoPostal()
```

Retorna el codigo postal de la agencia para la facturacion.

- Returns - codigo postal de la agencia.

```
CODIGO  
public void setCodigoPostal(String codigoPostal)
```

Asigna el codigo postal de la agencia para la facturacion.

- Param codigoPostal - codigo postal de la agencia.

PagoRequestDTO

DTO con los datos necesarios para procesar el pago de una reservacion. Incluye datos de facturacion y datos de tarjeta. Los datos de tarjeta solo se validan durante el proceso y nunca se almacenan en el sistema.

```
CODIGO  
public String getNit()
```

Retorna el NIT del cliente para la facturacion.

- Returns - NIT del cliente.

```
CODIGO  
public void setNit(String nit)
```

Asigna el NIT del cliente para la facturacion.

- Param nit - NIT del cliente.

```
CODIGO  
public String getCodigoPostal()
```

Retorna el codigo postal del cliente para la facturacion.

- Returns - codigo postal del cliente.

```
CODIGO  
public void setCodigoPostal(String codigoPostal)
```

Asigna el codigo postal del cliente para la facturacion.

- Param codigoPostal - codigo postal del cliente.

```
CODIGO  
public String getNumeroTarjeta()
```

Retorna el numero de tarjeta del cliente.

- Returns - numero de tarjeta.

```
CODIGO  
public void setNumeroTarjeta(String numeroTarjeta)
```

Asigna el numero de tarjeta del cliente.

- Param numeroTarjeta - numero de tarjeta.

```
CODIGO  
public String getNombreTitular()
```

Retorna el nombre del titular de la tarjeta.

- Returns - nombre del titular.

CODIGO

```
public void setNombreTitular(String nombreTitular)
```

Asigna el nombre del titular de la tarjeta.

- Param nombreTitular - nombre del titular.

CODIGO

```
public String getFechaVencimiento()
```

Retorna la fecha de vencimiento de la tarjeta en formato MM/YY.

- Returns - fecha de vencimiento.

CODIGO

```
public void setFechaVencimiento(String fechaVencimiento)
```

Asigna la fecha de vencimiento de la tarjeta en formato MM/YY.

- Param fechaVencimiento - fecha de vencimiento.

CODIGO

```
public String getCvv()
```

Retorna el CVV de la tarjeta del cliente.

- Returns - CVV de la tarjeta.

CODIGO

```
public void setCvv(String cvv)
```

Asigna el CVV de la tarjeta del cliente.

- Param cvv - CVV de la tarjeta.

CODIGO

```
public String getTokenAlianza()
```

CODIGO

```
public void setTokenAlianza(String tokenAlianza)
```

PagoResponseDTO

DTO con los datos de confirmacion retornados al cliente tras procesar un pago. Incluye los datos de la factura generada y el total cobrado.

```
CODIGO  
public class PagoResponseDTO
```

DTO con los datos de confirmacion retornados al cliente tras procesar un pago. Incluye los datos de la factura generada y el total cobrado.

```
CODIGO  
public int getFacturaId()
```

Retorna el identificador unico de la factura generada.

- Returns - ID de la factura.

```
CODIGO  
public void setFacturaId(int facturaId)
```

Asigna el identificador unico de la factura generada.

- Param facturaId - ID de la factura.

```
CODIGO  
public String getNoReservacion()
```

Retorna el numero de reservacion asociado al pago.

- Returns - numero de reservacion.

```
CODIGO  
public void setNoReservacion(String noReservacion)
```

Asigna el numero de reservacion asociado al pago.

- Param noReservacion - numero de reservacion.

```
CODIGO  
public String getEstado()
```

Retorna el estado actual de la reservacion tras el pago.

- Returns - nombre del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el estado actual de la reservacion tras el pago.

- Param estado - nombre del estado.

```
CODIGO  
public String getFecha()
```

Retorna la fecha en que se registro el pago.

- Returns - fecha del pago.

```
CODIGO  
public void setFecha(String fecha)
```

Asigna la fecha en que se registro el pago.

- Param fecha - fecha del pago.

```
CODIGO  
public String getNit()
```

Retorna el NIT del cliente usado en la facturacion.

- Returns - NIT del cliente.

```
CODIGO  
public void setNit(String nit)
```

Asigna el NIT del cliente usado en la facturacion.

- Param nit - NIT del cliente.

```
CODIGO  
public String getCodigoPostal()
```

Retorna el codigo postal del cliente usado en la facturacion.

- Returns - codigo postal del cliente.

```
CODIGO  
public void setCodigoPostal(String codigoPostal)
```

Asigna el codigo postal del cliente usado en la facturacion.

- Param codigoPostal - codigo postal del cliente.

```
CODIGO  
public double getTotal()
```

Retorna el monto total cobrado en el pago.

- Returns - total del pago.

```
CODIGO  
public void setTotal(double total)
```

Asigna el monto total cobrado en el pago.

- Param total - total del pago.

PaisDTO

DTO que representa un pais del catalogo del sistema.

```
CODIGO  
public PaisDTO(int id, String nombre)
```

Constructor para crear un pais con todos sus datos.

- Param id - identificador unico del pais.
- Param nombre - nombre del pais.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del pais.

- Returns - ID del pais.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del pais.

- Returns - nombre del pais.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del pais.

- Param id - ID del pais.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre del pais.

- Param nombre - nombre del pais.

PuedeCancelarDTO

DTO con el resultado de verificar si una reservacion puede ser cancelada. Incluye un indicador booleano y la razon en caso de que no sea posible cancelar.

```
CODIGO  
public PuedeCancelarDTO(boolean puedeCancelar, String razon)
```

Constructor que inicializa el resultado de la verificacion de cancelacion.

- Param puedeCancelar - indica si la reservacion puede cancelarse.
- Param razon - razon por la que no puede cancelarse, null si si puede.

```
CODIGO  
public boolean isPuedeCancelar()
```

Retorna si la reservacion puede ser cancelada.

- Returns - true si puede cancelarse, false en caso contrario.

```
CODIGO  
public String getRazon()
```

Retorna la razon por la que la reservacion no puede cancelarse.

- Returns - razon de la restriccion, o null si la cancelacion es posible.

ReservacionAgenciaResponseDTO

DTO con los datos de una reservacion realizada por una agencia externa. Incluye el desglose de habitaciones con sus precios calculados.

```
CODIGO  
public class ReservacionAgenciaResponseDTO
```

DTO con los datos de una reservacion realizada por una agencia externa. Incluye el desglose de habitaciones con sus precios calculados.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la reservacion.

- Returns - ID de la reservacion.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la reservacion.

- Param id - ID de la reservacion.

```
CODIGO  
public String getNoReservacion()
```

Retorna el numero de reservacion generado por el sistema.

- Returns - numero de reservacion.

```
CODIGO  
public void setNoReservacion(String noReservacion)
```

Asigna el numero de reservacion generado por el sistema.

- Param noReservacion - numero de reservacion.

```
CODIGO  
public double getTotal()
```

Retorna el monto total de la reservacion.

- Returns - total de la reservacion.

```
CODIGO  
public void setTotal(double total)
```

Asigna el monto total de la reservacion.

- Param total - total de la reservacion.

```
CODIGO  
public String getFechaCreacion()
```

Retorna la fecha en que se creo la reservacion.

- Returns - fecha de creacion.

```
CODIGO  
public void setFechaCreacion(String fechaCreacion)
```

Asigna la fecha en que se creo la reservacion.

- Param fechaCreacion - fecha de creacion.

```
CODIGO  
public String getFechaExpiracion()
```

Retorna la fecha en que expira la reservacion si no es pagada.

- Returns - fecha de expiracion.

```
CODIGO  
public void setFechaExpiracion(String fechaExpiracion)
```

Asigna la fecha en que expira la reservacion si no es pagada.

- Param fechaExpiracion - fecha de expiracion.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual de la reservacion.

- Returns - nombre del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual de la reservacion.

- Param estado - nombre del estado.

```
CODIGO  
public List<HabitacionAgenciaResponseDTO> getHabitaciones()
```

Retorna el desglose de habitaciones incluidas en la reservacion.

- Returns - lista de habitaciones con sus precios calculados.

```
CODIGO  
public void setHabitaciones(List<HabitacionAgenciaResponseDTO> habitaciones)
```

Asigna el desglose de habitaciones incluidas en la reservacion.

- Param habitaciones - lista de habitaciones con sus precios calculados.

ReservacionDetalleDTO

DTO con los datos completos de una reservacion y su detalle de habitacion. Incluye informacion del hotel, fechas, estado y IDs de imagenes asociadas.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la reservacion.

- Returns - ID de la reservacion.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la reservacion.

- Param id - ID de la reservacion.

```
CODIGO  
public String getNoReservacion()
```

Retorna el numero de reservacion generado por el sistema.

- Returns - numero de reservacion.

```
CODIGO  
public void setNoReservacion(String noReservacion)
```

Asigna el numero de reservacion generado por el sistema.

- Param noReservacion - numero de reservacion.

```
CODIGO  
public double getTotal()
```

Retorna el monto total de la reservacion.

- Returns - total de la reservacion.

```
CODIGO  
public void setTotal(double total)
```

Asigna el monto total de la reservacion.

- Param total - total de la reservacion.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual de la reservacion.

- Returns - nombre del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual de la reservacion.

- Param estado - nombre del estado.

```
CODIGO  
public String getFechaCreacion()
```

Retorna la fecha en que se creo la reservacion.

- Returns - fecha de creacion.

```
CODIGO  
public void setFechaCreacion(String fechaCreacion)
```

Asigna la fecha en que se creo la reservacion.

- Param fechaCreacion - fecha de creacion.

```
CODIGO  
public String getFechaExpiracion()
```

Retorna la fecha en que expira la reservacion si no es pagada.

- Returns - fecha de expiracion.

```
CODIGO  
public void setFechaExpiracion(String fechaExpiracion)
```

Asigna la fecha en que expira la reservacion si no es pagada.

- Param fechaExpiracion - fecha de expiracion.

```
CODIGO  
public String getFechaCancelacion()
```

Retorna la fecha en que se cancelo la reservacion.

- Returns - fecha de cancelacion, o null si no fue cancelada.

CODIGO

```
public void setFechaCancelacion(String fechaCancelacion)
```

Asigna la fecha en que se cancelo la reservacion.

- Param fechaCancelacion - fecha de cancelacion.

CODIGO

```
public String getMotivoCancelacion()
```

Retorna el motivo por el que se cancelo la reservacion.

- Returns - motivo de cancelacion, o null si no fue cancelada.

CODIGO

```
public void setMotivoCancelacion(String motivoCancelacion)
```

Asigna el motivo por el que se cancelo la reservacion.

- Param motivoCancelacion - motivo de cancelacion.

CODIGO

```
public int getDetalleId()
```

Retorna el identificador unico del detalle de la reservacion.

- Returns - ID del detalle.

CODIGO

```
public void setDetalleId(int detalleId)
```

Asigna el identificador unico del detalle de la reservacion.

- Param detalleId - ID del detalle.

CODIGO

```
public int getHabitacionId()
```

Retorna el ID de la habitacion reservada.

- Returns - ID de la habitacion.

CODIGO

```
public void setHabitacionId(int habitacionId)
```

Asigna el ID de la habitacion reservada.

- Param habitacionId - ID de la habitacion.

```
CODIGO  
public String getNumeroHabitacion()
```

Retorna el numero o identificador de la habitacion dentro del hotel.

- Returns - numero de la habitacion.

```
CODIGO  
public void setNumeroHabitacion(String numeroHabitacion)
```

Asigna el numero o identificador de la habitacion dentro del hotel.

- Param numeroHabitacion - numero de la habitacion.

```
CODIGO  
public String getFechaCheckIn()
```

Retorna la fecha de entrada en la habitacion.

- Returns - fecha de check-in.

```
CODIGO  
public void setFechaCheckIn(String fechaCheckIn)
```

Asigna la fecha de entrada en la habitacion.

- Param fechaCheckIn - fecha de check-in.

```
CODIGO  
public String getFechaCheckOut()
```

Retorna la fecha de salida de la habitacion.

- Returns - fecha de check-out.

```
CODIGO  
public void setFechaCheckOut(String fechaCheckOut)
```

Asigna la fecha de salida de la habitacion.

- Param fechaCheckOut - fecha de check-out.

```
CODIGO  
public int getCantidadPersonas()
```

Retorna la cantidad de personas que ocupan la habitacion.

- Returns - cantidad de personas.

```
CODIGO  
public void setCantidadPersonas(int cantidadPersonas)
```

Asigna la cantidad de personas que ocupan la habitacion.

- Param cantidadPersonas - cantidad de personas.

```
CODIGO  
public double getTotalDetalle()
```

Retorna el subtotal correspondiente a este detalle de habitacion.

- Returns - total del detalle.

```
CODIGO  
public void setTotalDetalle(double totalDetalle)
```

Asigna el subtotal correspondiente a este detalle de habitacion.

- Param totalDetalle - total del detalle.

```
CODIGO  
public String getDescripcionHabitacion()
```

Retorna la descripcion de la habitacion reservada.

- Returns - descripcion de la habitacion.

```
CODIGO  
public void setDescripcionHabitacion(String descripcionHabitacion)
```

Asigna la descripcion de la habitacion reservada.

- Param descripcionHabitacion - descripcion de la habitacion.

```
CODIGO  
public String getTipoHabitacion()
```

Retorna el nombre del tipo de habitacion reservada.

- Returns - tipo de habitacion.

```
CODIGO  
public void setTipoHabitacion(String tipoHabitacion)
```

Asigna el nombre del tipo de habitacion reservada.

- Param tipoHabitacion - tipo de habitacion.

```
CODIGO  
public String getTipoCama()
```

Retorna el tipo de cama de la habitacion reservada.

- Returns - tipo de cama.

```
CODIGO  
public void setTipoCama(String tipoCama)
```

Asigna el tipo de cama de la habitacion reservada.

- Param tipoCama - tipo de cama.

```
CODIGO  
public int getHotelId()
```

Retorna el ID del hotel al que pertenece la habitacion reservada.

- Returns - ID del hotel.

```
CODIGO  
public void setHotelId(int hotelId)
```

Asigna el ID del hotel al que pertenece la habitacion reservada.

- Param hotelId - ID del hotel.

```
CODIGO  
public String getNombreHotel()
```

Retorna el nombre del hotel al que pertenece la habitacion reservada.

- Returns - nombre del hotel.

```
CODIGO  
public void setNombreHotel(String nombreHotel)
```

Asigna el nombre del hotel al que pertenece la habitacion reservada.

- Param nombreHotel - nombre del hotel.

```
CODIGO  
public List<Integer> getImagenesHotelIds()
```

Retorna los IDs de imagenes asociadas al hotel.

- Returns - lista de IDs de imagenes del hotel.

```
CODIGO  
public void setImagenesHotelIds(List<Integer> imagenesHotelIds)
```

Asigna los IDs de imagenes asociadas al hotel.

- Param imagenesHotelIds - lista de IDs de imagenes del hotel.

```
CODIGO  
public List<Integer> getImagenesHabitacionIds()
```

Retorna los IDs de imagenes asociadas a la habitacion.

- Returns - lista de IDs de imagenes de la habitacion.

```
CODIGO  
public void setImagenesHabitacionIds(List<Integer> imagenesHabitacionIds)
```

Asigna los IDs de imagenes asociadas a la habitacion.

- Param imagenesHabitacionIds - lista de IDs de imagenes de la habitacion.

ReservacionRequestDTO

DTO con los datos necesarios para crear una nueva reservacion. Contiene la lista de habitaciones con sus fechas y cantidad de personas.

```
CODIGO  
public List<HabitacionReservaRequestDTO> getHabitaciones()
```

Retorna la lista de habitaciones incluidas en la solicitud de reservacion.

- Returns - lista de habitaciones a reservar.

```
CODIGO  
public void setHabitaciones(List<HabitacionReservaRequestDTO> habitaciones)
```

Asigna la lista de habitaciones incluidas en la solicitud de reservacion.

- Param habitaciones - lista de habitaciones a reservar.

ReservacionResponseDTO

DTO con los datos basicos de una reservacion retornados al cliente tras su creacion. Incluye el numero de reservacion, total, estado y fechas relevantes.

```
CODIGO  
public class ReservacionResponseDTO
```

DTO con los datos basicos de una reservacion retornados al cliente tras su creacion. Incluye el numero de reservacion, total, estado y fechas relevantes.

```
CODIGO  
public int getId()
```

Retorna el identificador unico de la reservacion.

- Returns - ID de la reservacion.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico de la reservacion.

- Param id - ID de la reservacion.

```
CODIGO  
public String getNoReservacion()
```

Retorna el numero de reservacion generado por el sistema.

- Returns - numero de reservacion.

```
CODIGO  
public void setNoReservacion(String noReservacion)
```

Asigna el numero de reservacion generado por el sistema.

- Param noReservacion - numero de reservacion.

```
CODIGO  
public double getTotal()
```

Retorna el monto total de la reservacion.

- Returns - total de la reservacion.

```
CODIGO  
public void setTotal(double total)
```

Asigna el monto total de la reservacion.

- Param total - total de la reservacion.

```
CODIGO  
public String getEstado()
```

Retorna el nombre del estado actual de la reservacion.

- Returns - nombre del estado.

```
CODIGO  
public void setEstado(String estado)
```

Asigna el nombre del estado actual de la reservacion.

- Param estado - nombre del estado.

```
CODIGO  
public String getFechaCreacion()
```

Retorna la fecha en que se creo la reservacion.

- Returns - fecha de creacion.

```
CODIGO  
public void setFechaCreacion(String fechaCreacion)
```

Asigna la fecha en que se creo la reservacion.

- Param fechaCreacion - fecha de creacion.

```
CODIGO  
public String getFechaExpiracion()
```

Retorna la fecha en que expira la reservacion si no es pagada.

- Returns - fecha de expiracion.

```
CODIGO  
public void setFechaExpiracion(String fechaExpiracion)
```

Asigna la fecha en que expira la reservacion si no es pagada.

- Param fechaExpiracion - fecha de expiracion.

SesionDTO

DTO con los datos de la sesion activa de un usuario autenticado. Se usa para verificar el estado de autenticacion y el rol del usuario.

```
CODIGO  
public int getUsuarioId()
```

Retorna el identificador unico del usuario autenticado.

- Returns - ID del usuario.

```
CODIGO  
public void setUsuarioId(int usuarioId)
```

Asigna el identificador unico del usuario autenticado.

- Param usuariold - ID del usuario.

```
CODIGO  
public String getUsername()
```

Retorna el nombre de usuario de la sesion activa.

- Returns - username del usuario.

```
CODIGO  
public void setUsername(String username)
```

Asigna el nombre de usuario de la sesion activa.

- Param username - username del usuario.

```
CODIGO  
public int getRolId()
```

Retorna el ID del rol asignado al usuario autenticado.

- Returns - ID del rol.

```
CODIGO  
public void setRolId(int rolId)
```

Asigna el ID del rol asignado al usuario autenticado.

- Param rolId - ID del rol.


```
CODIGO  
public String getRol()
```

Retorna el nombre del rol asignado al usuario autenticado.

- Returns - nombre del rol.

```
CODIGO  
public void setRol(String rol)
```

Asigna el nombre del rol asignado al usuario autenticado.

- Param rol - nombre del rol.

```
CODIGO  
public boolean isAutenticado()
```

Retorna si el usuario tiene una sesion activa autenticada.

- Returns - true si esta autenticado, false en caso contrario.

```
CODIGO  
public void setAutenticado(boolean autenticado)
```

Asigna el estado de autenticacion del usuario.

- Param autenticado - true si esta autenticado, false en caso contrario.

SubirImagenRequestDTO

DTO con los datos necesarios para subir una imagen al sistema. La imagen debe enviarse codificada en Base64.

```
CODIGO  
public String getBase64()
```

Retorna la imagen codificada en Base64.

- Returns - imagen en formato Base64.

```
CODIGO  
public void setBase64(String base64)
```

Asigna la imagen codificada en Base64.

- Param base64 - imagen en formato Base64.

TipoHabitacionResultadoDTO

DTO con los datos de un tipo de habitacion como resultado de una busqueda. Incluye las habitaciones fisicas disponibles de ese tipo para el rango de fechas solicitado.

CODIGO

```
public int getTipoHabitacionId()
```

Retorna el ID del tipo de habitacion en el catalogo.

- Returns - ID del tipo de habitacion.

CODIGO

```
public void setTipoHabitacionId(int tipoHabitacionId)
```

Asigna el ID del tipo de habitacion en el catalogo.

- Param tipoHabitacionId - ID del tipo de habitacion.

CODIGO

```
public String getTipoHabitacion()
```

Retorna el nombre del tipo de habitacion.

- Returns - nombre del tipo de habitacion.

CODIGO

```
public void setTipoHabitacion(String tipoHabitacion)
```

Asigna el nombre del tipo de habitacion.

- Param tipoHabitacion - nombre del tipo de habitacion.

CODIGO

```
public double getPrecioPorPersona()
```

Retorna el precio adicional por persona extra sobre la capacidad base.

- Returns - precio por persona adicional.

CODIGO

```
public void setPrecioPorPersona(double precioPorPersona)
```

Asigna el precio adicional por persona extra sobre la capacidad base.

- Param precioPorPersona - precio por persona adicional.

CODIGO

```
public double getPrecioPorNoche()
```

Retorna el precio base por noche del tipo de habitacion.

- Returns - precio por noche.

CODIGO

```
public void setPrecioPorNoche(double precioPorNoche)
```

Asigna el precio base por noche del tipo de habitacion.

- Param precioPorNoche - precio por noche.

CODIGO

```
public int getCapacidadMaxima()
```

Retorna la capacidad maxima de personas admitidas en este tipo de habitacion.

- Returns - capacidad maxima.

CODIGO

```
public void setCapacidadMaxima(int capacidadMaxima)
```

Asigna la capacidad maxima de personas admitidas en este tipo de habitacion.

- Param capacidadMaxima - capacidad maxima.

CODIGO

```
public String getTipoCama()
```

Retorna el tipo de cama disponible en este tipo de habitacion.

- Returns - tipo de cama.

CODIGO

```
public void setTipoCama(String tipoCama)
```

Asigna el tipo de cama disponible en este tipo de habitacion.

- Param tipoCama - tipo de cama.

CODIGO

```
public double getMetrosCuadrados()
```

Retorna la superficie en metros cuadrados de este tipo de habitacion.

- Returns - metros cuadrados.

CODIGO

```
public void setMetrosCuadrados(double metrosCuadrados)
```

Asigna la superficie en metros cuadrados de este tipo de habitacion.

- Param metrosCuadrados - metros cuadrados.

CODIGO

```
public List<Integer> getImagenesIds()
```

Retorna los IDs de imagenes asociadas a este tipo de habitacion.

- Returns - lista de IDs de imagenes.

CODIGO

```
public void setImagenesIds(List<Integer> imagenesIds)
```

Asigna los IDs de imagenes asociadas a este tipo de habitacion.

- Param imagenesIds - lista de IDs de imagenes.

CODIGO

```
public List<HabitacionResumenDTO> getHabitacionesDisponibles()
```

Retorna las habitaciones fisicas disponibles de este tipo para el rango de fechas solicitado.

- Returns - lista de habitaciones disponibles.

CODIGO

```
public void setHabitacionesDisponibles(List<HabitacionResumenDTO> habitacionesDisponibles)
```

Asigna las habitaciones fisicas disponibles de este tipo para el rango de fechas solicitado.

- Param habitacionesDisponibles - lista de habitaciones disponibles.

TokenAerolineaRequestDTO

DTO con los datos que envia la aerolinea para solicitar un token de alianza. La ciudad y pais se usan para identificar el destino del pasajero.

CODIGO

```
public String getCiudad()
```

CODIGO

```
public void setCiudad(String ciudad)
```

```
CODIGO  
public String getPais()
```

```
CODIGO  
public void setPais(String pais)
```

TokenAerolineaResponseDTO

DTO con la respuesta tras generar un token de alianza exitosamente. Contiene el token generado, la URL para redirigir al usuario y la fecha en que expira el token.

```
CODIGO  
public class TokenAerolineaResponseDTO
```

DTO con la respuesta tras generar un token de alianza exitosamente. Contiene el token generado, la URL para redirigir al usuario y la fecha en que expira el token.

```
CODIGO  
public TokenAerolineaResponseDTO(String token, String urlRedireccion, String fechaExpiracion)
```

```
CODIGO  
public String getToken()
```

```
CODIGO  
public String getUrlRedireccion()
```

```
CODIGO  
public String getFechaExpiracion()
```

TokenValidacionResponseDTO

DTO con la respuesta de validacion de un token de alianza. Contiene los datos necesarios para que el frontend configure la busqueda con descuento sin exponer logica interna.

```
CODIGO  
public class TokenValidacionResponseDTO
```

DTO con la respuesta de validacion de un token de alianza. Contiene los datos necesarios para que el frontend configure la busqueda con descuento sin exponer logica interna.

```
CODIGO  
public TokenValidacionResponseDTO(String ciudad, String pais,
```

```
CODIGO  
public String getCiudad()
```

```
CODIGO  
public String getPais()
```

```
CODIGO  
public double getPorcentajeDescuento()
```

```
CODIGO  
public String getFechaExpiracion()
```

UsuarioAdminDTO

DTO con los datos completos de un usuario para el panel de administracion. Incluye informacion personal, rol asignado y ubicacion geografica.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del usuario.

- Returns - ID del usuario.

```
CODIGO  
public String getUsername()
```

Retorna el nombre de usuario.

- Returns - username del usuario.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del usuario.

- Returns - nombre del usuario.

```
CODIGO  
public String getApellido()
```

Retorna el apellido del usuario.

- Returns - apellido del usuario.
-

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico del usuario.

- Returns - correo del usuario.
-

```
CODIGO  
public String getTelefono()
```

Retorna el numero de telefono del usuario.

- Returns - telefono del usuario.
-

```
CODIGO  
public String getFechaNacimiento()
```

Retorna la fecha de nacimiento del usuario.

- Returns - fecha de nacimiento del usuario.
-

```
CODIGO  
public int getRolId()
```

Retorna el ID del rol asignado al usuario.

- Returns - ID del rol.
-

```
CODIGO  
public String getRolNombre()
```

Retorna el nombre del rol asignado al usuario.

- Returns - nombre del rol.
-

```
CODIGO  
public String getCiudad()
```

Retorna la ciudad de residencia del usuario.

- Returns - nombre de la ciudad.
-

```
CODIGO  
public String getPais()
```

Retorna el pais de residencia del usuario.

- Returns - nombre del pais.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del usuario.

- Param id - ID del usuario.

```
CODIGO  
public void setUsername(String username)
```

Asigna el nombre de usuario.

- Param username - username del usuario.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre del usuario.

- Param nombre - nombre del usuario.

```
CODIGO  
public void setApellido(String apellido)
```

Asigna el apellido del usuario.

- Param apellido - apellido del usuario.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico del usuario.

- Param correo - correo del usuario.

```
CODIGO  
public void setTelefono(String telefono)
```

Asigna el numero de telefono del usuario.

- Param telefono - telefono del usuario.

CODIGO

```
public void setFechaNacimiento(String fechaNacimiento)
```

Asigna la fecha de nacimiento del usuario.

- Param fechaNacimiento - fecha de nacimiento del usuario.

CODIGO

```
public void setRolId(int rolId)
```

Asigna el ID del rol asignado al usuario.

- Param rolId - ID del rol.

CODIGO

```
public void setRolNombre(String rolNombre)
```

Asigna el nombre del rol asignado al usuario.

- Param rolNombre - nombre del rol.

CODIGO

```
public void setCiudad(String ciudad)
```

Asigna la ciudad de residencia del usuario.

- Param ciudad - nombre de la ciudad.

CODIGO

```
public void setPais(String pais)
```

Asigna el pais de residencia del usuario.

- Param pais - nombre del pais.

UsuarioPerfilResponseDTO

DTO con los datos completos del perfil de un usuario autenticado. Incluye informacion personal, ubicacion geografica y nacionalidades registradas.

CODIGO

```
public class UsuarioPerfilResponseDTO
```

DTO con los datos completos del perfil de un usuario autenticado. Incluye informacion personal, ubicacion geografica y nacionalidades registradas.

```
CODIGO  
public int getId()
```

Retorna el identificador unico del usuario.

- Returns - ID del usuario.

```
CODIGO  
public void setId(int id)
```

Asigna el identificador unico del usuario.

- Param id - ID del usuario.

```
CODIGO  
public String getUsername()
```

Retorna el nombre de usuario.

- Returns - username del usuario.

```
CODIGO  
public void setUsername(String username)
```

Asigna el nombre de usuario.

- Param username - username del usuario.

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico del usuario.

- Returns - correo del usuario.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico del usuario.

- Param correo - correo del usuario.

```
CODIGO  
public String getPasaporte()
```

Retorna el numero de pasaporte del usuario.

- Returns - numero de pasaporte.

```
CODIGO  
public void setPasaporte(String pasaporte)
```

Asigna el numero de pasaporte del usuario.

- Param pasaporte - numero de pasaporte.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del usuario.

- Returns - nombre del usuario.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre del usuario.

- Param nombre - nombre del usuario.

```
CODIGO  
public String getApellido()
```

Retorna el apellido del usuario.

- Returns - apellido del usuario.

```
CODIGO  
public void setApellido(String apellido)
```

Asigna el apellido del usuario.

- Param apellido - apellido del usuario.

```
CODIGO  
public String getTelefono()
```

Retorna el numero de telefono del usuario.

- Returns - telefono del usuario.

```
CODIGO  
public void setTelefono(String telefono)
```

Asigna el numero de telefono del usuario.

- Param telefono - telefono del usuario.

```
CODIGO  
public String getFechaNacimiento()
```

Retorna la fecha de nacimiento del usuario.

- Returns - fecha de nacimiento.

```
CODIGO  
public void setFechaNacimiento(String fechaNacimiento)
```

Asigna la fecha de nacimiento del usuario.

- Param fechaNacimiento - fecha de nacimiento.

```
CODIGO  
public int getRolId()
```

Retorna el ID del rol asignado al usuario.

- Returns - ID del rol.

```
CODIGO  
public void setRolId(int rolId)
```

Asigna el ID del rol asignado al usuario.

- Param rolId - ID del rol.

```
CODIGO  
public String getPais()
```

Retorna el pais de residencia del usuario.

- Returns - nombre del pais.

```
CODIGO  
public void setPais(String pais)
```

Asigna el pais de residencia del usuario.

- Param pais - nombre del pais.

```
CODIGO  
public String getCiudad()
```

Retorna la ciudad de residencia del usuario.

- Returns - nombre de la ciudad.

```
CODIGO  
public void setCiudad(String ciudad)
```

Asigna la ciudad de residencia del usuario.

- Param ciudad - nombre de la ciudad.

```
CODIGO  
public List<String> getNacionalidades()
```

Retorna la lista de nacionalidades registradas del usuario.

- Returns - lista de nacionalidades.

```
CODIGO  
public void setNacionalidades(List<String> nacionalidades)
```

Asigna la lista de nacionalidades registradas del usuario.

- Param nacionalidades - lista de nacionalidades.

UsuarioValidacionRequestDTO

DTO con los datos necesarios para registrar un nuevo usuario en el sistema. El pais y la ciudad se envian como nombres en texto. Si no existen, el servicio los crea. La fecha de nacimiento debe enviarse en formato YYYY-MM-DD.

```
CODIGO  
public String getUsername()
```

Retorna el nombre de usuario elegido para el registro.

- Returns - username del usuario.

```
CODIGO  
public void setUsername(String username)
```

Asigna el nombre de usuario elegido para el registro.

- Param username - username del usuario.

```
CODIGO  
public String getCorreo()
```

Retorna el correo electronico del usuario.

- Returns - correo del usuario.

```
CODIGO  
public void setCorreo(String correo)
```

Asigna el correo electronico del usuario.

- Param correo - correo del usuario.

```
CODIGO  
public String getContrasena()
```

Retorna la contraseña del usuario.

- Returns - contraseña del usuario.

```
CODIGO  
public void setContrasena(String contrasena)
```

Asigna la contraseña del usuario.

- Param contrasena - contraseña del usuario.

```
CODIGO  
public String getPasaporte()
```

Retorna el numero de pasaporte del usuario.

- Returns - numero de pasaporte.

```
CODIGO  
public void setPasaporte(String pasaporte)
```

Asigna el numero de pasaporte del usuario.

- Param pasaporte - numero de pasaporte.

```
CODIGO  
public String getNombre()
```

Retorna el nombre del usuario.

- Returns - nombre del usuario.

```
CODIGO  
public void setNombre(String nombre)
```

Asigna el nombre del usuario.

- Param nombre - nombre del usuario.

```
CODIGO  
public String getApellido()
```

Retorna el apellido del usuario.

- Returns - apellido del usuario.

```
CODIGO  
public void setApellido(String apellido)
```

Asigna el apellido del usuario.

- Param apellido - apellido del usuario.

```
CODIGO  
public String getTelefono()
```

Retorna el numero de telefono del usuario.

- Returns - telefono del usuario.

```
CODIGO  
public void setTelefono(String telefono)
```

Asigna el numero de telefono del usuario.

- Param telefono - telefono del usuario.

```
CODIGO  
public String getFechaNacimiento()
```

Retorna la fecha de nacimiento del usuario en formato YYYY-MM-DD.

- Returns - fecha de nacimiento.

```
CODIGO  
public void setFechaNacimiento(String fechaNacimiento)
```

Asigna la fecha de nacimiento del usuario en formato YYYY-MM-DD.

- Param fechaNacimiento - fecha de nacimiento.

```
CODIGO  
public String getPais()
```

Retorna el nombre del pais de residencia del usuario.

- Returns - nombre del pais.

```
CODIGO  
public void setPais(String pais)
```

Asigna el nombre del pais de residencia del usuario.

- Param pais - nombre del pais.

```
CODIGO  
public String getCiudad()
```

Retorna el nombre de la ciudad de residencia del usuario.

- Returns - nombre de la ciudad.

```
CODIGO  
public void setCiudad(String ciudad)
```

Asigna el nombre de la ciudad de residencia del usuario.

- Param ciudad - nombre de la ciudad.

```
CODIGO  
public List<String> getNacionalidades()
```

Retorna la lista de nacionalidades del usuario como nombres en texto.

- Returns - lista de nacionalidades.

```
CODIGO  
public void setNacionalidades(List<String> nacionalidades)
```

Asigna la lista de nacionalidades del usuario como nombres en texto.

- Param nacionalidades - lista de nacionalidades.

UsuarioValidacionResponseDTO

DTO con el resultado de validar si un username, correo o pasaporte ya existen en el sistema. Se usa antes del registro para informar al cliente de duplicados.

```
CODIGO  
public class UsuarioValidacionResponseDTO
```


DTO con el resultado de validar si un username, correo o pasaporte ya existen en el sistema. Se usa antes del registro para informar al cliente de duplicados.

CODIGO

```
public UsuarioValidacionResponseDTO(boolean usernameExiste, boolean correoExiste, boolean pasaporteExiste)
```

Constructor que inicializa el resultado de la validacion de duplicados.

- Param usernameExiste - true si el username ya esta registrado en el sistema.
- Param correoExiste - true si el correo ya esta registrado en el sistema.
- Param pasaporteExiste - true si el pasaporte ya esta registrado en el sistema.

CODIGO

```
public boolean isUsernameExiste()
```

Retorna si el username ya existe en el sistema.

- Returns - true si el username esta registrado, false en caso contrario.

CODIGO

```
public boolean isCorreoExiste()
```

Retorna si el correo ya existe en el sistema.

- Returns - true si el correo esta registrado, false en caso contrario.

CODIGO

```
public boolean isPasaporteExiste()
```

Retorna si el pasaporte ya existe en el sistema.

- Returns - true si el pasaporte esta registrado, false en caso contrario.

UsuarioWebserviceLibreDTO

DTO que representa un usuario webservice disponible para ser asignado a una entidad. Un usuario se considera "libre" cuando no tiene ni agencia ni aerolinea vinculadas.

CODIGO

```
public int getId()
```

Retorna el ID del usuario webservice.

- Returns - ID del usuario.

CODIGO

```
public String getUsername()
```

Retorna el nombre de usuario.

- Returns - username del usuario.
-

```
CODIGO  
public void setId(int id)
```

Asigna el ID del usuario webservice.

- Param id - ID del usuario.
-

```
CODIGO  
public void setUsername(String username)
```

Asigna el nombre de usuario.

- Param username - username del usuario.
-

Helpers

AerolineaAuthMiddleware

Middleware de autenticacion para rutas protegidas de aerolineas aliadas. Valida el token enviado en el header X-Aerolinea-Token e inyecta la identidad de la aerolinea en el contexto de la peticion.

CODIGO

```
public static boolean verificar(Context ctx)
```

Verifica el token de aerolinea en el header de la peticion. Si el token es valido, inyecta el ID, nombre y URL de la aerolinea como atributos del contexto para que los controllers puedan usarlos. a ninguna aerolinea activa, escribiendo ya la respuesta 401.

- Param ctx - contexto de la peticion HTTP de Javalin.
- Returns - true si el token es valido; false si falta o no corresponde

AgenciaAuthMiddleware

Middleware de autenticacion para rutas protegidas de agencias externas. Valida el token enviado en el header X-Agencia-Token e inyecta la identidad de la agencia en el contexto de la peticion.

CODIGO

```
public static boolean verificar(Context ctx)
```

Verifica el token de agencia en el header de la peticion. Si el token es valido, inyecta el ID, nombre y URL de la agencia como atributos del contexto para que los controllers puedan usarlos. Debe llamarse al inicio de cada ruta protegida de agencia. false si el token falta o no corresponde a ninguna agencia, en cuyo caso ya se escribe la respuesta 401 en el contexto.

- Param ctx - contexto de la peticion HTTP de Javalin.
- Returns - true si el token es valido y la agencia fue identificada;

AuthMiddleware

Middleware de autenticacion global para la aplicacion Javalin. Intercepta todas las peticiones entrantes y valida el token JWT antes de permitir el acceso a rutas protegidas.

CODIGO

```
public static void registrar(Javalin app)
```

Registra el middleware de autenticacion en la instancia de Javalin. Se ejecuta antes de cada peticion. Si la ruta es publica o pertenece al prefijo /agencia/, la deja pasar sin validar. De lo contrario, exige un cookie auth_token valido y extrae los claims del usuario para inyectarlos en el contexto. (misma situacion para /aerolinea/)

- Param app - instancia de Javalin donde se registra el middleware.

CamposDuplicadosException

Excepcion lanzada cuando uno o mas campos unicos ya existen en el sistema durante el registro de un usuario (username, correo o pasaporte duplicados). Incluye el detalle de cuales campos especificamente estan duplicados.

CODIGO

```
public CamposDuplicadosException(UsuarioValidacionResponseDTO detalle)
```

Crea la excepcion con el detalle de los campos duplicados encontrados.

- Param detalle - DTO con flags indicando cuales campos ya existen en el sistema.

CODIGO

```
public UsuarioValidacionResponseDTO getDetalle()
```

Retorna el detalle de los campos duplicados que causaron la excepcion.

- Returns - DTO con el estado de validacion de username, correo y pasaporte.

CombinacionHelper

Helper para calcular combinaciones de habitaciones que cubren un numero de personas. Genera particiones numericas usando las capacidades disponibles en stock, limitando a un maximo de 3 habitaciones por combinacion.

CODIGO

```
public static List<List<Integer>> calcular(int n, Map<Integer, Integer> stockPorCapacidad)
```

Genera las mejores combinaciones de habitaciones para alojar a N personas. Usa las capacidades disponibles en stock para formar particiones validas, descarta la combinacion de una sola habitacion exacta (ya cubierta por busqueda individual) y retorna las 3 mejores priorizando capacidades mas grandes.

- Param n - cantidad de personas a alojar.
- Param stockPorCapacidad - mapa de capacidad -> cantidad de habitaciones disponibles de esa capacidad.
- Returns - lista de hasta 3 combinaciones, donde cada combinacion es una lista de capacidades que suman N.

CredencialesInvalidasException

Excepcion lanzada cuando las credenciales proporcionadas por el usuario no coinciden con las registradas en el sistema durante el inicio de sesion o el cambio de contrasena.

EmailHelper

Helper para el envio de correos electronicos via SMTP usando Gmail. Configura la sesion con autenticacion y TLS, y expone un metodo estatico para enviar mensajes en formato HTML.

CODIGO

```
public static void enviar(String destinatario, String asunto, String cuerpoHtml)
```

Envia un correo electronico en formato HTML al destinatario indicado.

- Param destinatario - direccion de correo del receptor.
- Param asunto - asunto del mensaje.
- Param cuerpoHtml - contenido del mensaje en formato HTML.
- Throws RuntimeException - si ocurre un error durante el envio del correo.

JwtHelper

Helper para la generacion y validacion de tokens JWT. Maneja la firma con HMAC-SHA, la extraccion de claims y la verificacion de tokens en las peticiones autenticadas. El secreto se lee de la variable de entorno JWT_SECRET y tiene una duracion de 8 horas por token.

CODIGO

```
public static String generarToken(int usuarioId, String username, int rolId)
```

Genera un token JWT firmado con los datos de identidad del usuario. El token incluye el ID como subject y username y rolId como claims adicionales.

- Param usuariold - ID del usuario autenticado.
- Param username - nombre de usuario.
- Param rolId - ID del rol del usuario.
- Returns - token JWT compacto y firmado.

CODIGO

```
public static Claims verificarToken(String token)
```

Verifica la firma del token y retorna los claims si es valido.

- Param token - token JWT a verificar.
- Returns - claims extraidos del payload del token.
- Throws JwtException - si el token esta malformado, expirado o la firma no coincide.

CODIGO

```
public static int getUsuarioId(Claims claims)
```

Extrae el ID del usuario desde los claims del token.

- Param claims - claims obtenidos de un token verificado.
- Returns - ID del usuario como entero.

CODIGO

```
public static String getUsername(Claims claims)
```

Extrae el username desde los claims del token.

- Param claims - claims obtenidos de un token verificado.
- Returns - nombre de usuario.

CODIGO

```
public static int getRolId(Claims claims)
```

Extrae el ID de rol desde los claims del token.

- Param claims - claims obtenidos de un token verificado.
- Returns - ID del rol del usuario.

CODIGO

```
public static boolean esValido(String token)
```

Valida un token JWT sin lanzar excepciones. Util para verificar rapidamente si un token es usable antes de procesarlo.

- Param token - token JWT a validar.
- Returns - true si el token es valido y no ha expirado; false en cualquier otro caso.

PasswordHelper

Helper para el hasheo y verificacion de contraseñas usando BCrypt. Utiliza un factor de costo de 12 rondas para el salt, lo que ofrece un balance adecuado entre seguridad y rendimiento.

CODIGO

```
public static String hashear(String passwordPlano)
```

Genera el hash BCrypt de una contraseña en texto plano.

- Param passwordPlano - contraseña en texto plano a hashear.
- Returns - hash BCrypt listo para almacenar en base de datos.

CODIGO

```
public static boolean verificar(String passwordPlano, String passwordHasheado)
```

Verifica si una contraseña en texto plano coincide con su hash BCrypt.

- Param passwordPlano - contraseña en texto plano ingresada por el usuario.
- Param passwordHasheado - hash BCrypt almacenado en base de datos.
- Returns - true si la contraseña coincide con el hash; false en caso contrario.

PdfHelper

Helper para la generacion de comprobantes y facturas de reservacion en PDF. Construye el documento con iText 7, incluyendo header y footer por pagina, tabla de habitaciones con subtotales, datos fiscales opcionales y bloque de terminos y condiciones.

CODIGO

```
public static byte[] generarPdfReservacion(List<ReservacionDetalleDTO> detalles, Object[] factura)
```

Genera el PDF completo del comprobante o factura de una reservacion. Si se proporciona el arreglo de factura el documento incluye datos fiscales; de lo contrario se emite como comprobante simple.

- Param detalles - lista de DTOs con el detalle de cada habitacion.
- Param factura - arreglo [id, fechaEmision, nit, codigoPostal], o null si es comprobante.
- Returns - bytes del PDF generado listos para enviar como respuesta HTTP.

CODIGO

```
public void handleEvent(Event event)
```

CODIGO

```
public void handleEvent(Event event)
```

TarjetaHelper

Helper para la validacion de datos de tarjeta de credito o debito. Verifica formato del numero, nombre del titular, fecha de vencimiento y CVV antes de procesar cualquier pago.

CODIGO

```
public static void validar(PagoRequestDTO request)
```

Valida los datos de tarjeta contenidos en el request de pago. Verifica que el numero tenga 16 digitos, el titular no este vacio, la fecha de vencimiento sea valida y no este expirada, y el CVV tenga 3 o 4 digitos. o la tarjeta esta vencida.

- Param request - DTO con los datos de la tarjeta a validar.
- Throws IllegalArgumentException - si cualquiera de los campos no cumple el formato esperado

TokenHelper

Helper para la generacion de tokens de autenticacion seguros. Usa SecureRandom para garantizar aleatoriedad criptografica.

CODIGO

```
public static String generarTokenHash()
```

Genera un token aleatorio de 64 caracteres en formato hexadecimal. Se usa como token de acceso para agencias externas u otros casos donde se necesite un identificador unico e impredecible.

- Returns - string hexadecimal de 64 caracteres generado con SecureRandom.
-

INFRAESTRUCTURA

Config

ServerConfig

Configuracion del servidor Javalin. Lee el puerto desde la variable de entorno PORT (default 7000) y habilita CORS para el frontend en localhost:5173.

CODIGO

```
public static Javalin createServer()
```

Crea y arranca el servidor Javalin con CORS configurado. Permite credenciales desde <http://localhost:5173>.

- Returns - instancia de Javalin ya iniciada en el puerto configurado.

Data

DataAccessException

Excepcion personalizada para errores de acceso a datos. Envuelve cualquier excepcion de base de datos en una *RuntimeException*.

```
CODIGO  
public DataAccessException(String message, Throwable cause)
```

Crea una nueva excepcion de acceso a datos.

- Param message - descripcion del error ocurrido.
- Param cause - excepcion original que causo el fallo.

DatabaseManager

```
CODIGO  
public static <T> List<T> executeQuery()
```

```
CODIGO  
public static int executeUpdate()
```

```
CODIGO  
public static int executeInsertReturnId()
```

DatabaseTest

Clase de prueba para verificar la conexion a la base de datos Oracle. Solo debe usarse en desarrollo para validar credenciales y URL JDBC.

```
CODIGO  
public static void testConnection()
```

Intenta abrir una conexion JDBC con Oracle usando credenciales fijas. Imprime un mensaje de exito o error segun el resultado.

ResultSetMapper

Interfaz funcional para mapear una fila de un *ResultSet* a un objeto de tipo *T*. Se usa como parametro en *DatabaseManager.executeQuery* para convertir resultados SQL a entidades Java.

CODIGO

```
public interface ResultSetMapper<T>
```

Interfaz funcional para mapear una fila de un ResultSet a un objeto de tipo T. Se usa como parametro en DatabaseManager.executeQuery para convertir resultados SQL a entidades Java.
